STAR RX72N - rx_frame Library

1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy 1
1.1 Directories . . . 1

2 File Index 3
2.1 File List . . . 3

3 Directory Documentation 5
3.1 libs/rx_frame/inc Directory Reference . . . 5
3.2 libs Directory Reference . . . 5
3.3 libs/rx_frame Directory Reference . . . 6
3.4 libs/rx_frame/src Directory Reference . . . 6

4 File Documentation 7
4.1 libs/rx_frame/inc/rx_frame.h File Reference . . . 7
4.1.1 Detailed Description . . . 9
4.1.1.1 Overview . . . 9
4.1.1.2 Frame Format . . . 10
4.1.1.3 Frame Size Calculations . . . 10
4.1.1.4 Protocol Flow . . . 11
4.1.1.5 CRC-32 Details . . . 11
4.1.1.6 Thread Safety . . . 12
4.1.1.7 Memory Usage . . . 12
4.1.1.8 Performance Characteristics . . . 12
4.1.1.9 NASA Power of 10 Compliance . . . 12
4.1.1.10 SOLID Principles . . . 13
4.1.1.11 Hardware Requirements . . . 13
4.1.1.12 Integration Example . . . 13
4.1.1.13 References . . . 14
4.1.2 Data Structure Documentation . . . 14
4.1.2.1 struct rx_frame_header_t . . . 14
4.1.2.2 Memory Layout . . . 14
4.1.2.3 Wire Format vs In-Memory Format . . . 15
4.1.2.4 struct rx_frame_t . . . 17
4.1.2.5 Memory Layout . . . 17
4.1.2.6 Usage Patterns . . . 17
4.1.2.7 struct rx_frame_encoder_t . . . 19
4.1.2.8 Lifecycle . . . 20
4.1.2.9 struct rx_frame_decoder_t . . . 21
4.1.2.10 Lifecycle . . . 21
4.1.3 Enumeration Type Documentation . . . 22
4.1.3.1 rx_byte_order_constants_t . . . 22
4.1.3.2 rx_frame_constants_t . . . 23
4.1.3.3 Frame Layout . . . 23
4.1.3.4 Size Calculations . . . 23

4.1.3.5 rx_frame_flags_t . . . 25
4.1.3.6 Flag Descriptions . . . 26
4.1.3.7 Flag Combinations . . . 26
4.1.3.8 rx_frame_type_t . . . 27
4.1.3.9 Communication Pattern . . . 28
4.1.3.10 Type Descriptions . . . 28
4.1.3.11 rx_le16_byte_idx_t . . . 32
4.1.3.12 rx_le32_byte_idx_t . . . 32
4.1.3.13 rx_le32_shift_t . . . 32
4.1.4 Function Documentation . . . 33
4.1.4.1 rx_frame_create_ack() . . . 33
4.1.4.2 rx_frame_create_nack() . . . 34
4.1.4.3 rx_frame_create_ping() . . . 34
4.1.4.4 rx_frame_create_pong() . . . 36
4.1.4.5 rx_frame_create_reset() . . . 37
4.1.4.6 rx_frame_create_reset_ack() . . . 39
4.1.4.7 rx_frame_decode() . . . 40
4.1.4.8 rx_frame_decode_with_resync() . . . 41
4.1.4.9 rx_frame_decoder_deinit() . . . 45
4.1.4.10 rx_frame_decoder_init() . . . 46
4.1.4.11 rx_frame_encode() . . . 47
4.1.4.12 rx_frame_encoded_size() . . . 48
4.1.4.13 rx_frame_encoder_deinit() . . . 49
4.1.4.14 rx_frame_encoder_init() . . . 49
4.1.4.15 rx_frame_read_le16() . . . 50
4.1.4.16 rx_frame_read_le32() . . . 51
4.1.4.17 rx_frame_type_valid() . . . 51
4.1.4.18 rx_frame_write_le16() . . . 52
4.2 rx_frame.h . . . 52
4.3 libs/rx_frame/src/rx_frame.c File Reference . . . 67
4.3.1 Detailed Description . . . 69
4.3.1.1 Overview . . . 69
4.3.1.2 Module Architecture . . . 69
4.3.1.3 Encoding Process . . . 69
4.3.1.4 Decoding Process . . . 70
4.3.1.5 Memory Usage . . . 70
4.3.1.6 Performance (RX72N @ 240 MHz) . . . 70
4.3.1.7 NASA Power of 10 Compliance . . . 70
4.3.1.8 SOLID Principles . . . 71
4.3.1.9 Test Coverage . . . 71
4.3.2 Enumeration Type Documentation . . . 72
4.3.2.1 byte_shift_t . . . 72
4.3.2.2 frame_crc_init_t . . . 73

4.3.2.3 frame_offset_t . . . 73
4.3.2.4 frame_resync_discarded_t . . . 74
4.3.2.5 init_state_t . . . 75
4.3.3 Function Documentation . . . 75
4.3.3.1 internal_decode_header() . . . 75
4.3.3.2 internal_find_sync_offset() . . . 76
4.3.3.3 internal_read_le32() . . . 78
4.3.3.4 internal_verify_crc() . . . 79
4.3.3.5 internal_write_le32() . . . 80
4.3.3.6 rx_frame_create_ack() . . . 80
4.3.3.7 rx_frame_create_nack() . . . 81
4.3.3.8 rx_frame_create_ping() . . . 81
4.3.3.9 rx_frame_create_pong() . . . 82
4.3.3.10 rx_frame_create_reset() . . . 84
4.3.3.11 rx_frame_create_reset_ack() . . . 85
4.3.3.12 rx_frame_decode() . . . 86
4.3.3.13 rx_frame_decode_with_resync() . . . 87
4.3.3.14 rx_frame_decoder_deinit() . . . 89
4.3.3.15 rx_frame_decoder_init() . . . 89
4.3.3.16 rx_frame_encode() . . . 90
4.3.3.17 rx_frame_encoder_deinit() . . . 91
4.3.3.18 rx_frame_encoder_init() . . . 92
4.4 rx_frame.c . . . 92

Chapter 1

Directory Hierarchy

1.1 Directories

inc 5
rx_frame.h 7
libs 5
rx_frame 6
inc 5
rx_frame.h 7
src 6
rx_frame.c 67
rx_frame 6
inc 5
rx_frame.h 7
src 6
rx_frame.c 67
src 6
rx_frame.c 67

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

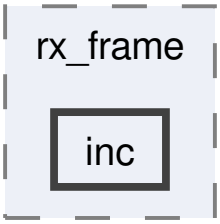
libs/rx_frame/inc/rx_frame.h
Frame Layer Protocol for SPI Communication 7
libs/rx_frame/src/rx_frame.c
Frame Layer Protocol Implementation 67

Chapter 3

Directory Documentation

3.1 libs/rx˙frame/inc Directory Reference

Directory dependency graph for inc:

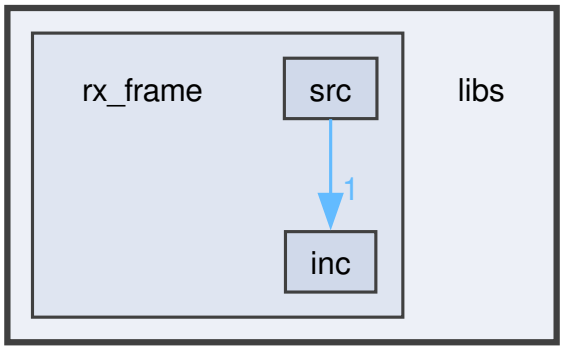


Files

- file rx_frame.h
  Frame Layer Protocol for SPI Communication.

3.2 libs Directory Reference

Directory dependency graph for libs:

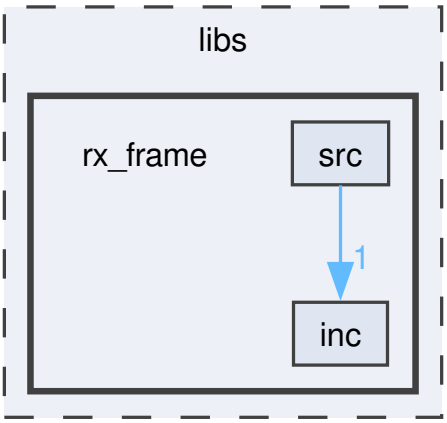


Directories

- directory rx_frame

3.3 libs/rx˙frame Directory Reference

Directory dependency graph for rx_frame:

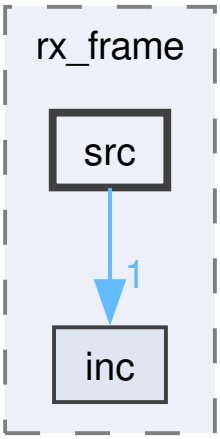


Directories

- directory inc
- directory src

3.4 libs/rx˙frame/src Directory Reference

Directory dependency graph for src:



Files

- file rx_frame.c

  Frame Layer Protocol Implementation.

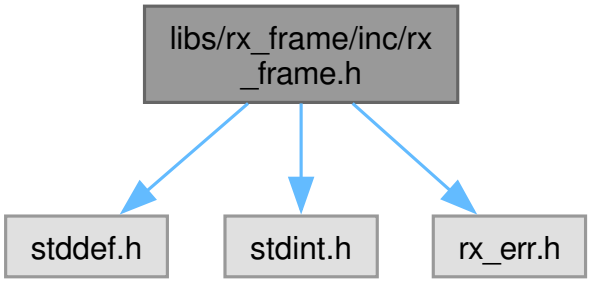
Chapter 4

File Documentation

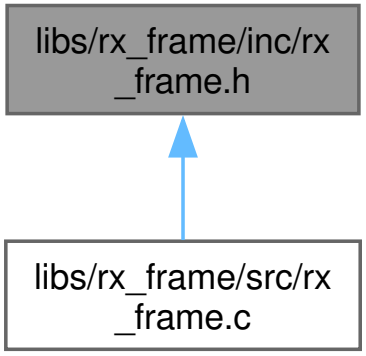
4.1 libs/rx˙frame/inc/rx˙frame.h File Reference

Frame Layer Protocol for SPI Communication.

#include <stddef.h>
#include <stdint.h>
#include "rx_err.h"
Include dependency graph for rx_frame.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct rx_frame_header_t

  Frame header containing control fields.
- struct rx_frame_t

  Complete frame structure containing header, payload, and CRC.
- struct rx_frame_encoder_t

  Frame encoder handle.
- struct rx_frame_decoder_t

  Frame decoder handle.

Enumerations

- enum rx_frame_constants_t : uint16_t {
  k_frame_sync_word = 0x55AA , k_frame_sync_size = 2 , k_frame_seq_size = 2 , k_frame_len_size = 2 ,
  k_frame_type_size = 1 , k_frame_flags_size = 1 , k_frame_crc_size = 4 , k_frame_header_size = 6 ,
  k_frame_min_payload = 0 , k_frame_max_payload = 1024 , k_frame_min_size = 12 , k_frame_max_size = 1036 ,
  k_frame_max_scan_bytes }

  Frame structure constants for SPI protocol.
- enum rx_frame_type_t : uint8_t {
  k_frame_type_ping = 0x00 , k_frame_type_pong = 0x01 , k_frame_type_command = 0x10 , k_frame_type_response = 0x11 ,
  k_frame_type_ack = 0x12 , k_frame_type_nack = 0x13 , k_frame_type_log_message = 0x20 , k_frame_type_reset_ack = 0xFE ,
  k_frame_type_reset = 0xFF }

  Frame types for SPI protocol (matches Go FrameType).
- enum rx_frame_flags_t : uint8_t {
  k_frame_flag_none = 0x00 , k_frame_flag_requires_ack = 0x01 , k_frame_flag_retransmit = 0x02 , k_frame_flag_priority = 0x04 ,
  k_frame_flag_fec_enabled = 0x08 , k_frame_flag_soft_nack = 0x10 }

  Frame control flags (matches Go FrameFlags).
- enum rx_le16_byte_idx_t : uint8_t { k_le16_byte_low = 0 , k_le16_byte_high = 1 }

  Byte indices for 16-bit little-endian serialization.
- enum rx_byte_order_constants_t : uint8_t { k_rx_le16_high_shift = (8) , k_rx_byte_mask = (0xFFU) }

  Byte manipulation constants for endianness conversions.
- enum rx_le32_byte_idx_t : uint8_t { k_le32_byte_0 = 0 , k_le32_byte_1 = 1 , k_le32_byte_2 = 2 , k_le32_byte_3 = 3 }

  Byte indices for 32-bit little-endian serialization.
- enum rx_le32_shift_t : uint8_t { k_rx_le32_shift_0 = 0 , k_rx_le32_shift_1 = 8 , k_rx_le32_shift_2 = 16 , k_rx_le32_shift_3 = 24 }

  Bit shift amounts for extracting bytes from 32-bit values.

Functions

- rx_err_t rx_frame_encoder_init (rx_frame_encoder_t *enc)

  Initialize frame encoder.
- rx_err_t rx_frame_encoder_deinit (rx_frame_encoder_t *enc)

  Deinitialize frame encoder.

- rx_err_t rx_frame_encode (const rx_frame_encoder_t *enc, const rx_frame_t *frame, uint8_t *output, uint32_t *output_len)

  Encode frame to wire format.
- static uint32_t rx_frame_encoded_size (uint32_t payload_len)

  Calculate encoded frame size.
- static uint16_t rx_frame_read_le16 (const uint8_t *buf)

  Read uint16 from little-endian buffer.
- static void rx_frame_write_le16 (uint8_t *buf, uint16_t val)

  Write uint16 to little-endian buffer.
- static uint32_t rx_frame_read_le32 (const uint8_t *buf)

  Read uint32 from little-endian buffer.
- rx_err_t rx_frame_decoder_init (rx_frame_decoder_t *dec)

  Initialize frame decoder.
- rx_err_t rx_frame_decoder_deinit (rx_frame_decoder_t *dec)

  Deinitialize frame decoder.
- rx_err_t rx_frame_decode (const rx_frame_decoder_t *dec, const uint8_t *data, uint32_t data_len, rx_frame_t *frame)

  Decode wire format to frame.
- rx_err_t rx_frame_decode_with_resync (const rx_frame_decoder_t *dec, const uint8_t *data, uint32_t data_len, rx_frame_t *frame, uint32_t *bytes_discarded_out)

  Decode wire format to frame with bounded sync-word resynchronization.
- rx_err_t rx_frame_create_ack (rx_frame_t *frame, uint16_t sequence)

  Create ACK frame for given sequence.
- rx_err_t rx_frame_create_nack (rx_frame_t *frame, uint16_t sequence, uint8_t flags)

  Create NACK frame for given sequence.
- rx_err_t rx_frame_create_ping (rx_frame_t *frame, uint16_t sequence, const uint8_t *payload, uint32_t payload_len)

  Create PING frame for keepalive/heartbeat.
- rx_err_t rx_frame_create_pong (rx_frame_t *frame, uint16_t sequence, const uint8_t *payload, uint32_t payload_len)

  Create PONG frame (response to PING).
- rx_err_t rx_frame_create_reset (rx_frame_t *frame, uint16_t sequence)

  Create RESET frame to reset communication state.
- rx_err_t rx_frame_create_reset_ack (rx_frame_t *frame, uint16_t sequence)

  Create RESET_ACK frame (response to RESET).
- static bool rx_frame_type_valid (uint8_t type)

  Check if frame type is valid.

4.1.1 Detailed Description

Frame Layer Protocol for SPI Communication.

4.1.1.1 Overview

This module implements the frame-level protocol for reliable SPI communication between the Raspberry Pi 5 gateway and the RX72N motor controller. It provides frame encoding, decoding, and CRC-32 integrity verification.

Key Features:

- Bit-exact compatible with star-gateway/internal/frame/ (Go implementation)
- IEEE 802.3 CRC-32 for data integrity
- Little-endian byte order for ALL multi-byte fields (SYNC, SEQ, LEN, CRC)
- Support for ACK/NACK flow control
- Optional forward error correction (FEC) flag

4.1.1.2 Frame Format

```
+---------+---------+---------+---------+---------+-------------+---------+
| SYNC    | SEQ     | LEN     | TYPE    | FLAGS   | PAYLOAD     | CRC-32  |
| 2 bytes | 2 bytes | 2 bytes | 1 byte  | 1 byte  | 0-1024 B    | 4 bytes |
| (LE)    | (LE)    | (LE)    |         |         |             | (LE)    |
+---------+---------+---------+---------+---------+-------------+---------+
```

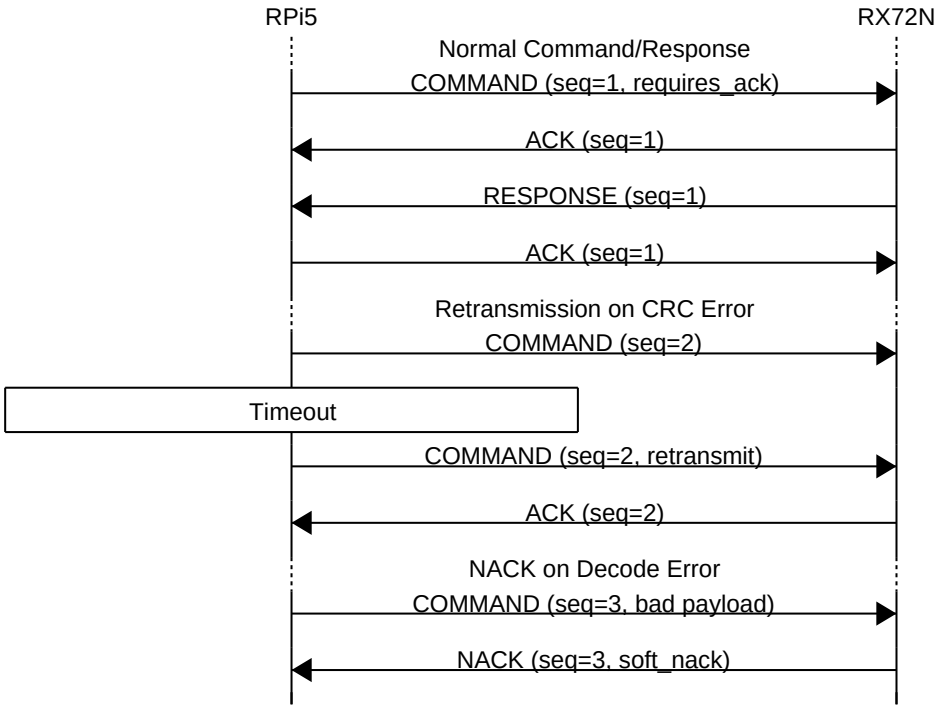
Field Descriptions:

Field	Size	Byte Order	Description
SYNC	2	Little-endian	Synchronization marker (0x55AA, wire: 0xAA 0x55)
SEQ	2	Little-endian	Sequence number for ordering/retransmit
LEN	2	Little-endian	Payload length in bytes (0-1024)
TYPE	1	N/A	Frame type (command, response, ack, nack)
FLAGS	1	N/A	Control flags (requires_ack, retransmit, etc.)
PAYLOAD	0-1024	N/A	Encoded protobuf message or empty
CRC-32	4	Little-endian	IEEE 802.3 CRC over SYNC+header+payload

4.1.1.3 Frame Size Calculations

- Minimum frame: 12 bytes (SYNC + header + CRC, no payload)
- Maximum frame: 1036 bytes (SYNC + header + 1024 payload + CRC)
- Overhead: 12 bytes fixed per frame

4.1.1.4 Protocol Flow



4.1.1.5 CRC-32 Details

The CRC-32 is computed using the IEEE 802.3 polynomial (0x04C11DB7) over the entire frame excluding the CRC field itself:

$$\text{CRC} = \text{CRC32\_IEEE}(\text{SYNC} \| \text{Header} \| \text{Payload})$$

CRC Coverage:

- SYNC word (2 bytes)
- SEQ (2 bytes)
- LEN (2 bytes)
- TYPE (1 byte)
- FLAGS (1 byte)
- PAYLOAD (0-1024 bytes)

Why Little-Endian CRC? The CRC-32 is stored in little-endian format (LSB first) to match the IEEE 802.3 bit transmission order. This ensures compatibility with Go's crc32.ChecksumIEEE() function.

4.1.1.6 Thread Safety

Component	Thread Safe	Notes
Encoder	[PASS] Yes	Stateless after init
Decoder	[PASS] Yes	Stateless after init
Utility functions	[PASS] Yes	Pure functions

4.1.1.7 Memory Usage

Component	Size	Notes
rx_frame_t	1034 bytes	Header + 1024-byte payload + CRC
rx_frame_encoder_t	1 byte	Initialization flag only
rx_frame_decoder_t	1 byte	Initialization flag only
Encoded frame buffer	1036 bytes	Caller-provided

4.1.1.8 Performance Characteristics

Measured on RX72N @ 240 MHz, -O2 optimization:

Operation	Empty Payload	64B Payload	255B Payload
Encode	~5 us	~15 us	~50 us
Decode	~5 us	~15 us	~50 us
CRC calc	~1 us	~6 us	~25 us

4.1.1.9 NASA Power of 10 Compliance

Rule	Status	Notes
1. Simple control flow	[PASS] Pass	No goto/setjmp/recursion
2. Fixed loop bounds	[PASS] Pass	Bounded by k_frame_max_payload
3. No dynamic memory	[PASS] Pass	All buffers caller-provided
4. Short functions	[PASS] Pass	All functions < 50 lines
5. Assertions	[PASS] Pass	NULL checks, state validation
6. Small scope	[PASS] Pass	Variables at minimal scope
7. Check returns	[PASS] Pass	All returns propagated
8. Limited preprocessor	[PASS] Pass	Only include guards
9. Restrict pointers	[PASS] Pass	No function pointers
10. Compiler warnings	[PASS] Pass	-Wall -Wextra -Werror

4.1.1.10 SOLID Principles

Principle	Implementation
Single Responsibility	Frame layer only - no protobuf knowledge
Open/Closed	New frame types via enum, no code changes
Liskov Substitution	Encoder/decoder handles are interchangeable
Interface Segregation	Separate encoder/decoder APIs
Dependency Inversion	Depends on rx_crc.h abstraction

4.1.1.11 Hardware Requirements

Resource	Requirement
SPI	10 Mbps, Full-duplex
DMA	Optional, improves throughput
CRC	Uses rx_crc module (HW or SW)

4.1.1.12 Integration Example

```
#include "rx_frame.h"
#include "rx_spi_comm.h"

// Transmit a command frame
void send_motor_command(uint8_t* protobuf_payload, uint32_t payload_len)
{
    rx_frame_encoder_t encoder;
    rx_frame_t frame;
    uint8_t wire_buffer[1036];
    uint32_t wire_len;

    // Initialize encoder
    rx_frame_encoder_init(&encoder);

    // Build frame
    frame.header.sequence = get_next_sequence();
    frame.header.length = payload_len;
    frame.header.type = k_frame_type_command;
    frame.header.flags = k_frame_flag_requires_ack;
    memcpy(frame.payload, protobuf_payload, payload_len);

    // Encode to wire format
    rx_err_t err = rx_frame_encode(&encoder, &frame, wire_buffer, &wire_len);
    if (err != k_rx_ok) {
        rx_log_error("FRAME", "Encode failed: %d", err);
        return;
    }

    // Send via SPI
    spi_transmit(wire_buffer, wire_len);
}

// Receive and decode a frame
rx_err_t receive_frame(uint8_t* wire_data, uint32_t wire_len, rx_frame_t* frame)
{
    rx_frame_decoder_t decoder;

    // Initialize decoder
    rx_frame_decoder_init(&decoder);

    // Decode frame (validates SYNC, parses header, verifies CRC)
    rx_err_t err = rx_frame_decode(&decoder, wire_data, wire_len, frame);
    if (err == k_rx_err_crc_mismatch) {
        rx_log_error("FRAME", "CRC mismatch, requesting retransmit");
        send_nack(frame->header.sequence);
        return err;
    }

    return err;
}
```

4.1.1.13 References

- star-gateway/internal/frame/: Go reference implementation
- docs/sections/01_nanopb_protocol.tex: Protocol specification
- IEEE 802.3: CRC-32 polynomial definition

See also

rx_frame.c Implementation file
rx_crc.h CRC-32 computation
rx_spi_comm.h SPI transport layer
rx_nanopb.h Protobuf encoding for payloads

Author

Locked, Inc.

Date

2026-01-29

Copyright

Since

Version 1.0.0

Definition in file rx_frame.h.

4.1.2 Data Structure Documentation

4.1.2.1 struct rx˙frame˙header˙t

Frame header containing control fields.

The frame header contains all control information needed to process a frame. All 16-bit fields are stored in host byte order within this structure; conversion to/from little-endian wire format is handled by the encoder/decoder.

4.1.2.2 Memory Layout

Offset	Size	Field	Type	Description
0	2	sequence	uint16↩ _t	Sequence number
2	2	length	uint16↩ _t	Payload length
4	1	type	uint8↩ _t	Frame type

Offset	Size	Field	Type	Description
5	1	flags	uint8↩ _t	Control flags
Total	6			No padding required

4.1.2.3 Wire Format vs In-Memory Format

Wire Format (little-endian):

- sequence: LSB at lower address
- length: LSB at lower address

In-Memory (this struct):

- sequence: Host byte order (little-endian on RX72N, matches wire)
- length: Host byte order

Usage Example:

```
rx_frame_header_t header = {
    .sequence = 0x1234,             // Will be sent as 0x34, 0x12 on wire (LE)
    .length = 64,                   // 64-byte payload
    .type = k_frame_type_command,
    .flags = k_frame_flag_requires_ack,
};
```

Invariant

length <= k_frame_max_payload (1024)

type is valid rx_frame_type_t value

See also

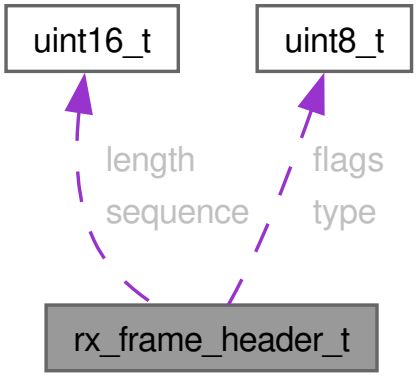
rx_frame_t Complete frame structure

Since

Version 1.0.0

Definition at line 651 of file rx_frame.h.

Collaboration diagram for rx_frame_header_t:



Data Fields

uint8_t	flags	Control flags. Bitfield of rx_frame_flags_t values controlling frame handling. Multiple flags can be combined with bitwise OR. Valid Values: Combination of rx_frame_flags_t values
uint16_t	length	Payload length in bytes. Length of the payload field (not including header or CRC). Zero for ACK/↩ NACK frames. Wire Format: Little-endian Valid Range: 0-1024 (k_frame_max_payload) Invariant length <= k_frame_max_payload
uint16_t	sequence	Frame sequence number. 16-bit sequence number for ordering, duplicate detection, and ACK/NACK correlation. Wraps around at 65535. Wire Format: Little-endian Valid Range: 0-65535

uint8_t	type	Frame type identifier. Identifies the message type (command, response, ack, nack). Valid Values: rx_frame_type_t enumeration See also rx_frame_type_valid() Validate type

4.1.2.4 struct rx˙frame˙t

Complete frame structure containing header, payload, and CRC.

This structure represents a complete decoded frame ready for processing. The encoder serializes this to wire format; the decoder populates it from wire format.

4.1.2.5 Memory Layout

Offset	Size	Field	Description
0	6	header	Frame header (rx_frame_header_t)
6	1024	payload	Payload buffer (max size)
1030	4	crc	CRC-32 checksum (for decoded frames)
Total	1034		

Note: Structure size is 1034 bytes. The actual payload length is indicated by header.length, not the buffer size.

4.1.2.6 Usage Patterns

Encoding (TX):

1. Populate header fields (sequence, length, type, flags)
2. Copy payload data to payload[] array
3. Call rx_frame_encode() - CRC is computed automatically

Decoding (RX):

1. Call rx_frame_decode() with wire data
2. Access header fields to determine frame type
3. Read payload[0..header.length-1] for message content
4. CRC field contains the received checksum (already verified)

Usage Example - Building a Frame:

```
rx_frame_t frame;
memset(&frame, 0, sizeof(frame));

// Set header
frame.header.sequence = 1;
frame.header.length = protobuf_len;
frame.header.type = k_frame_type_command;
frame.header.flags = k_frame_flag_requires_ack;

// Copy payload
memcpy(frame.payload, protobuf_data, protobuf_len);

// Encode (CRC computed internally)
uint8_t wire[1036];
uint32_t wire_len;
rx_frame_encode(&encoder, &frame, wire, &wire_len);
```

Usage Example - Processing Decoded Frame:

```
rx_frame_t frame;
rx_err_t err = rx_frame_decode(&decoder, wire_data, wire_len, &frame);

if (err == k_rx_ok) {
    // Frame valid, CRC verified
    if (frame.header.type == k_frame_type_command) {
        // Process payload[0..header.length-1]
        handle_command(frame.payload, frame.header.length);
    }
}
```

Invariant

header.length <= k_frame_max_payload

Note

The crc field is populated during decode, not used during encode

Warning

Payload data beyond header.length is undefined

See also

rx_frame_header_t Header structure

rx_frame_encode() Encode frame to wire format

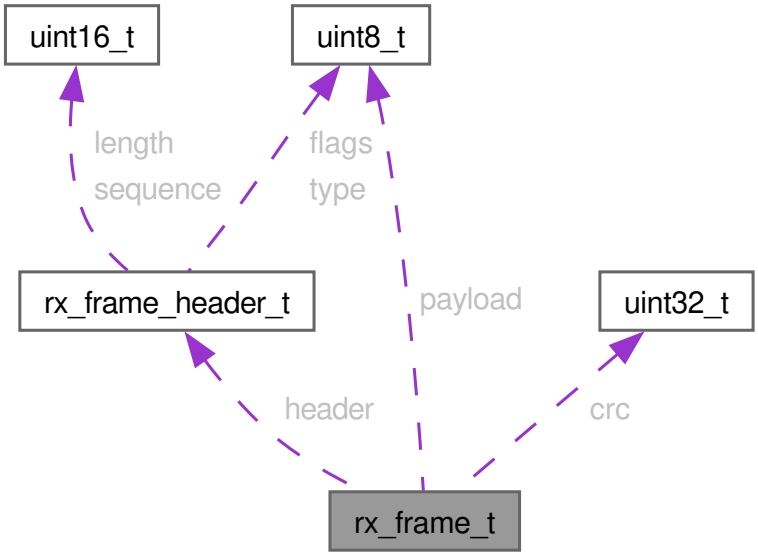
rx_frame_decode() Decode frame from wire format

Since

Version 1.0.0

Definition at line 769 of file rx_frame.h.

Collaboration diagram for rx_frame_t:



Data Fields

uint32_t	crc	CRC-32 checksum (IEEE 802.3). For received frames: Contains the CRC read from wire (verified). For transmitted frames: Not used (computed by encoder). Wire Format: Little-endian
rx_frame_header_t	header	Frame header containing control fields. Populated by caller (encode) or decoder (decode).
uint8_t	payload↩ [k_frame_max_payload]	Payload data buffer (maximum 1024 bytes). For transmit: Contains data to send (length in header.length). For receive:↩ Contains decoded payload (length in header.length). Note Only payload[0..header.length-1] is valid data.

4.1.2.7 struct rx˙frame˙encoder˙t

Frame encoder handle.

Lightweight handle structure for frame encoding operations. Must be initialized before use with rx_frame_encoder_init().

4.1.2.8 Lifecycle

1. Declare: rx_frame_encoder_t enc;
2. Initialize: rx_frame_encoder_init(&enc);
3. Use: rx_frame_encode(&enc, &frame, output, &output_len);
4. Deinitialize: rx_frame_encoder_deinit(&enc); (optional)

Memory Layout:

Offset	Size	Field	Description
0	1	initialized	Initialization flag
Total	1		No padding

Invariant

initialized is 0 or 1

Note

Structure is stateless after init; thread-safe for concurrent use

See also

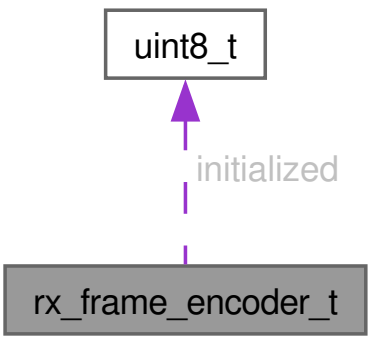
rx_frame_encoder_init() Initialize encoder
rx_frame_encode() Encode frame

Since

Version 1.0.0

Definition at line 823 of file rx_frame.h.

Collaboration diagram for rx_frame_encoder_t:



Data Fields

uint8_t	initialized	Initialization state flag. Non-zero (1) if initialized and ready for use. Valid Values: 0 (uninitialized), 1 (initialized)

4.1.2.9 struct rx˙frame˙decoder˙t

Frame decoder handle.

Lightweight handle structure for frame decoding operations. Must be initialized before use with rx_frame_decoder_init().

4.1.2.10 Lifecycle

1. Declare: rx_frame_decoder_t dec;
2. Initialize: rx_frame_decoder_init(&dec);
3. Use: rx_frame_decode(&dec, data, len, &frame);
4. Deinitialize: rx_frame_decoder_deinit(&dec); (optional)

Memory Layout:

Offset	Size	Field	Description
0	1	initialized	Initialization flag
Total	1		No padding

Invariant

initialized is 0 or 1

Note

Structure is stateless after init; thread-safe for concurrent use

See also

rx_frame_decoder_init() Initialize decoder

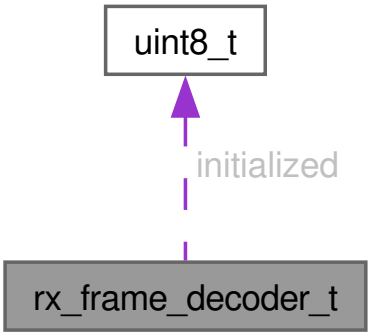
rx_frame_decode() Decode frame

Since

Version 1.0.0

Definition at line 860 of file rx_frame.h.

Collaboration diagram for rx_frame_decoder_t:



Data Fields

uint8_t	initialized	Initialization state flag. Non-zero (1) if initialized and ready for use. Valid Values: 0 (uninitialized), 1 (initialized)

4.1.3 Enumeration Type Documentation

4.1.3.1 rx_byte_order_constants_t

enum rx_byte_order_constants_t : uint8_t

Byte manipulation constants for endianness conversions.

Enumerator

k_rx_le16_high_shift	Bit shift for high byte in 16-bit value
k_rx_byte_mask	Mask to extract one byte

Definition at line 967 of file rx_frame.h.

```
00967             : uint8_t {
00968     k_rx_le16_high_shift = (8),     /**< Bit shift for high byte in 16-bit value */
00969     k_rx_byte_mask       = (0xFFU), /**< Mask to extract one byte */
00970 } rx_byte_order_constants_t;
```

4.1.3.2 rx_frame_constants_t

enum rx_frame_constants_t : uint16_t

Frame structure constants for SPI protocol.

These constants define the frame format parameters that must exactly match the Go implementation in star-gateway/internal/frame/. Any mismatch will cause protocol interoperability failures.

4.1.3.3 Frame Layout

```
Offset:  0       2       4       6    7    8          8+len
Field:   SYNC    SEQ     LEN     TYPE FLAGS PAYLOAD... CRC
Size:    2       2       2       1    1    0-1024     4
```

4.1.3.4 Size Calculations

Frame Type	Formula	Bytes
Minimum (no payload)	SYNC + Header + CRC	12
With N-byte payload	12 + N	12 - 1036
Maximum (1KB payload)	SYNC + Header + 1024 + CRC	1036

Usage Example:

```
// Validate frame size
if (wire_len < k_frame_min_size || wire_len > k_frame_max_size) {
    return k_rx_err_invalid_size;
}

// Check sync word
uint16_t sync = rx_frame_read_le16(wire_data);
if (sync != k_frame_sync_word) {
    return k_rx_err_protocol_error;
}

// Calculate expected size from payload length
uint16_t payload_len = rx_frame_read_le16(&wire_data[4]);
uint32_t expected = rx_frame_encoded_size(payload_len);
```

Warning

These values MUST match star-gateway/internal/frame/constants.go

Note

Using uint16_t underlying type to accommodate k_frame_max_payload (1024)

See also

star-gateway/internal/frame/ Go reference implementation

Since

Version 1.0.0

Enumerator

k_frame_sync_word	Frame synchronization marker (0x55AA). The sync word marks the start of a valid frame. Receivers scan for this pattern to establish frame boundaries. The value 0x55AA was chosen for: • Alternating bit pattern aids clock recovery • Unique enough to avoid false positives in random data • Matches common SPI debug patterns Wire Format: Little-endian (0xAA first, then 0x55)
k_frame_sync_size	SYNC field size in bytes (2). Two-byte sync marker at frame start.
k_frame_seq_size	SEQ field size in bytes (2). 16-bit sequence number for ordering and retransmit tracking.
k_frame_len_size	LEN field size in bytes (2). 16-bit payload length (max 1024).
k_frame_type_size	TYPE field size in bytes (1). Single byte for frame type (command, response, ack, nack).
k_frame_flags_size	FLAGS field size in bytes (1). Single byte for control flags (requires_ack, retransmit, etc.).
k_frame_crc_size	CRC-32 field size in bytes (4). IEEE 802.3 CRC-32 checksum.
k_frame_header_size	Header size excluding SYNC (6 bytes). SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1) = 6 bytes.
k_frame_min_payload	Minimum payload length (0 bytes). ACK and NACK frames have empty payloads.
k_frame_max_payload	Maximum payload length (1024 bytes). Limited by SPI DMA buffer size and protobuf message limits.
k_frame_min_size	Minimum frame size (12 bytes). SYNC(2) + Header(6) + CRC(4) = 12 bytes (no payload).
k_frame_max_size	Maximum frame size (1036 bytes). SYNC(2) + Header(6) + Payload(1024) + CRC(4) = 1036 bytes.
k_frame_max_scan_bytes	Maximum bytes to scan forward during sync recovery (1036). When the sync word is not found at offset 0, the decoder scans forward up to this many bytes looking for the sync pattern. The upper bound equals k_frame_max_size so recovery never reads more data than one complete maximum-size frame. This ensures bounded execution time (NASA Rule 2) and prevents unbounded stream consumption on permanently corrupt inputs. The scan discards all bytes before the discovered sync word and reattempts decoding from that position.

Definition at line 288 of file rx_frame.h.

```
00288             : uint16_t {
00289   k_frame_sync_word = 0x55AA, /**< @brief Frame synchronization marker (0x55AA)
00290                               * @details
00291                               * The sync word marks the start of a valid frame. Receivers scan for this
00292                               * pattern to establish frame boundaries. The value 0x55AA was chosen for:
00293                               * - Alternating bit pattern aids clock recovery
00294                               * - Unique enough to avoid false positives in random data
00295                               * - Matches common SPI debug patterns
```

```
00296                          * @par Wire Format: Little-endian (0xAA first, then 0x55)
00297                          */
00298
00299   k_frame_sync_size = 2, /**< @brief SYNC field size in bytes (2)
00300                           * @details Two-byte sync marker at frame start.
00301                           */
00302
00303   k_frame_seq_size = 2, /**< @brief SEQ field size in bytes (2)
00304                          * @details 16-bit sequence number for ordering and retransmit tracking.
00305                          */
00306
00307   k_frame_len_size = 2, /**< @brief LEN field size in bytes (2)
00308                          * @details 16-bit payload length (max 1024).
00309                          */
00310
00311   k_frame_type_size = 1, /**< @brief TYPE field size in bytes (1)
00312                           * @details Single byte for frame type (command, response, ack, nack).
00313                           */
00314
00315   k_frame_flags_size = 1, /**< @brief FLAGS field size in bytes (1)
00316                            * @details Single byte for control flags (requires_ack, retransmit, etc.).
00317                            */
00318
00319   k_frame_crc_size = 4, /**< @brief CRC-32 field size in bytes (4)
00320                          * @details IEEE 802.3 CRC-32 checksum.
00321                          */
00322
00323   k_frame_header_size = 6, /**< @brief Header size excluding SYNC (6 bytes)
00324                             * @details SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1) = 6 bytes.
00325                             */
00326
00327   k_frame_min_payload = 0, /**< @brief Minimum payload length (0 bytes)
00328                             * @details ACK and NACK frames have empty payloads.
00329                             */
00330
00331   k_frame_max_payload = 1024, /**< @brief Maximum payload length (1024 bytes)
00332                                * @details Limited by SPI DMA buffer size and protobuf message limits.
00333                                */
00334
00335   k_frame_min_size = 12, /**< @brief Minimum frame size (12 bytes)
00336                           * @details SYNC(2) + Header(6) + CRC(4) = 12 bytes (no payload).
00337                           */
00338
00339   k_frame_max_size = 1036, /**< @brief Maximum frame size (1036 bytes)
00340                             * @details SYNC(2) + Header(6) + Payload(1024) + CRC(4) = 1036 bytes.
00341                             */
00342
00343   k_frame_max_scan_bytes =
00344     k_frame_max_size, /**< @brief Maximum bytes to scan forward during sync recovery (1036)
00345                        * @details
00346                        * When the sync word is not found at offset 0, the decoder scans
00347                        * forward up to this many bytes looking for the sync pattern. The
00348                        * upper bound equals k_frame_max_size so recovery never reads more
00349                        * data than one complete maximum-size frame. This ensures bounded
00350                        * execution time (NASA Rule 2) and prevents unbounded stream
00351                        * consumption on permanently corrupt inputs.
00352                        *
00353                        * The scan discards all bytes before the discovered sync word and
00354                        * reattempts decoding from that position.
00355                        */
00356 } rx_frame_constants_t;
```

4.1.3.5 rx˙frame˙flags˙t

enum rx_frame_flags_t : uint8_t

Frame control flags (matches Go FrameFlags).

Bit flags that modify frame handling behavior. Multiple flags can be combined using bitwise OR.

4.1.3.6 Flag Descriptions

Flag	Bit	Purpose
REQUIRES_ACK	0	Sender expects ACK/NACK response
RETRANSMIT	1	This is a retransmission of previous frame
PRIORITY	2	Process before normal-priority frames
FEC_ENABLED	3	Payload uses forward error correction
SOFT_NACK	4	NACK with recoverable error info

4.1.3.7 Flag Combinations

```
// Typical command frame
frame.header.flags = k_frame_flag_requires_ack;

// Retransmission after timeout
frame.header.flags = k_frame_flag_requires_ack | k_frame_flag_retransmit;

// High-priority motor command
frame.header.flags = k_frame_flag_requires_ack | k_frame_flag_priority;

// FEC-protected data transfer
frame.header.flags = k_frame_flag_fec_enabled;

// NACK with error details
nack_frame.header.flags = k_frame_flag_soft_nack;
```

Usage Example:

```
// Check if ACK required
if (frame->header.flags & k_frame_flag_requires_ack) {
    send_ack(frame->header.sequence);
}

// Check if retransmit
if (frame->header.flags & k_frame_flag_retransmit) {
    rx_log_warn("FRAME", "Received retransmit seq=%u",
                frame->header.sequence);
    stats.retransmit_count++;
}
```

Warning

Values MUST match star-gateway/internal/frame/flags.go

Since

Version 1.0.0

Enumerator

k_frame_flag_none	No flags set (0x00). Default for frames that don't require special handling.
k_frame_flag_requires_ack	Frame requires acknowledgment (0x01, bit 0). Sender expects an ACK or NACK response. If no response within timeout, sender should retransmit with RETRANSMIT flag.
k_frame_flag_retransmit	Retransmission of previous frame (0x02, bit 1). Indicates this frame is a retry after timeout or NACK. Receiver should check for duplicate sequence numbers.

k_frame_flag_priority	High-priority frame (0x04, bit 2). Process before normal-priority frames in queue. Used for emergency stop and critical motor commands.
k_frame_flag_fec_enabled	Forward error correction enabled (0x08, bit 3). Payload is FEC-encoded (convolutional rate 1/2). FEC is enabled by default on the HARQ path (e.g., SPI transport with rx_spi_link). This flag indicates FEC encoding is present and receiver should apply Viterbi decoding before processing payload. The flag is ignored for transports that do not use HARQ (e.g., USB CDC, which relies on built-in hardware CRC and retry).
k_frame_flag_soft_nack	NACK with soft error information (0x10, bit 4). Used in NACK frames to indicate recoverable error. May include confidence bits or partial decode info.

Definition at line 562 of file rx_frame.h.

```
00562           : uint8_t {
00563   k_frame_flag_none = 0x00, /**< @brief No flags set (0x00)
00564                             * @details Default for frames that don't require special handling.
00565                             */
00566
00567   k_frame_flag_requires_ack = 0x01, /**< @brief Frame requires acknowledgment (0x01, bit 0)
00568                                      * @details
00569                                      * Sender expects an ACK or NACK response. If no response within
00570                                      * timeout, sender should retransmit with RETRANSMIT flag.
00571                                      */
00572
00573   k_frame_flag_retransmit = 0x02, /**< @brief Retransmission of previous frame (0x02, bit 1)
00574                                    * @details
00575                                    * Indicates this frame is a retry after timeout or NACK.
00576                                    * Receiver should check for duplicate sequence numbers.
00577                                    */
00578
00579   k_frame_flag_priority = 0x04, /**< @brief High-priority frame (0x04, bit 2)
00580                                  * @details
00581                                  * Process before normal-priority frames in queue.
00582                                  * Used for emergency stop and critical motor commands.
00583                                  */
00584
00585   k_frame_flag_fec_enabled = 0x08, /**< @brief Forward error correction enabled (0x08, bit 3)
00586                                     * @details
00587                                     * Payload is FEC-encoded (convolutional rate 1/2). FEC is enabled by default
00588                                     * on the HARQ path (e.g., SPI transport with rx_spi_link). This flag indicates
00589                                     * FEC encoding is present and receiver should apply Viterbi decoding before
00590                                     * processing payload. The flag is ignored for transports that do not use HARQ
00591                                     * (e.g., USB CDC, which relies on built-in hardware CRC and retry).
00592                                     */
00593
00594   k_frame_flag_soft_nack = 0x10, /**< @brief NACK with soft error information (0x10, bit 4)
00595                                   * @details
00596                                   * Used in NACK frames to indicate recoverable error.
00597                                   * May include confidence bits or partial decode info.
00598                                   */
00599 } rx_frame_flags_t;
```

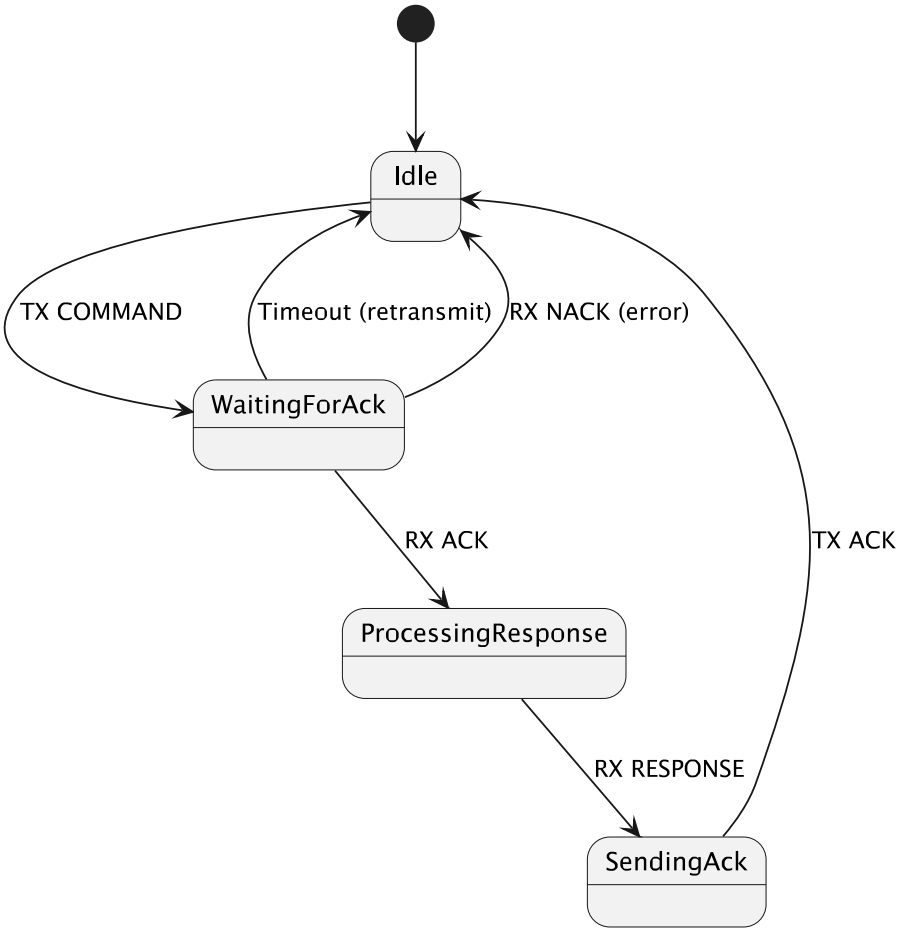
4.1.3.8 rx˙frame˙type˙t

enum rx_frame_type_t : uint8_t

Frame types for SPI protocol (matches Go FrameType).

Defines the type of message contained in a frame. The frame type determines the expected payload content and required response handling.

4.1.3.9 Communication Pattern



4.1.3.10 Type Descriptions

Type	Direction	Payload	Response
COMMAND	RPi5 -> RX72N	Protobuf	ACK, then RESPONSE
RESPONSE	RX72N -> RPi5	Protobuf	ACK
ACK	Either	None	None
NACK	Either	None (flags indicate reason)	Retransmit

Usage Example:

```
switch (frame->header.type) {
    case k_frame_type_command:
        // Process command, send ACK then RESPONSE
        send_ack(frame->header.sequence);
        process_command(frame->payload, frame->header.length);
        send_response(frame->header.sequence);
        break;
    case k_frame_type_ack:
        // Mark pending command as acknowledged
        mark_acked(frame->header.sequence);
        break;
    case k_frame_type_nack:
        // Retransmit the failed frame
        retransmit(frame->header.sequence);
```

```
        break;
    default:
        rx_log_error("FRAME", "Unknown type: %u", frame->header.type);
}
```

Warning

Values MUST match star-gateway/internal/frame/types.go

See also

rx_frame_type_valid() Validate frame type

Since

Version 1.0.0

Enumerator

k_frame_type_ping	Ping request (0x00). Keepalive/heartbeat message sent to check connection status. Receiver responds with PONG echoing the payload. Payload: 4-byte counter (little-endian uint32) or empty
k_frame_type_pong	Pong response (0x01). Response to PING request, confirms connection is alive. Echoes the PING counter payload for validation. Payload: Echo of PING counter or empty
k_frame_type_command	Command from controller to peripheral (0x10). Contains a protobuf-encoded command message from RPi5 to RX72N. On SPI: typically requires ACK and generates a RESPONSE frame. On USB CDC: no ACK expected (hardware reliability). Typical Payloads: MotorCommand, TelemetryRequest, ConfigWrite
k_frame_type_response	Response from peripheral to controller (0x11). Contains a protobuf-encoded response message from RX72N to RPi5. Sent after processing a COMMAND. Typical Payloads: MotorStatus, TelemetryData, ConfigAck

k_frame_type_ack	Positive acknowledgment (0x12, SPI only). Confirms successful reception and CRC validation of a frame. Empty payload; sequence number matches the acknowledged frame. Used only on SPI transport (USB CDC relies on hardware reliability). Payload: None (length = 0)
k_frame_type_nack	Negative acknowledgment (0x13, SPI only). Indicates a problem with received frame (CRC error, decode failure). Empty payload; flags indicate error type. Sender should retransmit. Used only on SPI transport (UART relies on CRC-32 + bounded resync). Payload: None (length = 0) Flags: May include k_frame_flag_soft_nack
k_frame_type_log_message	Firmware log message (0x20). Carries ASCII log text emitted by the RX72N runtime (rx_log_* macros). Payload is a raw UTF-8 / 7-bit ASCII byte sequence – NOT null-terminated and may span multiple lines. The gateway extracts the payload and writes it to its own log stream (stderr / log file). Multiplexing logs as framed messages prevents log bytes from colliding with the frame sync word on the shared UART-to-USB bridge byte stream. Payload: 1..1024 bytes of log text (no null terminator) Flags: Usually k_frame_flag_none
k_frame_type_reset_ack	Reset acknowledgment (0xFE). Confirms reset operation completed successfully. Sequence number matches the reset request. Payload: None (length = 0)
k_frame_type_reset	Reset request (0xFF). Request to reset communication state (sequence numbers, buffers, etc.). Receiver responds with RESET_↩ ACK. Payload: None (length = 0)

Definition at line 415 of file rx_frame.h.

```
00415           : uint8_t {
00416   /**
00417     * @brief Ping request (0x00)
00418     * @details
00419     * Keepalive/heartbeat message sent to check connection status.
00420     * Receiver responds with PONG echoing the payload.
```

```
00421     * @par Payload: 4-byte counter (little-endian uint32) or empty
00422     */
00423   k_frame_type_ping = 0x00,
00424
00425   /**
00426     * @brief Pong response (0x01)
00427     * @details
00428     * Response to PING request, confirms connection is alive.
00429     * Echoes the PING counter payload for validation.
00430     * @par Payload: Echo of PING counter or empty
00431     */
00432   k_frame_type_pong = 0x01,
00433
00434   /**
00435     * @brief Command from controller to peripheral (0x10)
00436     * @details
00437     * Contains a protobuf-encoded command message from RPi5 to RX72N.
00438     * On SPI: typically requires ACK and generates a RESPONSE frame.
00439     * On USB CDC: no ACK expected (hardware reliability).
00440     * @par Typical Payloads: MotorCommand, TelemetryRequest, ConfigWrite
00441     */
00442   k_frame_type_command = 0x10,
00443
00444   /**
00445     * @brief Response from peripheral to controller (0x11)
00446     * @details
00447     * Contains a protobuf-encoded response message from RX72N to RPi5.
00448     * Sent after processing a COMMAND.
00449     * @par Typical Payloads: MotorStatus, TelemetryData, ConfigAck
00450     */
00451   k_frame_type_response = 0x11,
00452
00453   /**
00454     * @brief Positive acknowledgment (0x12, SPI only)
00455     * @details
00456     * Confirms successful reception and CRC validation of a frame.
00457     * Empty payload; sequence number matches the acknowledged frame.
00458     * Used only on SPI transport (USB CDC relies on hardware reliability).
00459     * @par Payload: None (length = 0)
00460     */
00461   k_frame_type_ack = 0x12,
00462
00463   /**
00464     * @brief Negative acknowledgment (0x13, SPI only)
00465     * @details
00466     * Indicates a problem with received frame (CRC error, decode failure).
00467     * Empty payload; flags indicate error type. Sender should retransmit.
00468     * Used only on SPI transport (UART relies on CRC-32 + bounded resync).
00469     * @par Payload: None (length = 0)
00470     * @par Flags: May include k_frame_flag_soft_nack
00471     */
00472   k_frame_type_nack = 0x13,
00473
00474   /**
00475     * @brief Firmware log message (0x20)
00476     * @details
00477     * Carries ASCII log text emitted by the RX72N runtime (rx_log_* macros).
00478     * Payload is a raw UTF-8 / 7-bit ASCII byte sequence -- NOT null-terminated
00479     * and may span multiple lines. The gateway extracts the payload and writes
00480     * it to its own log stream (stderr / log file). Multiplexing logs as framed
00481     * messages prevents log bytes from colliding with the frame sync word on the
00482     * shared UART-to-USB bridge byte stream.
00483     * @par Payload: 1..1024 bytes of log text (no null terminator)
00484     * @par Flags: Usually k_frame_flag_none
00485     */
00486   k_frame_type_log_message = 0x20,
00487
00488   /**
00489     * @brief Reset acknowledgment (0xFE)
00490     * @details
00491     * Confirms reset operation completed successfully.
00492     * Sequence number matches the reset request.
00493     * @par Payload: None (length = 0)
00494     */
00495   k_frame_type_reset_ack = 0xFE,
00496
00497   /**
00498     * @brief Reset request (0xFF)
00499     * @details
00500     * Request to reset communication state (sequence numbers, buffers, etc.).
00501     * Receiver responds with RESET_ACK.
00502     * @par Payload: None (length = 0)
00503     */
00504   k_frame_type_reset = 0xFF,
00505 } rx_frame_type_t;
```

4.1.3.11 rx_le16_byte_idx_t

enum rx_le16_byte_idx_t : uint8_t

Byte indices for 16-bit little-endian serialization.

In little-endian format, the low byte (LSB) is stored at index 0 and the high byte (MSB) is stored at index 1.

Enumerator

k_le16_byte_low	Low byte (LSB) at index 0
k_le16_byte_high	High byte (MSB) at index 1

Definition at line 959 of file rx_frame.h.

```
00959             : uint8_t {
00960   k_le16_byte_low  = 0, /**< Low byte (LSB) at index 0 */
00961   k_le16_byte_high = 1, /**< High byte (MSB) at index 1 */
00962 } rx_le16_byte_idx_t;
```

4.1.3.12 rx_le32_byte_idx_t

enum rx_le32_byte_idx_t : uint8_t

Byte indices for 32-bit little-endian serialization.

Enumerator

k_le32_byte_0	Byte 0 (LSB)
k_le32_byte_1	Byte 1
k_le32_byte_2	Byte 2
k_le32_byte_3	Byte 3 (MSB)

Definition at line 975 of file rx_frame.h.

```
00975             : uint8_t {
00976   k_le32_byte_0 = 0, /**< Byte 0 (LSB) */
00977   k_le32_byte_1 = 1, /**< Byte 1 */
00978   k_le32_byte_2 = 2, /**< Byte 2 */
00979   k_le32_byte_3 = 3, /**< Byte 3 (MSB) */
00980 } rx_le32_byte_idx_t;
```

4.1.3.13 rx_le32_shift_t

enum rx_le32_shift_t : uint8_t

Bit shift amounts for extracting bytes from 32-bit values.

Enumerator

k_rx_le32_shift_0	Shift 0 bits for byte 0
k_rx_le32_shift_1	Shift 8 bits for byte 1
k_rx_le32_shift_2	Shift 16 bits for byte 2
k_rx_le32_shift_3	Shift 24 bits for byte 3

Definition at line 985 of file rx_frame.h.

```
00985             : uint8_t {
00986   k_rx_le32_shift_0 = 0,  /**< Shift 0 bits for byte 0 */
00987   k_rx_le32_shift_1 = 8,  /**< Shift 8 bits for byte 1 */
00988   k_rx_le32_shift_2 = 16, /**< Shift 16 bits for byte 2 */
00989   k_rx_le32_shift_3 = 24, /**< Shift 24 bits for byte 3 */
00990 } rx_le32_shift_t;
```

4.1.4 Function Documentation

4.1.4.1 rx˙frame˙create˙ack()

```
rx_err_t rx_frame_create_ack (
                rx_frame_t * frame,
                const uint16_t sequence)   [nodiscard]
```

Create ACK frame for given sequence.

Parameters

out	frame	Frame to initialize
in	sequence	Sequence number to acknowledge

Returns

k_rx_ok on success

Create ACK frame for given sequence.

Constructs a frame with type=ACK, empty payload, and the specified sequence number. The ACK response frames are used by the receiver to confirm successful reception of a command or data frame from the sender.

Definition at line 820 of file rx_frame.c.

```
00821 {
00822   if (frame == nullptr) {
00823     return k_rx_err_invalid_arg;
00824   }
00825
00826   *frame                = (rx_frame_t){};
00827   frame->header.sequence = sequence;
00828   frame->header.length   = 0;
00829   frame->header.type     = k_frame_type_ack;
00830   frame->header.flags    = k_frame_flag_none;
00831
00832   /* Post-condition: type == k_frame_type_ack guaranteed by the direct assignment above. */
00833   return k_rx_ok;
00834 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_type_ack, rx_frame_header_t::length, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.1.4.2 rx_frame_create_nack()

```
rx_err_t rx_frame_create_nack (
                rx_frame_t * frame,
                const uint16_t sequence,
                uint8_t flags)   [nodiscard]
```

Create NACK frame for given sequence.

Parameters

out	frame	Frame to initialize
in	sequence	Sequence number
in	flags	Additional flags (e.g., k_frame_flag_soft_nack)

Returns

k_rx_ok on success

Create NACK frame for given sequence.

Constructs a frame with type=NACK, empty payload, the specified sequence number, and error/status flags. NACK frames are sent by the receiver to indicate that a frame was not processed successfully due to an error condition (specified in flags).

Definition at line 845 of file rx_frame.c.

```
00846 {
00847   if (frame == nullptr) {
00848     return k_rx_err_invalid_arg;
00849   }
00850
00851   *frame                = (rx_frame_t){};
00852   frame->header.sequence = sequence;
00853   frame->header.length   = 0;
00854   frame->header.type     = k_frame_type_nack;
00855   frame->header.flags    = flags;
00856
00857   /* Post-condition: type == k_frame_type_nack guaranteed by the direct assignment above. */
00858   return k_rx_ok;
00859 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_type_nack, rx_frame_header_t::length, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.1.4.3 rx_frame_create_ping()

```
rx_err_t rx_frame_create_ping (
                rx_frame_t * frame,
                const uint16_t sequence,
                const uint8_t * payload,
                const uint32_t payload_len)   [nodiscard]
```

Create PING frame for keepalive/heartbeat.

Parameters

out	frame	Frame to initialize
in	sequence	Sequence number
in	payload	Optional payload to include (may be NULL if payload_len is 0)
in	payload_len	Payload length in bytes

Returns

k_rx_ok on success

Create PING frame for keepalive/heartbeat.

Initializes frame as a PING request for connection liveness checks. Receiver should respond with a PONG frame echoing the payload. Payload is optional; a 4-byte LE counter is typical.

Precondition

frame !=nullptr

payload != NULL when payload_len > 0

Postcondition

frame->header.type == k_frame_type_ping

frame->header.length == payload_len

Note

Stateless; safe to call from any context.

See also

rx_frame_create_pong() Corresponding response creator

Since

Version 1.0.0

Definition at line 880 of file rx_frame.c.

```
00884 {
00885   if (frame == nullptr) {
00886     return k_rx_err_invalid_arg;
00887   }
00888
00889   if (payload_len > 0 && payload == nullptr) {
00890     return k_rx_err_invalid_arg;
00891   }
00892
00893   if (payload_len > k_frame_max_payload) {
00894     return k_rx_err_invalid_size;
00895   }
00896
00897   *frame                = (rx_frame_t){};
00898   frame->header.sequence = sequence;
00899   frame->header.length   = (uint16_t)payload_len;
00900   frame->header.type     = k_frame_type_ping;
```

```
00901   frame->header.flags     = k_frame_flag_none;
00902
00903   if (payload_len > 0) {
00904     for (uint32_t i = 0U; i < payload_len; i++) {
00905       frame->payload[i] = payload[i];
00906     }
00907   }
00908
00909   /* Post-condition: type == k_frame_type_ping guaranteed by the direct assignment above. */
00910   return k_rx_ok;
00911 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_max_payload, k_frame_type_ping, rx_frame_header_t::length, rx_frame_t::payload, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.1.4.4 rx_frame_create_pong()

```
rx_err_t rx_frame_create_pong (
                rx_frame_t * frame,
                const uint16_t sequence,
                const uint8_t * payload,
                const uint32_t payload_len)   [nodiscard]
```

Create PONG frame (response to PING).

Parameters

out	frame	Frame to initialize
in	sequence	Sequence number (matches ping)
in	payload	Optional payload to include (may be NULL if payload_len is 0)
in	payload_len	Payload length in bytes

Returns

k_rx_ok on success

Create PONG frame (response to PING).

Initializes frame as a PONG response to a received PING. Echoes the PING payload so the sender can validate round-trip data.

Precondition

frame !=nullptr

payload != NULL when payload_len > 0

Postcondition

frame->header.type == k_frame_type_pong

frame->header.length == payload_len

Note

Stateless; safe to call from any context.

See also

rx_frame_create_ping() Corresponding request creator

Since

Version 1.0.0

Definition at line 931 of file rx_frame.c.

```
00935 {
00936   if (frame == nullptr) {
00937     return k_rx_err_invalid_arg;
00938   }
00939
00940   if (payload_len > 0 && payload == nullptr) {
00941     return k_rx_err_invalid_arg;
00942   }
00943
00944   if (payload_len > k_frame_max_payload) {
00945     return k_rx_err_invalid_size;
00946   }
00947
00948   *frame                 = (rx_frame_t){};
00949   frame->header.sequence = sequence;
00950   frame->header.length   = (uint16_t)payload_len;
00951   frame->header.type     = k_frame_type_pong;
00952   frame->header.flags    = k_frame_flag_none;
00953
00954   if (payload_len > 0) {
00955     for (uint32_t i = 0U; i < payload_len; i++) {
00956       frame->payload[i] = payload[i];
00957     }
00958   }
00959
00960   /* Post-condition: type == k_frame_type_pong guaranteed by the direct assignment above. */
00961   return k_rx_ok;
00962 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_max_payload, k_frame_type_pong, rx_frame_header_t::length, rx_frame_t::payload, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.1.4.5 rx˙frame˙create˙reset()

```
rx_err_t rx_frame_create_reset (
                rx_frame_t * frame,
                const uint16_t sequence)   [nodiscard]
```

Create RESET frame to reset communication state.

Parameters

out	frame	Frame to initialize
in	sequence	Sequence number

Returns

k_rx_ok on success

Create RESET frame to reset communication state.

Initializes frame as a RESET request that asks the peer to reset communication state (sequence numbers, buffers). The receiver should respond with a RESET_ACK frame. Payload is always empty.

Precondition

frame !=nullptr

Caller has determined that a reset is necessary

Postcondition

frame->header.type == k_frame_type_reset

frame->header.length == 0

Note

Stateless; safe to call from any context.

Warning

Reset clears all sequence number state on both sides.

See also

rx_frame_create_reset_ack() Corresponding acknowledgment creator

Since

Version 1.0.0

Definition at line 984 of file rx_frame.c.

```
00985 {
00986   if (frame == nullptr) {
00987     return k_rx_err_invalid_arg;
00988   }
00989
00990   *frame                = (rx_frame_t){};
00991   frame->header.sequence = sequence;
00992   frame->header.length   = 0;
00993   frame->header.type     = k_frame_type_reset;
00994   frame->header.flags    = k_frame_flag_none;
00995
00996   /* Post-condition: type == k_frame_type_reset guaranteed by the direct assignment above. */
00997   return k_rx_ok;
00998 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_type_reset, rx_frame_header_t::length, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.1.4.6 rx˙frame˙create˙reset˙ack()

```
rx_err_t rx_frame_create_reset_ack (
                rx_frame_t * frame,
                const uint16_t sequence)   [nodiscard]
```

Create RESET_ACK frame (response to RESET).

Parameters

out	frame	Frame to initialize
in	sequence	Sequence number (matches reset)

Returns

k_rx_ok on success

Create RESET_ACK frame (response to RESET).

Initializes frame as a RESET_ACK response to a received RESET request. Confirms that the receiver has completed its reset procedure. Payload is always empty; sequence matches the RESET request.

Precondition

frame !=nullptr

A RESET frame was received and processed

Postcondition

frame->header.type == k_frame_type_reset_ack

frame->header.length == 0

Note

Stateless; safe to call from any context.

See also

rx_frame_create_reset() Corresponding request creator

Since

Version 1.0.0

Definition at line 1019 of file rx_frame.c.

```
01020 {
01021   if (frame == nullptr) {
01022     return k_rx_err_invalid_arg;
01023   }
01024
01025   *frame                = (rx_frame_t){};
01026   frame->header.sequence = sequence;
01027   frame->header.length   = 0;
01028   frame->header.type     = k_frame_type_reset_ack;
01029   frame->header.flags    = k_frame_flag_none;
01030
01031   /* Post-condition: type == k_frame_type_reset_ack guaranteed by the direct assignment above. */
01032   return k_rx_ok;
01033 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_type_reset_ack, rx_frame_header_t::length, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.1.4.7 rx_frame_decode()

```
rx_err_t rx_frame_decode (
        const rx_frame_decoder_t * dec,
        const uint8_t * data,
        const uint32_t data_len,
        rx_frame_t * frame)    [nodiscard]
```

Decode wire format to frame.

Parses wire format and validates SYNC word and CRC-32.

Parameters

in	dec	Decoder handle
in	data	Wire format data
in	data_len	Data length
out	frame	Decoded frame

Returns

k_rx_ok on success

k_rx_err_invalid_arg if any pointer is nullptr

k_rx_err_invalid_state if decoder not initialized

k_rx_err_invalid_size if data too short

k_rx_err_protocol_error if SYNC word invalid

k_rx_err_crc_mismatch if CRC validation fails

Decode wire format to frame.

Validates the decoder state, extracts the frame header, copies the payload, and verifies the CRC-32 checksum.

Note

This function performs validation at multiple points:

- Null pointer checks on all parameters
- Decoder initialization check
- Header parsing via internal_decode_header()
- CRC verification via internal_verify_crc() (may surface CRC backend errors)

Definition at line 671 of file rx_frame.c.

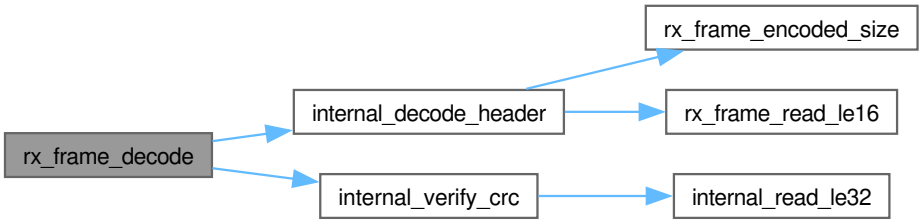
```
00675 {
00676   if (dec == nullptr || data == nullptr || frame == nullptr) {
00677     return k_rx_err_invalid_arg;
00678   }
00679
00680   if (dec->initialized != k_state_initialized) {
00681     return k_rx_err_invalid_state;
00682   }
00683
00684   uint32_t offset;
00685   rx_err_t err = internal_decode_header(data, data_len, frame, &offset);
00686   if (err != k_rx_ok) {
00687     return err;
```

```
00688   }
00689
00690   /* Read PAYLOAD */
00691   if (frame->header.length > 0) {
00692     for (uint32_t i = 0U; i < (uint32_t)frame->header.length; i++) {
00693       frame->payload[i] = data[offset + i];
00694     }
00695     offset += frame->header.length;
00696   }
00697
00698   err = internal_verify_crc(data, data_len, offset, &frame->crc);
00699   if (err != k_rx_ok) {
00700     return err;
00701   }
00702   return k_rx_ok;
00703 }
```

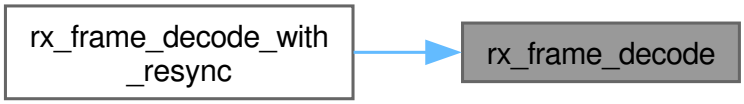
References rx_frame_t::crc, rx_frame_t::header, rx_frame_decoder_t::initialized, internal_decode_header(), internal_verify_crc(), k_state_initialized, rx_frame_header_t::length, and rx_frame_t::payload.

Referenced by rx_frame_decode_with_resync().

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.4.8 rx_frame_decode_with_resync()

```
rx_err_t rx_frame_decode_with_resync (
                const rx_frame_decoder_t * dec,
                const uint8_t * data,
                const uint32_t data_len,
                rx_frame_t * frame,
                uint32_t * bytes_discarded_out)    [nodiscard]
```

Decode wire format to frame with bounded sync-word resynchronization.

Extends rx_frame_decode() with automatic stream recovery. When the sync word is not found at offset 0 (i.e., the stream is byte-misaligned due to a dropped or inserted byte), this function scans forward up to k_frame_max_scan_bytes positions looking for the next sync word occurrence. Bytes discarded during the scan are reported via bytes_discarded_out for diagnostic logging by the caller.

If the sync word is located within the scan window, decoding proceeds from that offset as if data had been provided aligned. If no sync word is found within the bounded window, k_rx_err_protocol_error is returned.

Algorithm:

1. Attempt normal decode at offset 0.
2. On k_rx_err_protocol_error (sync mismatch), invoke internal_find_sync_offset() to locate the next sync word.
3. If found, set *bytes_discarded_out = sync_offset – caller must advance the stream read pointer by this value even if the subsequent decode fails (e.g., k_rx_err_crc_mismatch), to avoid infinite re-scan.
4. Decode from the discovered offset (trimmed view of data).
5. All other errors (CRC, size) are propagated unchanged.

Parameters

in	dec	Initialized frame decoder handle
in	data	Raw byte buffer (may be misaligned)
in	data_len	Length of data buffer in bytes
out	frame	Decoded frame (header, payload, CRC)
out	bytes_discarded_out	Number of bytes skipped to reach sync word (0 when sync was found at offset 0)

Returns

rx_err_t Error code

Return values

k_rx_ok	Frame decoded and CRC verified
k_rx_err_invalid_arg	Any pointer parameter is nullptr
k_rx_err_invalid_state	Decoder not initialized
k_rx_err_invalid_size	Data buffer too short to contain a valid frame
k_rx_err_protocol_error	No sync word found within k_frame_max_scan_bytes
k_rx_err_crc_mismatch	Sync found but CRC validation failed

Precondition

dec must be initialized via rx_frame_decoder_init()

data must point to a buffer of at least data_len bytes

frame must be a non-null pointer to a writable rx_frame_t

bytes_discarded_out must be a non-null pointer to a writable uint32_t

Postcondition

On k_rx_ok, frame contains the decoded frame with verified CRC

bytes_discarded_out contains the number of bytes skipped (>= 0)

On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer

Note

Thread-safe after init; the decoder handle is read-only and the function uses no shared mutable state (consistent with the file-level "Stateless after init").

The scan window is bounded at k_frame_max_scan_bytes (NASA Rule 2).

Caller should log bytes_discarded_out > 0 as a diagnostic warning.

Warning

Caller must advance its read pointer by *bytes_discarded_out bytes after any call that returns k↩ _rx_ok or k_rx_err_crc_mismatch to avoid infinite re-scan; *bytes_discarded_out is unmodified only when k_rx_err_invalid_arg is returned.

Performance:

Worst case: scans k_frame_max_scan_bytes (1036) before failing. Best case: identical to rx_frame_decode() (0 bytes discarded).

Example:

```
rx_frame_decoder_t dec;
rx_frame_decoder_init(&dec);

rx_frame_t frame;
uint32_t discarded = 0;
rx_err_t err = rx_frame_decode_with_resync(&dec, wire_buf, wire_len,
                                           &frame, &discarded);
if (discarded > 0) {
    rx_log_warn("FRAME", "Stream resync: discarded %u bytes", discarded);
}
if (err == k_rx_ok) {
    process_frame(&frame);
}
```

See also

rx_frame_decode() Simple decode without resynchronization

k_frame_max_scan_bytes Maximum scan window (1036 bytes)

Since

Version 1.0.0

Decode wire format to frame with bounded sync-word resynchronization.

Provides stream recovery when the byte stream has become misaligned due to a dropped or inserted byte. On sync mismatch, scans forward up to k_frame_max_scan_bytes positions for the next occurrence of the sync word, discards the leading bytes, and reattempts decoding from that position.

This function is safe for repeated calls on a stream: the caller must advance its stream read pointer by *bytes_discarded_out on k_rx_ok and also on k_rx_err_crc_mismatch (any case where bytes_↩ discarded_out is set) to avoid re-processing discarded bytes in subsequent calls.

Recovery algorithm:

1. Attempt rx_frame_decode() on data[0..data_len-1].
2. If result is not k_rx_err_protocol_error, return immediately.
3. Call internal_find_sync_offset() to locate next sync word in data[1..].
4. If not found, return k_rx_err_protocol_error with *bytes_discarded_out = 0.
5. Decode from data[sync_offset..data_len-1] using the trimmed sub-buffer.
6. Write sync_offset to *bytes_discarded_out and return result.

Precondition

dec must be initialized via rx_frame_decoder_init()

data must point to a buffer of at least data_len bytes

frame must point to a valid rx_frame_t storage location (not NULL)

bytes_discarded_out must point to a valid uint32_t storage location (not NULL)

Postcondition

On k_rx_ok, *bytes_discarded_out == bytes skipped to reach sync word

On k_rx_ok, frame contains a fully verified decoded frame

On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer

Note

Parameter order follows the canonical convention: decoder input -> data input -> frame output -> diagnostic output (matching rx_frame_decode()).

Not thread-safe. Caller must synchronize access.

The scan is bounded at k_frame_max_scan_bytes (NASA Rule 2).

Log *bytes_discarded_out > 0 as a stream-health diagnostic warning.

Warning

Caller must advance stream read pointer by *bytes_discarded_out after any call that returns k_↩ rx_ok or k_rx_err_crc_mismatch to avoid infinite re-scan; *bytes_discarded_out is unmodified only when k_rx_err_invalid_arg is returned.

Example:

```
uint32_t discarded = k_bytes_discarded_none;
rx_err_t err = rx_frame_decode_with_resync(dec, data, data_len, &frame, &discarded);
if (err == k_rx_ok) {
    // Advance stream read pointer past any discarded bytes
    stream_read_ptr += discarded;
} else if (err == k_rx_err_crc_mismatch) {
    // Sync was found but CRC failed -- still advance to avoid infinite re-scan
    stream_read_ptr += discarded;
    rx_log_error(TAG, "sync found but CRC failed");
} else {
    rx_log_error(TAG, "no sync word within scan window");
}
```

See also

rx_frame_decode() Simple decode without stream recovery

internal_find_sync_offset() Bounded sync word scanner

k_frame_max_scan_bytes Scan window upper bound

Since

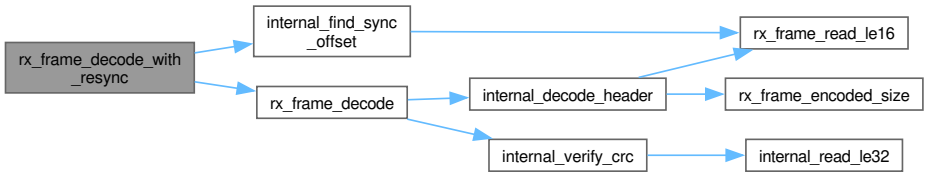
Version 1.0.0

Definition at line 769 of file rx_frame.c.

```
00774 {
00775   if (dec == nullptr || data == nullptr || frame == nullptr || bytes_discarded_out == nullptr) {
00776     return k_rx_err_invalid_arg;
00777   }
00778
00779   *bytes_discarded_out = k_bytes_discarded_none;
00780
00781   /* Fast path: attempt aligned decode first */
00782   rx_err_t err = rx_frame_decode(dec, data, data_len, frame);
00783   if (err != k_rx_err_protocol_error) {
00784     /* Either success, or an error other than sync mismatch (CRC, size, etc.) */
00785     return err;
00786   }
00787
00788   /* Sync word not at offset 0: scan forward for next sync word */
00789   uint32_t sync_offset = (uint32_t)k_frame_offset_start;
00790   err                  = internal_find_sync_offset(data, data_len, &sync_offset);
00791   if (err != k_rx_ok) {
00792     /* No sync word found within bounded window */
00793     return k_rx_err_protocol_error;
00794   }
00795
00796   /* Report discarded bytes now: caller needs this to advance past the sync
00797    * even if the subsequent decode fails (e.g. CRC mismatch at new offset) */
00798   *bytes_discarded_out = sync_offset;
00799
00800   /* Reattempt decode from the discovered sync offset */
00801   err = rx_frame_decode(dec, &data[sync_offset], data_len - sync_offset, frame);
00802
00803   return err;
00804 }
```

References internal_find_sync_offset(), k_bytes_discarded_none, k_frame_offset_start, and rx_frame_decode().

Here is the call graph for this function:



4.1.4.9 rx˙frame˙decoder˙deinit()

rx_err_t rx_frame_decoder_deinit (
rx_frame_decoder_t * dec) [nodiscard]

Deinitialize frame decoder.

Parameters

in,out	dec	Pointer to decoder handle

Returns

k_rx_ok on success

Deinitialize frame decoder.

Marks the decoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Definition at line 645 of file rx_frame.c.

```
00646 {
00647   if (dec == nullptr) {
00648     return k_rx_err_invalid_arg;
00649   }
00650
00651   dec->initialized = k_state_uninitialized;
00652   /* Post-condition: dec->initialized == k_state_uninitialized is guaranteed by the
00653    * direct assignment above. */
00654   return k_rx_ok;
00655 }
```

References rx_frame_decoder_t::initialized, and k_state_uninitialized.

4.1.4.10 rx_frame_decoder_init()

```
rx_err_t rx_frame_decoder_init (
                rx_frame_decoder_t * dec)   [nodiscard]
```

Initialize frame decoder.

Parameters

out	dec	Pointer to decoder handle

Returns

k_rx_ok on success, k_rx_err_invalid_arg if dec is nullptr

Initialize frame decoder.

Prepares the decoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Definition at line 625 of file rx_frame.c.

```
00626 {
00627   if (dec == nullptr) {
00628     return k_rx_err_invalid_arg;
00629   }
00630
00631   dec->initialized = k_state_initialized;
00632   /* Post-condition: dec->initialized == k_state_initialized is guaranteed by the
00633    * direct assignment above. */
00634   return k_rx_ok;
00635 }
```

References rx_frame_decoder_t::initialized, and k_state_initialized.

4.1.4.11 rx_frame_encode()

```
rx_err_t rx_frame_encode (
        const rx_frame_encoder_t * enc,
        const rx_frame_t * frame,
        uint8_t * output,
        uint32_t * output_len)   [nodiscard]
```

Encode frame to wire format.

Serializes frame to wire format with SYNC, header, payload, and CRC-32. All multi-byte fields are little-endian (SYNC, SEQ, LEN, CRC-32).

Parameters

in	enc	Encoder handle
in	frame	Frame to encode
out	output	Output buffer (min k_frame_min_size + payload bytes)
out	output_len	Actual number of bytes written

Returns

k_rx_ok on success

k_rx_err_invalid_arg if any pointer is nullptr

k_rx_err_invalid_state if encoder not initialized

k_rx_err_invalid_size if payload exceeds max

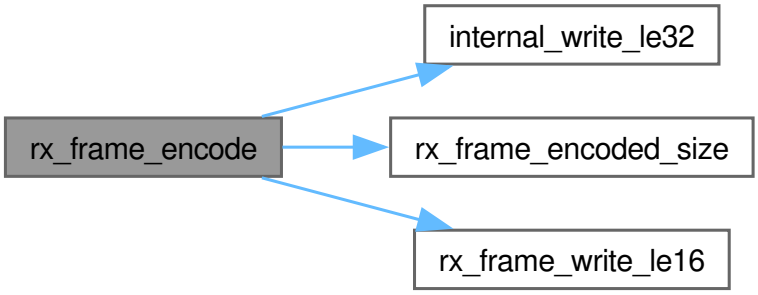
Definition at line 549 of file rx_frame.c.

```
00553 {
00554   if (enc == nullptr || frame == nullptr || output == nullptr || output_len == nullptr) {
00555     return k_rx_err_invalid_arg;
00556   }
00557
00558   if (enc->initialized != k_state_initialized) {
00559     return k_rx_err_invalid_state;
00560   }
00561
00562   /* Validate payload size */
00563   if (frame->header.length > k_frame_max_payload) {
00564     return k_rx_err_invalid_size;
00565   }
00566
00567   /* Calculate total frame size */
00568   uint32_t frame_size = rx_frame_encoded_size(frame->header.length);
00569   uint32_t offset     = k_frame_offset_start;
00570
00571   /* Write SYNC word (little-endian: wire bytes 0xAA, 0x55) */
00572   rx_frame_write_le16(&output[offset], k_frame_sync_word);
00573   offset += k_frame_sync_size;
00574
00575   /* Write SEQ (little-endian) */
00576   rx_frame_write_le16(&output[offset], frame->header.sequence);
00577   offset += k_frame_seq_size;
00578
00579   /* Write LEN (little-endian) */
00580   rx_frame_write_le16(&output[offset], frame->header.length);
00581   offset += k_frame_len_size;
00582
00583   /* Write TYPE (1 byte) */
00584   output[offset] = frame->header.type;
00585   offset += k_frame_type_size;
00586
00587   /* Write FLAGS (1 byte) */
00588   output[offset] = frame->header.flags;
00589   offset += k_frame_flags_size;
00590
00591   /* Write PAYLOAD */
```

```
00592   if (frame->header.length > 0) {
00593     for (uint32_t i = 0U; i < (uint32_t)frame->header.length; i++) {
00594       output[offset + i] = frame->payload[i];
00595     }
00596     offset += frame->header.length;
00597   }
00598
00599   /* Calculate CRC-32 over SYNC + Header + Payload (IEEE 802.3 polynomial).
00600    * rx_crc32_ieee always returns k_rx_ok for non-null buffer with valid length. */
00601   uint32_t crc = k_frame_crc32_init;
00602   (void)rx_crc32_ieee(output, offset, &crc);
00603
00604   /* Write CRC-32 (little-endian to match IEEE 802.3 LSB-first order).
00605    * internal_write_le32 always returns k_rx_ok when preconditions are met. */
00606   (void)internal_write_le32(&output[offset], frame_size - offset, crc);
00607
00608   *output_len = frame_size;
00609   return k_rx_ok;
00610 }
```

References rx_frame_header_t::flags, rx_frame_t::header, rx_frame_encoder_t::initialized, internal_write_le32(), k_frame_crc32_init, k_frame_flags_size, k_frame_len_size, k_frame_max_payload, k_frame_offset_start, k_frame_seq_size, k_frame_sync_size, k_frame_sync_word, k_frame_type_size, k_state_initialized, rx_frame_header_t::length, rx_frame_t::payload, rx_frame_encoded_size(), rx_frame_write_le16(), rx_frame_header_t::sequence, and rx_frame_header_t::type.

Here is the call graph for this function:



4.1.4.12 rx_frame_encoded_size()

```
uint32_t rx_frame_encoded_size (
                uint32_t payload_len)   [inline], [static]
```

Calculate encoded frame size.

Parameters

in	payload_len	Payload length

Returns

Total frame size in bytes

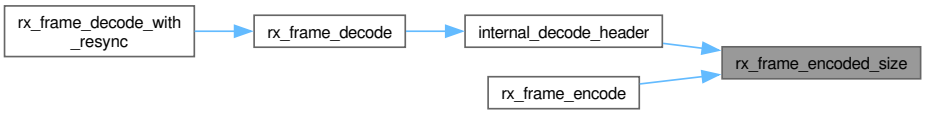
Definition at line 931 of file rx_frame.h.

```
00932 {
00933   return k_frame_sync_size + k_frame_header_size + payload_len + k_frame_crc_size;
00934 }
```

References k_frame_crc_size, k_frame_header_size, and k_frame_sync_size.

Referenced by internal_decode_header(), and rx_frame_encode().

Here is the caller graph for this function:



4.1.4.13 rx˙frame˙encoder˙deinit()

rx_err_t rx_frame_encoder_deinit (
rx_frame_encoder_t * enc) [nodiscard]

Deinitialize frame encoder.

Parameters

in,out	enc	Pointer to encoder handle

Returns

k_rx_ok on success

Deinitialize frame encoder.

Marks the encoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Definition at line 537 of file rx_frame.c.

```
00538 {
00539   if (enc == nullptr) {
00540     return k_rx_err_invalid_arg;
00541   }
00542
00543   enc->initialized = k_state_uninitialized;
00544   /* Post-condition: enc->initialized == k_state_uninitialized is guaranteed by the
00545    * direct assignment above. */
00546   return k_rx_ok;
00547 }
```

References rx_frame_encoder_t::initialized, and k_state_uninitialized.

4.1.4.14 rx˙frame˙encoder˙init()

rx_err_t rx_frame_encoder_init (
rx_frame_encoder_t * enc) [nodiscard]

Initialize frame encoder.

Parameters

out	enc	Pointer to encoder handle

Returns

k_rx_ok on success, k_rx_err_invalid_arg if enc is nullptr

Initialize frame encoder.

Prepares the encoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Definition at line 517 of file rx_frame.c.

```
00518 {
00519   if (enc == nullptr) {
00520     return k_rx_err_invalid_arg;
00521   }
00522
00523   enc->initialized = k_state_initialized;
00524   /* Post-condition: enc->initialized == k_state_initialized is guaranteed by the
00525    * direct assignment above. */
00526   return k_rx_ok;
00527 }
```

References rx_frame_encoder_t::initialized, and k_state_initialized.

4.1.4.15 rx_frame_read_le16()

```
uint16_t rx_frame_read_le16 (
            const uint8_t * buf)    [inline], [static]
```

Read uint16 from little-endian buffer.

Reads a 16-bit unsigned integer from a buffer in little-endian format. The low byte (LSB) is at index 0, the high byte (MSB) is at index 1.

Parameters

in	buf	Input buffer (at least 2 bytes)

Returns

Decoded 16-bit value in host byte order

Note

This is the preferred function for parsing multi-byte protocol fields.

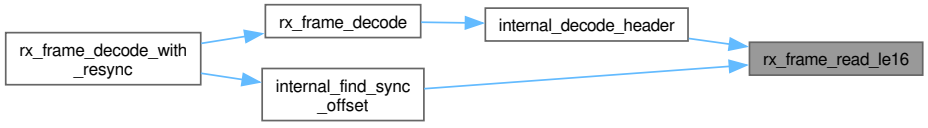
Definition at line 1003 of file rx_frame.h.

```
01004 {
01005   return (uint16_t)buf[k_le16_byte_low] | ((uint16_t)buf[k_le16_byte_high] << k_rx_le16_high_shift);
01006 }
```

References k_le16_byte_high, k_le16_byte_low, and k_rx_le16_high_shift.

Referenced by internal_decode_header(), and internal_find_sync_offset().

Here is the caller graph for this function:



4.1.4.16 rx_frame_read_le32()

```
uint32_t rx_frame_read_le32 (
            const uint8_t * buf)    [inline], [static]
```

Read uint32 from little-endian buffer.

Reads a 32-bit unsigned integer from a buffer in little-endian format.

Parameters

in	buf	Input buffer (at least 4 bytes)

Returns

Decoded 32-bit value in host byte order

Definition at line 1035 of file rx_frame.h.

```
01036 {
01037   return (uint32_t)buf[k_le32_byte_0] | ((uint32_t)buf[k_le32_byte_1] << k_rx_le32_shift_1) |
01038          ((uint32_t)buf[k_le32_byte_2] << k_rx_le32_shift_2) |
01039          ((uint32_t)buf[k_le32_byte_3] << k_rx_le32_shift_3);
01040 }
```

References k_le32_byte_0, k_le32_byte_1, k_le32_byte_2, k_le32_byte_3, k_rx_le32_shift_1, k_rx_le32_shift_2, and k_rx_le32_shift_3.

4.1.4.17 rx_frame_type_valid()

```
bool rx_frame_type_valid (
            uint8_t type)    [inline], [static]
```

Check if frame type is valid.

Static inline for same reasons as rx_frame_encoded_size() above.

Parameters

in	type	Frame type to check

Returns

true if valid, false otherwise

Definition at line 1251 of file rx_frame.h.

```
01252 {
01253   switch (type) {
01254     case k_frame_type_ping:
01255     case k_frame_type_pong:
01256     case k_frame_type_command:
01257     case k_frame_type_response:
01258     case k_frame_type_ack:
01259     case k_frame_type_nack:
01260     case k_frame_type_log_message:
01261     case k_frame_type_reset_ack:
01262     case k_frame_type_reset:
01263       return true;
01264     default:
01265       return false;
01266   }
01267 }
```

References k_frame_type_ack, k_frame_type_command, k_frame_type_log_message, k_frame_type_nack, k_frame_type_ping, k_frame_type_pong, k_frame_type_reset, k_frame_type_reset_ack, and k_frame_type_response.

4.1.4.18 rx_frame_write_le16()

```
void rx_frame_write_le16 (
                uint8_t * buf,
                uint16_t val)   [inline], [static]
```

Write uint16 to little-endian buffer.

Writes a 16-bit unsigned integer to a buffer in little-endian format. The low byte (LSB) is written at index 0, the high byte (MSB) is written at index 1.

Parameters

out	buf	Output buffer (at least 2 bytes)
in	val	Value to write in host byte order

Note

This is the preferred function for serializing multi-byte protocol fields.

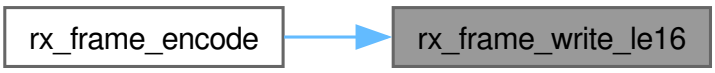
Definition at line 1021 of file rx_frame.h.

```
01022 {
01023   buf[k_le16_byte_low]  = (uint8_t)(val & k_rx_byte_mask);
01024   buf[k_le16_byte_high] = (uint8_t)(val » k_rx_le16_high_shift);
01025 }
```

References k_le16_byte_high, k_le16_byte_low, k_rx_byte_mask, and k_rx_le16_high_shift.

Referenced by rx_frame_encode().

Here is the caller graph for this function:



4.2 rx_frame.h

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_frame.h
00003  * @brief Frame Layer Protocol for SPI Communication
00004  *
00005  * @details
00006  * ## Overview
00007  *
00008  * This module implements the frame-level protocol for reliable SPI communication
00009  * between the Raspberry Pi 5 gateway and the RX72N motor controller. It provides
00010  * frame encoding, decoding, and CRC-32 integrity verification.
00011  *
00012  * **Key Features:**
00013  * - Bit-exact compatible with star-gateway/internal/frame/ (Go implementation)
00014  * - IEEE 802.3 CRC-32 for data integrity
00015  * - Little-endian byte order for ALL multi-byte fields (SYNC, SEQ, LEN, CRC)
00016  * - Support for ACK/NACK flow control
00017  * - Optional forward error correction (FEC) flag
00018  *
00019  * ## Frame Format
00020  *
00021  * ```
```

```
00022  * +---------+---------+---------+---------+---------+--------------+---------+
00023  * | SYNC    | SEQ     | LEN     | TYPE    | FLAGS   | PAYLOAD      | CRC-32  |
00024  * | 2 bytes | 2 bytes | 2 bytes | 1 byte  | 1 byte  | 0-1024 B     | 4 bytes |
00025  * | (LE)    | (LE)    | (LE)    |         |         |              | (LE)    |
00026  * +---------+---------+---------+---------+---------+--------------+---------+
00027  * ```
00028  *
00029  * **Field Descriptions:**
00030  *
00031  * | Field | Size | Byte Order | Description |
00032  * |-------|------|------------|-------------|
00033  * | SYNC | 2 | Little-endian | Synchronization marker (0x55AA, wire: 0xAA 0x55) |
00034  * | SEQ | 2 | Little-endian | Sequence number for ordering/retransmit |
00035  * | LEN | 2 | Little-endian | Payload length in bytes (0-1024) |
00036  * | TYPE | 1 | N/A | Frame type (command, response, ack, nack) |
00037  * | FLAGS | 1 | N/A | Control flags (requires_ack, retransmit, etc.) |
00038  * | PAYLOAD | 0-1024 | N/A | Encoded protobuf message or empty |
00039  * | CRC-32 | 4 | Little-endian | IEEE 802.3 CRC over SYNC+header+payload |
00040  *
00041  * ## Frame Size Calculations
00042  *
00043  * - **Minimum frame**: 12 bytes (SYNC + header + CRC, no payload)
00044  * - **Maximum frame**: 1036 bytes (SYNC + header + 1024 payload + CRC)
00045  * - **Overhead**: 12 bytes fixed per frame
00046  *
00047  * ## Protocol Flow
00048  *
00049  * @msc
00050  * RPi5, RX72N;
00051  *
00052  * ... [label="Normal Command/Response"];
00053  * RPi5 => RX72N [label="COMMAND (seq=1, requires_ack)"];
00054  * RPi5 <= RX72N [label="ACK (seq=1)"];
00055  * RPi5 <= RX72N [label="RESPONSE (seq=1)"];
00056  * RPi5 => RX72N [label="ACK (seq=1)"];
00057  *
00058  * ... [label="Retransmission on CRC Error"];
00059  * RPi5 => RX72N [label="COMMAND (seq=2)"];
00060  * RPi5 box RPi5 [label="Timeout"];
00061  * RPi5 => RX72N [label="COMMAND (seq=2, retransmit)"];
00062  * RPi5 <= RX72N [label="ACK (seq=2)"];
00063  *
00064  * ... [label="NACK on Decode Error"];
00065  * RPi5 => RX72N [label="COMMAND (seq=3, bad payload)"];
00066  * RPi5 <= RX72N [label="NACK (seq=3, soft_nack)"];
00067  * @endmsc
00068  *
00069  * ## CRC-32 Details
00070  *
00071  * The CRC-32 is computed using the IEEE 802.3 polynomial (0x04C11DB7) over
00072  * the entire frame excluding the CRC field itself:
00073  *
00074  * @f[
00075  *   \text{CRC} = \text{CRC32\_IEEE}(\text{SYNC} \| \text{Header} \| \text{Payload})
00076  * @f]
00077  *
00078  * **CRC Coverage:**
00079  * - SYNC word (2 bytes)
00080  * - SEQ (2 bytes)
00081  * - LEN (2 bytes)
00082  * - TYPE (1 byte)
00083  * - FLAGS (1 byte)
00084  * - PAYLOAD (0-1024 bytes)
00085  *
00086  * **Why Little-Endian CRC?**
00087  * The CRC-32 is stored in little-endian format (LSB first) to match the
00088  * IEEE 802.3 bit transmission order. This ensures compatibility with
00089  * Go's `crc32.ChecksumIEEE()` function.
00090  *
00091  * ## Thread Safety
00092  *
00093  * | Component | Thread Safe | Notes |
00094  * |-----------|-------------|-------|
00095  * | Encoder | [PASS] Yes | Stateless after init |
00096  * | Decoder | [PASS] Yes | Stateless after init |
00097  * | Utility functions | [PASS] Yes | Pure functions |
00098  *
00099  * ## Memory Usage
00100  *
00101  * | Component | Size | Notes |
00102  * |-----------|------|-------|
00103  * | rx_frame_t | 1034 bytes | Header + 1024-byte payload + CRC |
00104  * | rx_frame_encoder_t | 1 byte | Initialization flag only |
00105  * | rx_frame_decoder_t | 1 byte | Initialization flag only |
00106  * | Encoded frame buffer | 1036 bytes | Caller-provided |
00107  *
00108  * ## Performance Characteristics
```

```
00109  *
00110  * **Measured on RX72N @ 240 MHz, -O2 optimization:**
00111  *
00112  * | Operation | Empty Payload | 64B Payload | 255B Payload |
00113  * |-----------|---------------|-------------|--------------|
00114  * | Encode | ~5 us | ~15 us | ~50 us |
00115  * | Decode | ~5 us | ~15 us | ~50 us |
00116  * | CRC calc | ~1 us | ~6 us | ~25 us |
00117  *
00118  * ## NASA Power of 10 Compliance
00119  *
00120  * | Rule | Status | Notes |
00121  * |------|--------|-------|
00122  * | 1. Simple control flow | [PASS] Pass | No goto/setjmp/recursion |
00123  * | 2. Fixed loop bounds | [PASS] Pass | Bounded by k_frame_max_payload |
00124  * | 3. No dynamic memory | [PASS] Pass | All buffers caller-provided |
00125  * | 4. Short functions | [PASS] Pass | All functions < 50 lines |
00126  * | 5. Assertions | [PASS] Pass | NULL checks, state validation |
00127  * | 6. Small scope | [PASS] Pass | Variables at minimal scope |
00128  * | 7. Check returns | [PASS] Pass | All returns propagated |
00129  * | 8. Limited preprocessor | [PASS] Pass | Only include guards |
00130  * | 9. Restrict pointers | [PASS] Pass | No function pointers |
00131  * | 10. Compiler warnings | [PASS] Pass | -Wall -Wextra -Werror |
00132  *
00133  * ## SOLID Principles
00134  *
00135  * | Principle | Implementation |
00136  * |-----------|----------------|
00137  * | **Single Responsibility** | Frame layer only - no protobuf knowledge |
00138  * | **Open/Closed** | New frame types via enum, no code changes |
00139  * | **Liskov Substitution** | Encoder/decoder handles are interchangeable |
00140  * | **Interface Segregation** | Separate encoder/decoder APIs |
00141  * | **Dependency Inversion** | Depends on rx_crc.h abstraction |
00142  *
00143  * ## Hardware Requirements
00144  *
00145  * | Resource | Requirement |
00146  * |----------|-------------|
00147  * | **SPI** | 10 Mbps, Full-duplex |
00148  * | **DMA** | Optional, improves throughput |
00149  * | **CRC** | Uses rx_crc module (HW or SW) |
00150  *
00151  * ## Integration Example
00152  *
00153  * @code{.c}
00154  * #include "rx_frame.h"
00155  * #include "rx_spi_comm.h"
00156  *
00157  * // Transmit a command frame
00158  * void send_motor_command(uint8_t* protobuf_payload, uint32_t payload_len)
00159  * {
00160  *     rx_frame_encoder_t encoder;
00161  *     rx_frame_t frame;
00162  *     uint8_t wire_buffer[1036];
00163  *     uint32_t wire_len;
00164  *
00165  *     // Initialize encoder
00166  *     rx_frame_encoder_init(&encoder);
00167  *
00168  *     // Build frame
00169  *     frame.header.sequence = get_next_sequence();
00170  *     frame.header.length = payload_len;
00171  *     frame.header.type = k_frame_type_command;
00172  *     frame.header.flags = k_frame_flag_requires_ack;
00173  *     memcpy(frame.payload, protobuf_payload, payload_len);
00174  *
00175  *     // Encode to wire format
00176  *     rx_err_t err = rx_frame_encode(&encoder, &frame, wire_buffer, &wire_len);
00177  *     if (err != k_rx_ok) {
00178  *         rx_log_error("FRAME", "Encode failed: %d", err);
00179  *         return;
00180  *     }
00181  *
00182  *     // Send via SPI
00183  *     spi_transmit(wire_buffer, wire_len);
00184  * }
00185  *
00186  * // Receive and decode a frame
00187  * rx_err_t receive_frame(uint8_t* wire_data, uint32_t wire_len, rx_frame_t* frame)
00188  * {
00189  *     rx_frame_decoder_t decoder;
00190  *
00191  *     // Initialize decoder
00192  *     rx_frame_decoder_init(&decoder);
00193  *
00194  *     // Decode frame (validates SYNC, parses header, verifies CRC)
00195  *     rx_err_t err = rx_frame_decode(&decoder, wire_data, wire_len, frame);
```

```
00196 *     if (err == k_rx_err_crc_mismatch) {
00197 *         rx_log_error("FRAME", "CRC mismatch, requesting retransmit");
00198 *         send_nack(frame->header.sequence);
00199 *         return err;
00200 *     }
00201 *
00202 *     return err;
00203 * }
00204 * @endcode
00205 *
00206 * ## References
00207 *
00208 * - star-gateway/internal/frame/: Go reference implementation
00209 * - docs/sections/01_nanopb_protocol.tex: Protocol specification
00210 * - IEEE 802.3: CRC-32 polynomial definition
00211 * @see rx_frame.c Implementation file
00212 * @see rx_crc.h CRC-32 computation
00213 * @see rx_spi_comm.h SPI transport layer
00214 * @see rx_nanopb.h Protobuf encoding for payloads
00215 *
00216 * @author Locked, Inc.
00217 * @date 2026-01-29
00218 * @copyright Copyright (c) 2026 Locked Inc.
00219 * SPDX-License-Identifier: MIT
00220 * @since Version 1.0.0
00221 */
00222
00223 #pragma once
00224
00225 #include <stddef.h>
00226 #include <stdint.h>
00227
00228 #include "rx_err.h"
00229
00230 #ifdef __cplusplus
00231 extern "C" {
00232 #endif
00233
00234 /*
      ============================================================================
00235 * Frame Constants (must match Go implementation exactly)
00236 *
      ============================================================================
00237 */
00238
00239 /**
00240 * @enum rx_frame_constants_t
00241 * @brief Frame structure constants for SPI protocol
00242 *
00243 * @details
00244 * These constants define the frame format parameters that must exactly match
00245 * the Go implementation in star-gateway/internal/frame/. Any mismatch will
00246 * cause protocol interoperability failures.
00247 *
00248 * ## Frame Layout
00249 *
00250 * ```
00251 * Offset:  0       2       4       6     7     8          8+len
00252 * Field:   SYNC    SEQ     LEN     TYPE  FLAGS PAYLOAD... CRC
00253 * Size:    2       2       2       1     1     0-1024     4
00254 * ```
00255 *
00256 * ## Size Calculations
00257 *
00258 * | Frame Type | Formula | Bytes |
00259 * |------------|---------|-------|
00260 * | Minimum (no payload) | SYNC + Header + CRC | 12 |
00261 * | With N-byte payload | 12 + N | 12 - 1036 |
00262 * | Maximum (1KB payload) | SYNC + Header + 1024 + CRC | 1036 |
00263 *
00264 * @par Usage Example:
00265 * @code{.c}
00266 * // Validate frame size
00267 * if (wire_len < k_frame_min_size || wire_len > k_frame_max_size) {
00268 *     return k_rx_err_invalid_size;
00269 * }
00270 *
00271 * // Check sync word
00272 * uint16_t sync = rx_frame_read_le16(wire_data);
00273 * if (sync != k_frame_sync_word) {
00274 *     return k_rx_err_protocol_error;
00275 * }
00276 *
00277 * // Calculate expected size from payload length
00278 * uint16_t payload_len = rx_frame_read_le16(&wire_data[4]);
00279 * uint32_t expected = rx_frame_encoded_size(payload_len);
00280 * @endcode
```

```
00281  *
00282  * @warning These values MUST match star-gateway/internal/frame/constants.go
00283  * @note Using uint16_t underlying type to accommodate k_frame_max_payload (1024)
00284  *
00285  * @see star-gateway/internal/frame/ Go reference implementation
00286  * @since Version 1.0.0
00287  */
00288 typedef enum : uint16_t {
00289   k_frame_sync_word = 0x55AA, /**< @brief Frame synchronization marker (0x55AA)
00290                                * @details
00291                                * The sync word marks the start of a valid frame. Receivers scan for this
00292                                * pattern to establish frame boundaries. The value 0x55AA was chosen for:
00293                                * - Alternating bit pattern aids clock recovery
00294                                * - Unique enough to avoid false positives in random data
00295                                * - Matches common SPI debug patterns
00296                                * @par Wire Format: Little-endian (0xAA first, then 0x55)
00297                                */
00298
00299   k_frame_sync_size = 2, /**< @brief SYNC field size in bytes (2)
00300                           * @details Two-byte sync marker at frame start.
00301                           */
00302
00303   k_frame_seq_size = 2, /**< @brief SEQ field size in bytes (2)
00304                          * @details 16-bit sequence number for ordering and retransmit tracking.
00305                          */
00306
00307   k_frame_len_size = 2, /**< @brief LEN field size in bytes (2)
00308                          * @details 16-bit payload length (max 1024).
00309                          */
00310
00311   k_frame_type_size = 1, /**< @brief TYPE field size in bytes (1)
00312                           * @details Single byte for frame type (command, response, ack, nack).
00313                           */
00314
00315   k_frame_flags_size = 1, /**< @brief FLAGS field size in bytes (1)
00316                            * @details Single byte for control flags (requires_ack, retransmit, etc.).
00317                            */
00318
00319   k_frame_crc_size = 4, /**< @brief CRC-32 field size in bytes (4)
00320                          * @details IEEE 802.3 CRC-32 checksum.
00321                          */
00322
00323   k_frame_header_size = 6, /**< @brief Header size excluding SYNC (6 bytes)
00324                             * @details SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1) = 6 bytes.
00325                             */
00326
00327   k_frame_min_payload = 0, /**< @brief Minimum payload length (0 bytes)
00328                             * @details ACK and NACK frames have empty payloads.
00329                             */
00330
00331   k_frame_max_payload = 1024, /**< @brief Maximum payload length (1024 bytes)
00332                                * @details Limited by SPI DMA buffer size and protobuf message limits.
00333                                */
00334
00335   k_frame_min_size = 12, /**< @brief Minimum frame size (12 bytes)
00336                           * @details SYNC(2) + Header(6) + CRC(4) = 12 bytes (no payload).
00337                           */
00338
00339   k_frame_max_size = 1036, /**< @brief Maximum frame size (1036 bytes)
00340                             * @details SYNC(2) + Header(6) + Payload(1024) + CRC(4) = 1036 bytes.
00341                             */
00342
00343   k_frame_max_scan_bytes =
00344     k_frame_max_size, /**< @brief Maximum bytes to scan forward during sync recovery (1036)
00345                        * @details
00346                        * When the sync word is not found at offset 0, the decoder scans
00347                        * forward up to this many bytes looking for the sync pattern. The
00348                        * upper bound equals k_frame_max_size so recovery never reads more
00349                        * data than one complete maximum-size frame. This ensures bounded
00350                        * execution time (NASA Rule 2) and prevents unbounded stream
00351                        * consumption on permanently corrupt inputs.
00352                        *
00353                        * The scan discards all bytes before the discovered sync word and
00354                        * reattempts decoding from that position.
00355                        */
00356 } rx_frame_constants_t;
00357
00358 /**
00359  * @enum rx_frame_type_t
00360  * @brief Frame types for SPI protocol (matches Go FrameType)
00361  *
00362  * @details
00363  * Defines the type of message contained in a frame. The frame type determines
00364  * the expected payload content and required response handling.
00365  *
00366  * ## Communication Pattern
00367  *
```

```
00368  * @startuml
00369  * [*] --> Idle
00370  *
00371  * Idle --> WaitingForAck : TX COMMAND
00372  * WaitingForAck --> ProcessingResponse : RX ACK
00373  * WaitingForAck --> Idle : Timeout (retransmit)
00374  * WaitingForAck --> Idle : RX NACK (error)
00375  *
00376  * ProcessingResponse --> SendingAck : RX RESPONSE
00377  * SendingAck --> Idle : TX ACK
00378  * @enduml
00379  *
00380  * ## Type Descriptions
00381  *
00382  * | Type | Direction | Payload | Response |
00383  * |------|-----------|---------|----------|
00384  * | COMMAND | RPi5 -> RX72N | Protobuf | ACK, then RESPONSE |
00385  * | RESPONSE | RX72N -> RPi5 | Protobuf | ACK |
00386  * | ACK | Either | None | None |
00387  * | NACK | Either | None (flags indicate reason) | Retransmit |
00388  *
00389  * @par Usage Example:
00390  * @code{.c}
00391  * switch (frame->header.type) {
00392  *     case k_frame_type_command:
00393  *         // Process command, send ACK then RESPONSE
00394  *         send_ack(frame->header.sequence);
00395  *         process_command(frame->payload, frame->header.length);
00396  *         send_response(frame->header.sequence);
00397  *         break;
00398  *     case k_frame_type_ack:
00399  *         // Mark pending command as acknowledged
00400  *         mark_acked(frame->header.sequence);
00401  *         break;
00402  *     case k_frame_type_nack:
00403  *         // Retransmit the failed frame
00404  *         retransmit(frame->header.sequence);
00405  *         break;
00406  *     default:
00407  *         rx_log_error("FRAME", "Unknown type: %u", frame->header.type);
00408  * }
00409  * @endcode
00410  *
00411  * @warning Values MUST match star-gateway/internal/frame/types.go
00412  * @see rx_frame_type_valid() Validate frame type
00413  * @since Version 1.0.0
00414  */
00415 typedef enum : uint8_t {
00416     /**
00417      * @brief Ping request (0x00)
00418      * @details
00419      * Keepalive/heartbeat message sent to check connection status.
00420      * Receiver responds with PONG echoing the payload.
00421      * @par Payload: 4-byte counter (little-endian uint32) or empty
00422      */
00423     k_frame_type_ping = 0x00,
00424
00425     /**
00426      * @brief Pong response (0x01)
00427      * @details
00428      * Response to PING request, confirms connection is alive.
00429      * Echoes the PING counter payload for validation.
00430      * @par Payload: Echo of PING counter or empty
00431      */
00432     k_frame_type_pong = 0x01,
00433
00434     /**
00435      * @brief Command from controller to peripheral (0x10)
00436      * @details
00437      * Contains a protobuf-encoded command message from RPi5 to RX72N.
00438      * On SPI: typically requires ACK and generates a RESPONSE frame.
00439      * On USB CDC: no ACK expected (hardware reliability).
00440      * @par Typical Payloads: MotorCommand, TelemetryRequest, ConfigWrite
00441      */
00442     k_frame_type_command = 0x10,
00443
00444     /**
00445      * @brief Response from peripheral to controller (0x11)
00446      * @details
00447      * Contains a protobuf-encoded response message from RX72N to RPi5.
00448      * Sent after processing a COMMAND.
00449      * @par Typical Payloads: MotorStatus, TelemetryData, ConfigAck
00450      */
00451     k_frame_type_response = 0x11,
00452
00453     /**
00454      * @brief Positive acknowledgment (0x12, SPI only)
```

```
00455     * @details
00456     * Confirms successful reception and CRC validation of a frame.
00457     * Empty payload; sequence number matches the acknowledged frame.
00458     * Used only on SPI transport (USB CDC relies on hardware reliability).
00459     * @par Payload: None (length = 0)
00460     */
00461   k_frame_type_ack = 0x12,
00462
00463   /**
00464     * @brief Negative acknowledgment (0x13, SPI only)
00465     * @details
00466     * Indicates a problem with received frame (CRC error, decode failure).
00467     * Empty payload; flags indicate error type. Sender should retransmit.
00468     * Used only on SPI transport (UART relies on CRC-32 + bounded resync).
00469     * @par Payload: None (length = 0)
00470     * @par Flags: May include k_frame_flag_soft_nack
00471     */
00472   k_frame_type_nack = 0x13,
00473
00474   /**
00475     * @brief Firmware log message (0x20)
00476     * @details
00477     * Carries ASCII log text emitted by the RX72N runtime (rx_log_* macros).
00478     * Payload is a raw UTF-8 / 7-bit ASCII byte sequence -- NOT null-terminated
00479     * and may span multiple lines. The gateway extracts the payload and writes
00480     * it to its own log stream (stderr / log file). Multiplexing logs as framed
00481     * messages prevents log bytes from colliding with the frame sync word on the
00482     * shared UART-to-USB bridge byte stream.
00483     * @par Payload: 1..1024 bytes of log text (no null terminator)
00484     * @par Flags: Usually k_frame_flag_none
00485     */
00486   k_frame_type_log_message = 0x20,
00487
00488   /**
00489     * @brief Reset acknowledgment (0xFE)
00490     * @details
00491     * Confirms reset operation completed successfully.
00492     * Sequence number matches the reset request.
00493     * @par Payload: None (length = 0)
00494     */
00495   k_frame_type_reset_ack = 0xFE,
00496
00497   /**
00498     * @brief Reset request (0xFF)
00499     * @details
00500     * Request to reset communication state (sequence numbers, buffers, etc.).
00501     * Receiver responds with RESET_ACK.
00502     * @par Payload: None (length = 0)
00503     */
00504   k_frame_type_reset = 0xFF,
00505 } rx_frame_type_t;
00506
00507 /**
00508  * @enum rx_frame_flags_t
00509  * @brief Frame control flags (matches Go FrameFlags)
00510  *
00511  * @details
00512  * Bit flags that modify frame handling behavior. Multiple flags can be
00513  * combined using bitwise OR.
00514  *
00515  * ## Flag Descriptions
00516  *
00517  * | Flag | Bit | Purpose |
00518  * |------|-----|---------|
00519  * | REQUIRES_ACK | 0 | Sender expects ACK/NACK response |
00520  * | RETRANSMIT | 1 | This is a retransmission of previous frame |
00521  * | PRIORITY | 2 | Process before normal-priority frames |
00522  * | FEC_ENABLED | 3 | Payload uses forward error correction |
00523  * | SOFT_NACK | 4 | NACK with recoverable error info |
00524  *
00525  * ## Flag Combinations
00526  *
00527  * @code{.c}
00528  * // Typical command frame
00529  * frame.header.flags = k_frame_flag_requires_ack;
00530  *
00531  * // Retransmission after timeout
00532  * frame.header.flags = k_frame_flag_requires_ack | k_frame_flag_retransmit;
00533  *
00534  * // High-priority motor command
00535  * frame.header.flags = k_frame_flag_requires_ack | k_frame_flag_priority;
00536  *
00537  * // FEC-protected data transfer
00538  * frame.header.flags = k_frame_flag_fec_enabled;
00539  *
00540  * // NACK with error details
00541  * nack_frame.header.flags = k_frame_flag_soft_nack;
```

```
00542  * @endcode
00543  *
00544  * @par Usage Example:
00545  * @code{.c}
00546  * // Check if ACK required
00547  * if (frame->header.flags & k_frame_flag_requires_ack) {
00548  *     send_ack(frame->header.sequence);
00549  * }
00550  *
00551  * // Check if retransmit
00552  * if (frame->header.flags & k_frame_flag_retransmit) {
00553  *     rx_log_warn("FRAME", "Received retransmit seq=%u",
00554  *                 frame->header.sequence);
00555  *     stats.retransmit_count++;
00556  * }
00557  * @endcode
00558  *
00559  * @warning Values MUST match star-gateway/internal/frame/flags.go
00560  * @since Version 1.0.0
00561  */
00562 typedef enum : uint8_t {
00563     k_frame_flag_none = 0x00, /**< @brief No flags set (0x00)
00564                                * @details Default for frames that don't require special handling.
00565                                */
00566
00567     k_frame_flag_requires_ack = 0x01, /**< @brief Frame requires acknowledgment (0x01, bit 0)
00568                                        * @details
00569                                        * Sender expects an ACK or NACK response. If no response within
00570                                        * timeout, sender should retransmit with RETRANSMIT flag.
00571                                        */
00572
00573     k_frame_flag_retransmit = 0x02, /**< @brief Retransmission of previous frame (0x02, bit 1)
00574                                      * @details
00575                                      * Indicates this frame is a retry after timeout or NACK.
00576                                      * Receiver should check for duplicate sequence numbers.
00577                                      */
00578
00579     k_frame_flag_priority = 0x04, /**< @brief High-priority frame (0x04, bit 2)
00580                                    * @details
00581                                    * Process before normal-priority frames in queue.
00582                                    * Used for emergency stop and critical motor commands.
00583                                    */
00584
00585     k_frame_flag_fec_enabled = 0x08, /**< @brief Forward error correction enabled (0x08, bit 3)
00586                                       * @details
00587                                       * Payload is FEC-encoded (convolutional rate 1/2). FEC is enabled by default
00588                                       * on the HARQ path (e.g., SPI transport with rx_spi_link). This flag indicates
00589                                       * FEC encoding is present and receiver should apply Viterbi decoding before
00590                                       * processing payload. The flag is ignored for transports that do not use HARQ
00591                                       * (e.g., USB CDC, which relies on built-in hardware CRC and retry).
00592                                       */
00593
00594     k_frame_flag_soft_nack = 0x10, /**< @brief NACK with soft error information (0x10, bit 4)
00595                                     * @details
00596                                     * Used in NACK frames to indicate recoverable error.
00597                                     * May include confidence bits or partial decode info.
00598                                     */
00599 } rx_frame_flags_t;
00600
00601 /*
    ============================================================================
00602  * Frame Structures
00603  *
    ============================================================================
00604  */
00605
00606 /**
00607  * @struct rx_frame_header_t
00608  * @brief Frame header containing control fields
00609  *
00610  * @details
00611  * The frame header contains all control information needed to process a frame.
00612  * All 16-bit fields are stored in host byte order within this structure;
00613  * conversion to/from little-endian wire format is handled by the encoder/decoder.
00614  *
00615  * ## Memory Layout
00616  *
00617  * | Offset | Size | Field | Type | Description |
00618  * |--------|------|-------|------|-------------|
00619  * | 0 | 2 | sequence | uint16_t | Sequence number |
00620  * | 2 | 2 | length | uint16_t | Payload length |
00621  * | 4 | 1 | type | uint8_t | Frame type |
00622  * | 5 | 1 | flags | uint8_t | Control flags |
00623  * | **Total** | **6** | | | No padding required |
00624  *
00625  * ## Wire Format vs In-Memory Format
00626  *
```

```
00627  * **Wire Format (little-endian):**
00628  * - sequence: LSB at lower address
00629  * - length: LSB at lower address
00630  *
00631  * **In-Memory (this struct):**
00632  * - sequence: Host byte order (little-endian on RX72N, matches wire)
00633  * - length: Host byte order
00634  *
00635  * @par Usage Example:
00636  * @code{.c}
00637  * rx_frame_header_t header = {
00638  *     .sequence = 0x1234,            // Will be sent as 0x34, 0x12 on wire (LE)
00639  *     .length = 64,                  // 64-byte payload
00640  *     .type = k_frame_type_command,
00641  *     .flags = k_frame_flag_requires_ack,
00642  * };
00643  * @endcode
00644  *
00645  * @invariant length <= k_frame_max_payload (1024)
00646  * @invariant type is valid rx_frame_type_t value
00647  *
00648  * @see rx_frame_t Complete frame structure
00649  * @since Version 1.0.0
00650  */
00651 typedef struct {
00652     /**
00653      * @brief Frame sequence number
00654      * @details
00655      * 16-bit sequence number for ordering, duplicate detection, and
00656      * ACK/NACK correlation. Wraps around at 65535.
00657      * @par Wire Format: Little-endian
00658      * @par Valid Range: 0-65535
00659      */
00660     uint16_t sequence;
00661
00662     /**
00663      * @brief Payload length in bytes
00664      * @details
00665      * Length of the payload field (not including header or CRC).
00666      * Zero for ACK/NACK frames.
00667      * @par Wire Format: Little-endian
00668      * @par Valid Range: 0-1024 (k_frame_max_payload)
00669      * @invariant length <= k_frame_max_payload
00670      */
00671     uint16_t length;
00672
00673     /**
00674      * @brief Frame type identifier
00675      * @details
00676      * Identifies the message type (command, response, ack, nack).
00677      * @par Valid Values: rx_frame_type_t enumeration
00678      * @see rx_frame_type_valid() Validate type
00679      */
00680     uint8_t type;
00681
00682     /**
00683      * @brief Control flags
00684      * @details
00685      * Bitfield of rx_frame_flags_t values controlling frame handling.
00686      * Multiple flags can be combined with bitwise OR.
00687      * @par Valid Values: Combination of rx_frame_flags_t values
00688      */
00689     uint8_t flags;
00690 } rx_frame_header_t;
00691
00692 /**
00693  * @struct rx_frame_t
00694  * @brief Complete frame structure containing header, payload, and CRC
00695  *
00696  * @details
00697  * This structure represents a complete decoded frame ready for processing.
00698  * The encoder serializes this to wire format; the decoder populates it
00699  * from wire format.
00700  *
00701  * ## Memory Layout
00702  *
00703  * | Offset | Size | Field | Description |
00704  * |--------|------|-------|-------------|
00705  * | 0 | 6 | header | Frame header (rx_frame_header_t) |
00706  * | 6 | 1024 | payload | Payload buffer (max size) |
00707  * | 1030 | 4 | crc | CRC-32 checksum (for decoded frames) |
00708  * | **Total** | **1034** | | |
00709  *
00710  * **Note:** Structure size is 1034 bytes. The actual payload length is
00711  * indicated by header.length, not the buffer size.
00712  *
00713  * ## Usage Patterns
```

```
00714  *
00715  * **Encoding (TX):**
00716  * 1. Populate header fields (sequence, length, type, flags)
00717  * 2. Copy payload data to payload[] array
00718  * 3. Call rx_frame_encode() - CRC is computed automatically
00719  *
00720  * **Decoding (RX):**
00721  * 1. Call rx_frame_decode() with wire data
00722  * 2. Access header fields to determine frame type
00723  * 3. Read payload[0..header.length-1] for message content
00724  * 4. CRC field contains the received checksum (already verified)
00725  *
00726  * @par Usage Example - Building a Frame:
00727  * @code{.c}
00728  * rx_frame_t frame;
00729  * memset(&frame, 0, sizeof(frame));
00730  *
00731  * // Set header
00732  * frame.header.sequence = 1;
00733  * frame.header.length = protobuf_len;
00734  * frame.header.type = k_frame_type_command;
00735  * frame.header.flags = k_frame_flag_requires_ack;
00736  *
00737  * // Copy payload
00738  * memcpy(frame.payload, protobuf_data, protobuf_len);
00739  *
00740  * // Encode (CRC computed internally)
00741  * uint8_t wire[1036];
00742  * uint32_t wire_len;
00743  * rx_frame_encode(&encoder, &frame, wire, &wire_len);
00744  * @endcode
00745  *
00746  * @par Usage Example - Processing Decoded Frame:
00747  * @code{.c}
00748  * rx_frame_t frame;
00749  * rx_err_t err = rx_frame_decode(&decoder, wire_data, wire_len, &frame);
00750  *
00751  * if (err == k_rx_ok) {
00752  *     // Frame valid, CRC verified
00753  *     if (frame.header.type == k_frame_type_command) {
00754  *         // Process payload[0..header.length-1]
00755  *         handle_command(frame.payload, frame.header.length);
00756  *     }
00757  * }
00758  * @endcode
00759  *
00760  * @invariant header.length <= k_frame_max_payload
00761  * @note The crc field is populated during decode, not used during encode
00762  * @warning Payload data beyond header.length is undefined
00763  *
00764  * @see rx_frame_header_t Header structure
00765  * @see rx_frame_encode() Encode frame to wire format
00766  * @see rx_frame_decode() Decode frame from wire format
00767  * @since Version 1.0.0
00768  */
00769 typedef struct {
00770     /**
00771      * @brief Frame header containing control fields
00772      * @details Populated by caller (encode) or decoder (decode).
00773      */
00774     rx_frame_header_t header;
00775
00776     /**
00777      * @brief Payload data buffer (maximum 1024 bytes)
00778      * @details
00779      * For transmit: Contains data to send (length in header.length).
00780      * For receive: Contains decoded payload (length in header.length).
00781      * @note Only payload[0..header.length-1] is valid data.
00782      */
00783     uint8_t payload[k_frame_max_payload];
00784
00785     /**
00786      * @brief CRC-32 checksum (IEEE 802.3)
00787      * @details
00788      * For received frames: Contains the CRC read from wire (verified).
00789      * For transmitted frames: Not used (computed by encoder).
00790      * @par Wire Format: Little-endian
00791      */
00792     uint32_t crc;
00793 } rx_frame_t;
00794
00795 /**
00796  * @struct rx_frame_encoder_t
00797  * @brief Frame encoder handle
00798  *
00799  * @details
00800  * Lightweight handle structure for frame encoding operations.
```

```
00801  * Must be initialized before use with rx_frame_encoder_init().
00802  *
00803  * ## Lifecycle
00804  *
00805  * 1. Declare: `rx_frame_encoder_t enc;`
00806  * 2. Initialize: `rx_frame_encoder_init(&enc);`
00807  * 3. Use: `rx_frame_encode(&enc, &frame, output, &output_len);`
00808  * 4. Deinitialize: `rx_frame_encoder_deinit(&enc);` (optional)
00809  *
00810  * @par Memory Layout:
00811  * | Offset | Size | Field | Description |
00812  * |--------|------|-------|-------------|
00813  * | 0 | 1 | initialized | Initialization flag |
00814  * | **Total** | **1** | | No padding |
00815  *
00816  * @invariant initialized is 0 or 1
00817  * @note Structure is stateless after init; thread-safe for concurrent use
00818  *
00819  * @see rx_frame_encoder_init() Initialize encoder
00820  * @see rx_frame_encode() Encode frame
00821  * @since Version 1.0.0
00822  */
00823 typedef struct {
00824     /**
00825      * @brief Initialization state flag
00826      * @details Non-zero (1) if initialized and ready for use.
00827      * @par Valid Values: 0 (uninitialized), 1 (initialized)
00828      */
00829     uint8_t initialized;
00830 } rx_frame_encoder_t;
00831
00832 /**
00833  * @struct rx_frame_decoder_t
00834  * @brief Frame decoder handle
00835  *
00836  * @details
00837  * Lightweight handle structure for frame decoding operations.
00838  * Must be initialized before use with rx_frame_decoder_init().
00839  *
00840  * ## Lifecycle
00841  *
00842  * 1. Declare: `rx_frame_decoder_t dec;`
00843  * 2. Initialize: `rx_frame_decoder_init(&dec);`
00844  * 3. Use: `rx_frame_decode(&dec, data, len, &frame);`
00845  * 4. Deinitialize: `rx_frame_decoder_deinit(&dec);` (optional)
00846  *
00847  * @par Memory Layout:
00848  * | Offset | Size | Field | Description |
00849  * |--------|------|-------|-------------|
00850  * | 0 | 1 | initialized | Initialization flag |
00851  * | **Total** | **1** | | No padding |
00852  *
00853  * @invariant initialized is 0 or 1
00854  * @note Structure is stateless after init; thread-safe for concurrent use
00855  *
00856  * @see rx_frame_decoder_init() Initialize decoder
00857  * @see rx_frame_decode() Decode frame
00858  * @since Version 1.0.0
00859  */
00860 typedef struct {
00861     /**
00862      * @brief Initialization state flag
00863      * @details Non-zero (1) if initialized and ready for use.
00864      * @par Valid Values: 0 (uninitialized), 1 (initialized)
00865      */
00866     uint8_t initialized;
00867 } rx_frame_decoder_t;
00868
00869 /*
      ================================================================================
00870  * Encoder API
00871  *
      ================================================================================
00872  */
00873
00874 /**
00875  * @brief Initialize frame encoder
00876  *
00877  * @param[out] enc Pointer to encoder handle
00878  * @return k_rx_ok on success, k_rx_err_invalid_arg if enc is nullptr
00879  */
00880 [[nodiscard]] rx_err_t rx_frame_encoder_init(rx_frame_encoder_t* enc);
00881
00882 /**
00883  * @brief Deinitialize frame encoder
00884  *
00885  * @param[in,out] enc Pointer to encoder handle
```

```
00886  * @return k_rx_ok on success
00887  */
00888 [[nodiscard]] rx_err_t rx_frame_encoder_deinit(rx_frame_encoder_t* enc);
00889
00890 /**
00891  * @brief Encode frame to wire format
00892  *
00893  * Serializes frame to wire format with SYNC, header, payload, and CRC-32.
00894  * All multi-byte fields are little-endian (SYNC, SEQ, LEN, CRC-32).
00895  *
00896  * @param[in]  enc        Encoder handle
00897  * @param[in]  frame      Frame to encode
00898  * @param[out] output     Output buffer (min k_frame_min_size + payload bytes)
00899  * @param[out] output_len Actual number of bytes written
00900  *
00901  * @return k_rx_ok on success
00902  * @return k_rx_err_invalid_arg if any pointer is nullptr
00903  * @return k_rx_err_invalid_state if encoder not initialized
00904  * @return k_rx_err_invalid_size if payload exceeds max
00905  */
00906 [[nodiscard]] rx_err_t rx_frame_encode(const rx_frame_encoder_t* enc,
00907                                        const rx_frame_t*         frame,
00908                                        uint8_t*                  output,
00909                                        uint32_t*                 output_len);
00910
00911 /*
      ============================================================================
00912  * Utility Functions (Static Inline)
00913  *
00914  * NOTE: These are intentionally static inline in the header because:
00915  * 1. They are trivial one-liner calculations (single arithmetic operation)
00916  * 2. They may be called in performance-sensitive code paths
00917  * 3. They need to be available across multiple translation units
00918  * 4. Function call overhead would exceed the actual computation cost
00919  *
00920  * For such trivial operations, static inline is the standard C pattern and
00921  * results in better performance with no binary size increase.
00922  *
      ============================================================================
00923  */
00924
00925 /**
00926  * @brief Calculate encoded frame size
00927  *
00928  * @param[in] payload_len Payload length
00929  * @return Total frame size in bytes
00930  */
00931 static inline uint32_t rx_frame_encoded_size(uint32_t payload_len)
00932 {
00933   return k_frame_sync_size + k_frame_header_size + payload_len + k_frame_crc_size;
00934 }
00935
00936 /*
      ============================================================================
00937  * Byte Ordering Utilities (Static Inline)
00938  *
00939  * Little-endian read/write for wire format serialization.
00940  * Used by rx_frame, rx_usb_comm, and rx_spi_comm modules.
00941  *
00942  * Little-endian format stores the least significant byte (LSB) at the lowest
00943  * memory address. For a 16-bit value:
00944  *   - buf[0] = low byte (LSB)
00945  *   - buf[1] = high byte (MSB)
00946  *
00947  * Example: 0x55AA stored in little-endian:
00948  *   buf[0] = 0xAA (low byte)
00949  *   buf[1] = 0x55 (high byte)
00950  *
      ============================================================================
00951  */
00952
00953 /**
00954  * @brief Byte indices for 16-bit little-endian serialization
00955  *
00956  * In little-endian format, the low byte (LSB) is stored at index 0 and the
00957  * high byte (MSB) is stored at index 1.
00958  */
00959 typedef enum : uint8_t {
00960   k_le16_byte_low  = 0, /**< Low byte (LSB) at index 0 */
00961   k_le16_byte_high = 1, /**< High byte (MSB) at index 1 */
00962 } rx_le16_byte_idx_t;
00963
00964 /**
00965  * @brief Byte manipulation constants for endianness conversions
00966  */
00967 typedef enum : uint8_t {
00968   k_rx_le16_high_shift = (8),     /**< Bit shift for high byte in 16-bit value */
```

```
00969   k_rx_byte_mask       = (0xFFU), /**< Mask to extract one byte */
00970 } rx_byte_order_constants_t;
00971
00972 /**
00973  * @brief Byte indices for 32-bit little-endian serialization
00974  */
00975 typedef enum : uint8_t {
00976   k_le32_byte_0 = 0, /**< Byte 0 (LSB) */
00977   k_le32_byte_1 = 1, /**< Byte 1 */
00978   k_le32_byte_2 = 2, /**< Byte 2 */
00979   k_le32_byte_3 = 3, /**< Byte 3 (MSB) */
00980 } rx_le32_byte_idx_t;
00981
00982 /**
00983  * @brief Bit shift amounts for extracting bytes from 32-bit values
00984  */
00985 typedef enum : uint8_t {
00986   k_rx_le32_shift_0 = 0,  /**< Shift 0 bits for byte 0 */
00987   k_rx_le32_shift_1 = 8,  /**< Shift 8 bits for byte 1 */
00988   k_rx_le32_shift_2 = 16, /**< Shift 16 bits for byte 2 */
00989   k_rx_le32_shift_3 = 24, /**< Shift 24 bits for byte 3 */
00990 } rx_le32_shift_t;
00991
00992 /**
00993  * @brief Read uint16 from little-endian buffer
00994  *
00995  * Reads a 16-bit unsigned integer from a buffer in little-endian format.
00996  * The low byte (LSB) is at index 0, the high byte (MSB) is at index 1.
00997  *
00998  * @param[in] buf Input buffer (at least 2 bytes)
00999  * @return Decoded 16-bit value in host byte order
01000  *
01001  * @note This is the preferred function for parsing multi-byte protocol fields.
01002  */
01003 static inline uint16_t rx_frame_read_le16(const uint8_t* buf)
01004 {
01005   return (uint16_t)buf[k_le16_byte_low] | ((uint16_t)buf[k_le16_byte_high] « k_rx_le16_high_shift);
01006 }
01007
01008 /**
01009  * @brief Write uint16 to little-endian buffer
01010  *
01011  * Writes a 16-bit unsigned integer to a buffer in little-endian format.
01012  * The low byte (LSB) is written at index 0, the high byte (MSB) is written
01013  * at index 1.
01014  *
01015  * @param[out] buf Output buffer (at least 2 bytes)
01016  * @param[in]  val Value to write in host byte order
01017  *
01018  * @note This is the preferred function for serializing multi-byte protocol
01019  * fields.
01020  */
01021 static inline void rx_frame_write_le16(uint8_t* buf, uint16_t val)
01022 {
01023   buf[k_le16_byte_low]  = (uint8_t)(val & k_rx_byte_mask);
01024   buf[k_le16_byte_high] = (uint8_t)(val » k_rx_le16_high_shift);
01025 }
01026
01027 /**
01028  * @brief Read uint32 from little-endian buffer
01029  *
01030  * Reads a 32-bit unsigned integer from a buffer in little-endian format.
01031  *
01032  * @param[in] buf Input buffer (at least 4 bytes)
01033  * @return Decoded 32-bit value in host byte order
01034  */
01035 static inline uint32_t rx_frame_read_le32(const uint8_t* buf)
01036 {
01037   return (uint32_t)buf[k_le32_byte_0] | ((uint32_t)buf[k_le32_byte_1] « k_rx_le32_shift_1) |
01038          ((uint32_t)buf[k_le32_byte_2] « k_rx_le32_shift_2) |
01039          ((uint32_t)buf[k_le32_byte_3] « k_rx_le32_shift_3);
01040 }
01041
01042 /*
      ============================================================================
01043  * Decoder API
01044  *
      ============================================================================
01045  */
01046
01047 /**
01048  * @brief Initialize frame decoder
01049  *
01050  * @param[out] dec Pointer to decoder handle
01051  * @return k_rx_ok on success, k_rx_err_invalid_arg if dec is nullptr
01052  */
01053 [[nodiscard]] rx_err_t rx_frame_decoder_init(rx_frame_decoder_t* dec);
```

```
01054
01055 /**
01056  * @brief Deinitialize frame decoder
01057  *
01058  * @param[in,out] dec Pointer to decoder handle
01059  * @return k_rx_ok on success
01060  */
01061 [[nodiscard]] rx_err_t rx_frame_decoder_deinit(rx_frame_decoder_t* dec);
01062
01063 /**
01064  * @brief Decode wire format to frame
01065  *
01066  * Parses wire format and validates SYNC word and CRC-32.
01067  *
01068  * @param[in]  dec      Decoder handle
01069  * @param[in]  data     Wire format data
01070  * @param[in]  data_len Data length
01071  * @param[out] frame    Decoded frame
01072  *
01073  * @return k_rx_ok on success
01074  * @return k_rx_err_invalid_arg if any pointer is nullptr
01075  * @return k_rx_err_invalid_state if decoder not initialized
01076  * @return k_rx_err_invalid_size if data too short
01077  * @return k_rx_err_protocol_error if SYNC word invalid
01078  * @return k_rx_err_crc_mismatch if CRC validation fails
01079  */
01080 [[nodiscard]] rx_err_t rx_frame_decode(const rx_frame_decoder_t* dec,
01081                                        const uint8_t*           data,
01082                                        uint32_t                 data_len,
01083                                        rx_frame_t*              frame);
01084
01085 /**
01086  * @brief Decode wire format to frame with bounded sync-word resynchronization
01087  *
01088  * @details
01089  * Extends rx_frame_decode() with automatic stream recovery. When the sync word
01090  * is not found at offset 0 (i.e., the stream is byte-misaligned due to a
01091  * dropped or inserted byte), this function scans forward up to
01092  * k_frame_max_scan_bytes positions looking for the next sync word occurrence.
01093  * Bytes discarded during the scan are reported via bytes_discarded_out for
01094  * diagnostic logging by the caller.
01095  *
01096  * If the sync word is located within the scan window, decoding proceeds from
01097  * that offset as if data had been provided aligned. If no sync word is found
01098  * within the bounded window, k_rx_err_protocol_error is returned.
01099  *
01100  * Algorithm:
01101  * 1. Attempt normal decode at offset 0.
01102  * 2. On k_rx_err_protocol_error (sync mismatch), invoke
01103  *    internal_find_sync_offset() to locate the next sync word.
01104  * 3. If found, set *bytes_discarded_out = sync_offset -- caller must advance the
01105  *    stream read pointer by this value even if the subsequent decode fails
01106  *    (e.g., k_rx_err_crc_mismatch), to avoid infinite re-scan.
01107  * 4. Decode from the discovered offset (trimmed view of data).
01108  * 5. All other errors (CRC, size) are propagated unchanged.
01109  *
01110  * @param[in]  dec                Initialized frame decoder handle
01111  * @param[in]  data               Raw byte buffer (may be misaligned)
01112  * @param[in]  data_len           Length of data buffer in bytes
01113  * @param[out] frame              Decoded frame (header, payload, CRC)
01114  * @param[out] bytes_discarded_out  Number of bytes skipped to reach sync word
01115  *                                (0 when sync was found at offset 0)
01116  *
01117  * @return rx_err_t Error code
01118  * @retval k_rx_ok               Frame decoded and CRC verified
01119  * @retval k_rx_err_invalid_arg  Any pointer parameter is nullptr
01120  * @retval k_rx_err_invalid_state Decoder not initialized
01121  * @retval k_rx_err_invalid_size  Data buffer too short to contain a valid frame
01122  * @retval k_rx_err_protocol_error No sync word found within k_frame_max_scan_bytes
01123  * @retval k_rx_err_crc_mismatch  Sync found but CRC validation failed
01124  *
01125  * @pre dec must be initialized via rx_frame_decoder_init()
01126  * @pre data must point to a buffer of at least data_len bytes
01127  * @pre frame must be a non-null pointer to a writable rx_frame_t
01128  * @pre bytes_discarded_out must be a non-null pointer to a writable uint32_t
01129  * @post On k_rx_ok, frame contains the decoded frame with verified CRC
01130  * @post bytes_discarded_out contains the number of bytes skipped (>= 0)
01131  * @post On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer
01132  *
01133  * @note Thread-safe after init; the decoder handle is read-only and the function
01134  *       uses no shared mutable state (consistent with the file-level "Stateless after init").
01135  * @note The scan window is bounded at k_frame_max_scan_bytes (NASA Rule 2).
01136  * @note Caller should log bytes_discarded_out > 0 as a diagnostic warning.
01137  *
01138  * @warning Caller must advance its read pointer by *bytes_discarded_out bytes after any
01139  *          call that returns k_rx_ok or k_rx_err_crc_mismatch to avoid infinite re-scan;
01140  *          *bytes_discarded_out is unmodified only when k_rx_err_invalid_arg is returned.
```

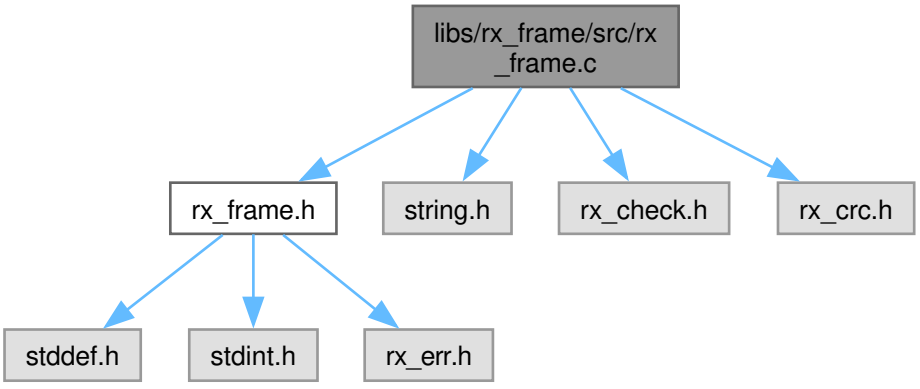
```
01141  *
01142  * @par Performance:
01143  * Worst case: scans k_frame_max_scan_bytes (1036) before failing.
01144  * Best case: identical to rx_frame_decode() (0 bytes discarded).
01145  *
01146  * @par Example:
01147  * @code{.c}
01148  * rx_frame_decoder_t dec;
01149  * rx_frame_decoder_init(&dec);
01150  *
01151  * rx_frame_t frame;
01152  * uint32_t discarded = 0;
01153  * rx_err_t err = rx_frame_decode_with_resync(&dec, wire_buf, wire_len,
01154  *                                            &frame, &discarded);
01155  * if (discarded > 0) {
01156  *     rx_log_warn("FRAME", "Stream resync: discarded %u bytes", discarded);
01157  * }
01158  * if (err == k_rx_ok) {
01159  *     process_frame(&frame);
01160  * }
01161  * @endcode
01162  *
01163  * @see rx_frame_decode() Simple decode without resynchronization
01164  * @see k_frame_max_scan_bytes Maximum scan window (1036 bytes)
01165  * @since Version 1.0.0
01166  */
01167 [[nodiscard]] rx_err_t rx_frame_decode_with_resync(const rx_frame_decoder_t* dec,
01168                                                    const uint8_t*           data,
01169                                                    uint32_t                 data_len,
01170                                                    rx_frame_t*              frame,
01171                                                    uint32_t*                bytes_discarded_out);
01172
01173 /*
     ================================================================================
01174  * Frame Helper Functions
01175  *
     ================================================================================
01176  */
01177
01178 /**
01179  * @brief Create ACK frame for given sequence
01180  *
01181  * @param[out] frame Frame to initialize
01182  * @param[in]  sequence Sequence number to acknowledge
01183  * @return k_rx_ok on success
01184  */
01185 [[nodiscard]] rx_err_t rx_frame_create_ack(rx_frame_t* frame, uint16_t sequence);
01186
01187 /**
01188  * @brief Create NACK frame for given sequence
01189  *
01190  * @param[out] frame Frame to initialize
01191  * @param[in]  sequence Sequence number
01192  * @param[in]  flags Additional flags (e.g., k_frame_flag_soft_nack)
01193  * @return k_rx_ok on success
01194  */
01195 [[nodiscard]] rx_err_t rx_frame_create_nack(rx_frame_t* frame, uint16_t sequence, uint8_t flags);
01196
01197 /**
01198  * @brief Create PING frame for keepalive/heartbeat
01199  *
01200  * @param[out] frame Frame to initialize
01201  * @param[in]  sequence Sequence number
01202  * @param[in]  payload Optional payload to include (may be NULL if payload_len is 0)
01203  * @param[in]  payload_len Payload length in bytes
01204  * @return k_rx_ok on success
01205  */
01206 [[nodiscard]] rx_err_t rx_frame_create_ping(rx_frame_t*    frame,
01207                                             uint16_t       sequence,
01208                                             const uint8_t* payload,
01209                                             uint32_t       payload_len);
01210
01211 /**
01212  * @brief Create PONG frame (response to PING)
01213  *
01214  * @param[out] frame Frame to initialize
01215  * @param[in]  sequence Sequence number (matches ping)
01216  * @param[in]  payload Optional payload to include (may be NULL if payload_len is 0)
01217  * @param[in]  payload_len Payload length in bytes
01218  * @return k_rx_ok on success
01219  */
01220 [[nodiscard]] rx_err_t rx_frame_create_pong(rx_frame_t*    frame,
01221                                             uint16_t       sequence,
01222                                             const uint8_t* payload,
01223                                             uint32_t       payload_len);
01224
01225 /**
```

```
01226  * @brief Create RESET frame to reset communication state
01227  *
01228  * @param[out] frame Frame to initialize
01229  * @param[in]  sequence Sequence number
01230  * @return k_rx_ok on success
01231  */
01232 [[nodiscard]] rx_err_t rx_frame_create_reset(rx_frame_t* frame, uint16_t sequence);
01233
01234 /**
01235  * @brief Create RESET_ACK frame (response to RESET)
01236  *
01237  * @param[out] frame Frame to initialize
01238  * @param[in]  sequence Sequence number (matches reset)
01239  * @return k_rx_ok on success
01240  */
01241 [[nodiscard]] rx_err_t rx_frame_create_reset_ack(rx_frame_t* frame, uint16_t sequence);
01242
01243 /**
01244  * @brief Check if frame type is valid
01245  *
01246  * Static inline for same reasons as rx_frame_encoded_size() above.
01247  *
01248  * @param[in] type Frame type to check
01249  * @return true if valid, false otherwise
01250  */
01251 static inline bool rx_frame_type_valid(uint8_t type)
01252 {
01253   switch (type) {
01254     case k_frame_type_ping:
01255     case k_frame_type_pong:
01256     case k_frame_type_command:
01257     case k_frame_type_response:
01258     case k_frame_type_ack:
01259     case k_frame_type_nack:
01260     case k_frame_type_log_message:
01261     case k_frame_type_reset_ack:
01262     case k_frame_type_reset:
01263       return true;
01264     default:
01265       return false;
01266   }
01267 }
01268
01269 #ifdef __cplusplus
01270 }
01271 #endif
```

4.3 libs/rx˙frame/src/rx˙frame.c File Reference

Frame Layer Protocol Implementation.

#include ”rx_frame.h”
#include <string.h>
#include ”rx_check.h”
#include ”rx_crc.h”
Include dependency graph for rx_frame.c:



Enumerations

- enum byte_shift_t : uint8_t { k_shift_byte_0 = 0 , k_shift_byte_1 = 8 , k_shift_byte_2 = 16 , k_shift_byte_3 = 24 }

  Bit shift amounts for extracting bytes from multi-byte values.
- enum init_state_t : uint8_t { k_state_uninitialized = 0 , k_state_initialized = 1 }

  Initialization state values.
- enum frame_offset_t : uint8_t { k_frame_offset_start = 0 , k_frame_scan_start_offset = 1 }

  Frame byte offsets used during encoding, decoding, and sync scanning.
- enum frame_resync_discarded_t : uint32_t { k_bytes_discarded_none = 0U }

  Byte-discard counter initial value for rx_frame_decode_with_resync().
- enum frame_crc_init_t : uint32_t { k_frame_crc32_init = 0U }

  Initial value for CRC-32 output variable before rx_crc32_ieee() writes it.

Functions

- static rx_err_t internal_write_le32 (uint8_t *buf, const uint32_t buf_len, const uint32_t val)

  Write uint32 in little-endian format (for CRC-32).
- static rx_err_t internal_read_le32 (const uint8_t *buf, const uint32_t buf_len, uint32_t *out_val)

  Read uint32 in little-endian format (for CRC-32).
- static rx_err_t internal_decode_header (const uint8_t *data, const uint32_t data_len, rx_frame_t *frame, uint32_t *offset_out)

  Decode frame header from raw data buffer.
- static rx_err_t internal_verify_crc (const uint8_t *data, uint32_t data_len, uint32_t offset, uint32_t *crc_out)

  Verify CRC-32 checksum at specified offset in data buffer.
- static rx_err_t internal_find_sync_offset (const uint8_t *data, const uint32_t data_len, uint32_t *offset_out)

  Scan a byte buffer for the frame sync word with a fixed upper bound.
- rx_err_t rx_frame_encoder_init (rx_frame_encoder_t *enc)

  Initialize a frame encoder instance.
- rx_err_t rx_frame_encoder_deinit (rx_frame_encoder_t *enc)

  Deinitialize a frame encoder instance.
- rx_err_t rx_frame_encode (const rx_frame_encoder_t *enc, const rx_frame_t *frame, uint8_t *output, uint32_t *output_len)

  Encode frame to wire format.
- rx_err_t rx_frame_decoder_init (rx_frame_decoder_t *dec)

  Initialize a frame decoder instance.
- rx_err_t rx_frame_decoder_deinit (rx_frame_decoder_t *dec)

  Deinitialize a frame decoder instance.
- rx_err_t rx_frame_decode (const rx_frame_decoder_t *dec, const uint8_t *data, const uint32_t data_len, rx_frame_t *frame)

  Decode a frame from raw data.
- rx_err_t rx_frame_decode_with_resync (const rx_frame_decoder_t *dec, const uint8_t *data, const uint32_t data_len, rx_frame_t *frame, uint32_t *bytes_discarded_out)

  Decode a frame from raw data with bounded sync-word resynchronization.
- rx_err_t rx_frame_create_ack (rx_frame_t *frame, const uint16_t sequence)

  Create an ACK (acknowledgment) frame.
- rx_err_t rx_frame_create_nack (rx_frame_t *frame, const uint16_t sequence, uint8_t flags)

  Create a NACK (negative acknowledgment) frame.

- rx_err_t rx_frame_create_ping (rx_frame_t *frame, const uint16_t sequence, const uint8_t *payload, const uint32_t payload_len)

  Create a PING keepalive frame.
- rx_err_t rx_frame_create_pong (rx_frame_t *frame, const uint16_t sequence, const uint8_t *payload, const uint32_t payload_len)

  Create a PONG response frame.
- rx_err_t rx_frame_create_reset (rx_frame_t *frame, const uint16_t sequence)

  Create a RESET request frame.
- rx_err_t rx_frame_create_reset_ack (rx_frame_t *frame, const uint16_t sequence)

  Create a RESET_ACK confirmation frame.

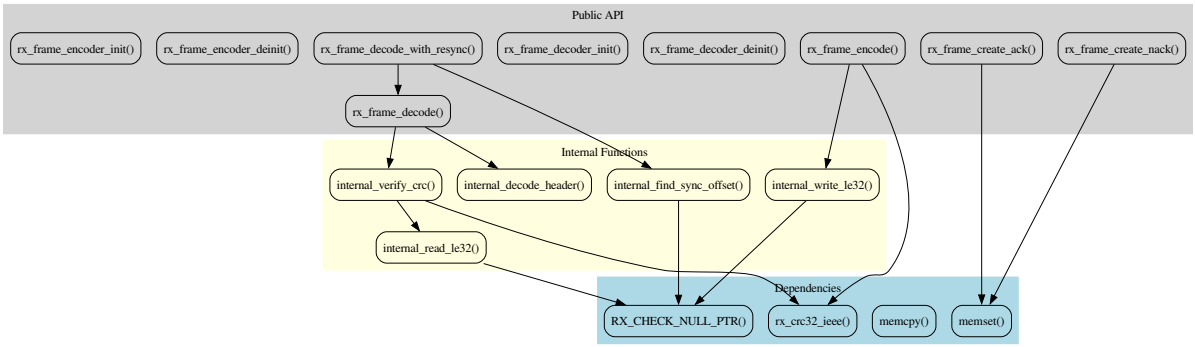
4.3.1 Detailed Description

Frame Layer Protocol Implementation.

4.3.1.1 Overview

This file implements the frame encoding and decoding operations for the SPI communication protocol between RPi5 and RX72N. The implementation is bit-exact compatible with the Go reference in star-gateway/internal/frame/.

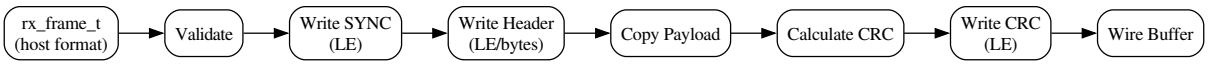
4.3.1.2 Module Architecture



4.3.1.3 Encoding Process

The encoder serializes a rx_frame_t structure to wire format:

1. Validate inputs - Check pointers, initialization state, payload size
2. Write SYNC - 2-byte marker (0x55AA) in little-endian (wire: 0xAA, 0x55)
3. Write header - SEQ, LEN in little-endian; TYPE, FLAGS as bytes
4. Copy payload - Direct byte copy (no endian conversion)
5. Compute CRC - IEEE 802.3 CRC-32 over SYNC+header+payload
6. Write CRC - 4-byte CRC in little-endian format



4.3.1.4 Decoding Process

The decoder parses wire format into a rx_frame_t structure:

1. Validate inputs - Check pointers, initialization state
2. Verify SYNC - Check for 0x55AA marker
3. Parse header - Extract SEQ, LEN, TYPE, FLAGS with endian conversion
4. Validate length - Ensure payload fits in buffer
5. Copy payload - Direct byte copy
6. Verify CRC - Compute expected CRC, compare with received

4.3.1.5 Memory Usage

Function	Stack	Notes
rx_frame_encode()	32 bytes	Variables, return address
rx_frame_decode()	32 bytes	Variables, return address
internal_decode_header()	24 bytes	Local variables
internal_verify_crc()	20 bytes	CRC values

4.3.1.6 Performance (RX72N @ 240 MHz)

Operation	Empty	64B	256B	1KB
Encode	5 us	12 us	35 us	130 us
Decode	5 us	12 us	35 us	130 us

Bottleneck: CRC calculation dominates for larger payloads. Hardware CRC reduces time by ~4x.

4.3.1.7 NASA Power of 10 Compliance

Rule	Status	Notes
1. Simple control flow	↩ [PASS]↩	Sequential processing, no goto
2. Fixed loop bounds	↩ [PASS]↩	Payload loop bounded by payload length; sync scanner bounded by k_frame_max_scan_bytes
3. No dynamic memory	↩ [PASS]↩	All buffers caller-provided

Rule	Status	Notes
4. Short functions	↩ [PASS]↩	Largest function ∼50 lines
5. Assertions	↩ [PASS]↩	All inputs validated, post-conditions checked
6. Small scope	↩ [PASS]↩	Variables at point of use
7. Check returns	↩ [PASS]↩	All errors propagated
8. Limited preprocessor	↩ [PASS]↩	Only includes
9. Restrict pointers	↩ [PASS]↩	No function pointers
10. Compiler warnings	↩ [PASS]↩	Clean build with -Wall -Wextra -Werror

4.3.1.8 SOLID Principles

Principle	Implementation
Single Responsibility	Encode/decode frames only
Open/Closed	New types/flags via enums
Liskov Substitution	N/A (no inheritance)
Interface Segregation	Separate encoder/decoder
Dependency Inversion	Depends on rx_crc abstraction

4.3.1.9 Test Coverage

See tests/test_rx_frame.c for comprehensive unit tests covering:

- Encode/decode round-trip
- CRC validation
- Boundary conditions (min/max payload)
- Error cases (NULL pointers, invalid sizes)

See also

rx_frame.h Public API and frame format documentation
rx_crc.h CRC-32 computation
star-gateway/internal/frame/ Go reference implementation

Author

Locked, Inc.

Date

2026-01-29

Copyright

Since

Version 1.0.0

Definition in file rx_frame.c.

4.3.2 Enumeration Type Documentation

4.3.2.1 byte_shift_t

enum byte_shift_t : uint8_t

Bit shift amounts for extracting bytes from multi-byte values.

Each byte position requires shifting by (position * 8) bits.

Enumerator

k_shift_byte_0	No shift needed for byte 0 (LSB)
k_shift_byte_1	Shift 8 bits for byte 1
k_shift_byte_2	Shift 16 bits for byte 2
k_shift_byte_3	Shift 24 bits for byte 3 (MSB)

Definition at line 200 of file rx_frame.c.

```
00200             : uint8_t {
00201     k_shift_byte_0 = 0,  /**< No shift needed for byte 0 (LSB) */
00202     k_shift_byte_1 = 8,  /**< Shift 8 bits for byte 1 */
00203     k_shift_byte_2 = 16, /**< Shift 16 bits for byte 2 */
00204     k_shift_byte_3 = 24, /**< Shift 24 bits for byte 3 (MSB) */
00205 } byte_shift_t;
```

4.3.2.2 frame˙crc˙init˙t

enum frame_crc_init_t : uint32_t

Initial value for CRC-32 output variable before rx_crc32_ieee() writes it.

k_frame_crc32_init is used to pre-initialize the calculated_crc output variable in internal_verify_crc() before passing its address to rx_crc32_ieee(). The value is immediately overwritten on success; the named constant satisfies the STAR no-magic-numbers policy (NASA Rule 8) and makes the intent of the initialization explicit.

Invariant

k_frame_crc32_init == 0 (matches zero-init convention)

See also

internal_verify_crc() Only user of this constant

Since

Version 1.0.0

Enumerator

k_frame_crc32_init	Pre-initialization value for CRC output variable

Definition at line 292 of file rx_frame.c.

```
00292             : uint32_t {
00293     k_frame_crc32_init = 0U, /**< Pre-initialization value for CRC output variable */
00294 } frame_crc_init_t;
```

4.3.2.3 frame˙offset˙t

enum frame_offset_t : uint8_t

Frame byte offsets used during encoding, decoding, and sync scanning.

Defines named byte-offset constants used by the frame encoder, decoder, and the bounded sync-word scanner. k_frame_offset_start marks the beginning of a frame buffer for normal aligned decoding. k↩ _frame_scan_start_offset is the index at which internal_find_sync_offset() begins its forward scan – offset 0 is omitted because rx_frame_decode() has already tested it before invoking the scanner, so the scan starts at the next candidate position.

Invariant

k_frame_scan_start_offset > k_frame_offset_start, ensuring the scanner never re-tests the byte that rx_frame_decode() already checked.

```
// Normal aligned decode starts at k_frame_offset_start (0)
rx_err_t err = rx_frame_decode(dec, data, data_len, frame);

// On sync mismatch, internal_find_sync_offset() scans from offset 1:
uint8_t sync_off = k_frame_scan_start_offset;
// ... scanner advances sync_off until sync word found or scan limit reached
```

See also

rx_frame_decode() Uses k_frame_offset_start for normal aligned decoding

internal_find_sync_offset() Begins scan at k_frame_scan_start_offset

Since

Version 1.0.0

Enumerator

k_frame_offset_start	Start offset for frame parsing
k_frame_scan_start_offset	First offset checked in internal_find_sync_offset(); offset 0 is presumed already tested by rx_frame_decode()

Definition at line 244 of file rx_frame.c.

```
00244             : uint8_t {
00245   k_frame_offset_start      = 0, /**< Start offset for frame parsing */
00246   k_frame_scan_start_offset = 1, /**< First offset checked in internal_find_sync_offset();
00247                                       offset 0 is presumed already tested by rx_frame_decode() */
00248 } frame_offset_t;
```

4.3.2.4 frame_resync_discarded_t

enum frame_resync_discarded_t : uint32_t

Byte-discard counter initial value for rx_frame_decode_with_resync().

k_bytes_discarded_none is the zero sentinel written to *bytes_discarded_out at the start of rx_frame_decode_with_resync() before any sync search. Using a named constant makes the intent auditable and prevents a raw 0 literal from appearing in the implementation (NASA Rule 8 / STAR no-magic-numbers policy).

Invariant

k_bytes_discarded_none == 0 (matches the initial no-discard state)

```
uint32_t bytes_discarded = k_bytes_discarded_none;
rx_err_t err = rx_frame_decode_with_resync(&dec, data, data_len, &frame, &bytes_discarded);
```

See also

rx_frame_decode_with_resync() Only caller of this constant

Since

Version 1.0.0

Enumerator

k_bytes_discarded_none	No bytes have been discarded; stream is aligned

Definition at line 271 of file rx_frame.c.

```
00271             : uint32_t {
00272   k_bytes_discarded_none = 0U, /**< No bytes have been discarded; stream is aligned */
00273 } frame_resync_discarded_t;
```

4.3.2.5 init_state_t

enum init_state_t : uint8_t

Initialization state values.

Enumerator

k_state_uninitialized	Object not initialized
k_state_initialized	Object initialized and ready

Definition at line 210 of file rx_frame.c.

```
00210             : uint8_t {
00211   k_state_uninitialized = 0, /**< Object not initialized */
00212   k_state_initialized   = 1, /**< Object initialized and ready */
00213 } init_state_t;
```

4.3.3 Function Documentation

4.3.3.1 internal_decode_header()

rx_err_t internal_decode_header (
const uint8_t * data,
const uint32_t data_len,
rx_frame_t * frame,
uint32_t * offset_out) [static]

Decode frame header from raw data buffer.

Extracts and validates frame header fields (sync word, sequence, length, type, flags) from the beginning of a raw data buffer. Verifies the sync word matches expected value and validates payload length is within bounds. Outputs the offset where payload data begins for subsequent processing.

Definition at line 346 of file rx_frame.c.

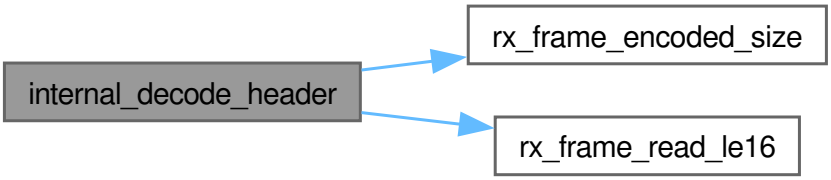
```
00350 {
00351   /* Pre-conditions: data, frame, and offset_out are always valid non-null pointers;
00352    * validated by the public callers before forwarding to this internal function. */
00353
00354   if (data_len < k_frame_min_size) {
00355     return k_rx_err_invalid_size;
00356   }
00357
00358   uint32_t offset    = k_frame_offset_start;
00359   uint16_t sync_word = rx_frame_read_le16(&data[offset]);
00360   if (sync_word != k_frame_sync_word) {
00361     return k_rx_err_protocol_error;
00362   }
00363   offset += k_frame_sync_size;
00364
00365   frame->header.sequence = rx_frame_read_le16(&data[offset]);
00366   offset += k_frame_seq_size;
00367
00368   frame->header.length = rx_frame_read_le16(&data[offset]);
00369   offset += k_frame_len_size;
00370
00371   if (frame->header.length > k_frame_max_payload) {
00372     return k_rx_err_invalid_size;
00373   }
00374
00375   uint32_t expected_size = rx_frame_encoded_size(frame->header.length);
00376   if (data_len < expected_size) {
00377     return k_rx_err_invalid_size;
00378   }
00379
00380   frame->header.type = data[offset];
00381   offset += k_frame_type_size;
```

```
00382
00383   frame->header.flags = data[offset];
00384   offset += k_frame_flags_size;
00385
00386   /* offset_out is always non-null (caller guarantees); assign unconditionally. */
00387   *offset_out = offset;
00388
00389   return k_rx_ok;
00390 }
```

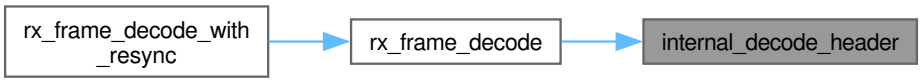
References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flags_size, k_frame_len_size, k_frame_max_payload, k_frame_min_size, k_frame_offset_start, k_frame_seq_size, k_frame_sync_size, k_frame_sync_word, k_frame_type_size, rx_frame_header_t::length, rx_frame_encoded_size(), rx_frame_read_le16(), rx_frame_header_t::sequence, and rx_frame_header_t::type.

Referenced by rx_frame_decode().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.2 internal˙find˙sync˙offset()

```
rx_err_t internal_find_sync_offset (
                const uint8_t * data,
                const uint32_t data_len,
                uint32_t * offset_out)    [static]
```

Scan a byte buffer for the frame sync word with a fixed upper bound.

Performs a linear scan of the buffer starting at offset 1 (offset 0 is presumed to have already failed sync check) looking for the 2-byte sync pattern k_frame_sync_word in little-endian wire encoding (first byte 0xAA, second byte 0x55). The scan is bounded at k_frame_max_scan_bytes to satisfy NASA Rule 2 (fixed loop upper-bounds) and prevent infinite consumption of a permanently corrupt stream.

The function stops and reports success at the first occurrence of the sync pattern. If no pattern is found within the window, k_rx_err_protocol_error is returned.

Algorithm:

1. Validate preconditions (non-null pointers, minimum buffer size).
2. Compute scan limit = min(data_len - k_frame_sync_size, k_frame_max_scan_bytes).
3. Iterate i from 1 to scan_limit inclusive: a. Read 16-bit LE value at data[i]. b. If it matches k_frame_sync_word, write i to offset_out and return k_rx_ok.
4. Return k_rx_err_protocol_error if no match found.

Precondition

data must point to a readable buffer of at least data_len bytes

data_len must be >= k_frame_sync_size (2)

Postcondition

On k_rx_ok, offset_out is in range [1, min(data_len - k_frame_sync_size, k_frame_max_scan↩ _bytes)]

On error, offset_out is unchanged

Note

Not thread-safe. Caller must ensure exclusive access to data buffer.

The loop bound is min(data_len - k_frame_sync_size, k_frame_max_scan_bytes), which is statically bounded by k_frame_max_scan_bytes = 1036 (NASA Rule 2).

Performance:

O(N) where N <= k_frame_max_scan_bytes. Each iteration performs one 16-bit LE read and one comparison. Worst case: 1036 iterations (no sync found).

Example:

```
// Buffer where offset k_frame_offset_start failed sync; sync is at index 1
uint8_t data[] = {0x00, 0xAA, 0x55, 0x01, 0x00};
uint32_t offset = k_frame_offset_start;
rx_err_t err = internal_find_sync_offset(data, sizeof(data), &offset);
if (err == k_rx_ok) {
    // offset == 1: decode from data[offset]
} else {
    // k_rx_err_protocol_error: no sync found within scan window
}
```

See also

rx_frame_decode_with_resync() Public API that calls this function

k_frame_sync_word Sync word value (0x55AA)

k_frame_max_scan_bytes Maximum scan window (1036 bytes)

Since

Version 1.0.0

Definition at line 481 of file rx_frame.c.

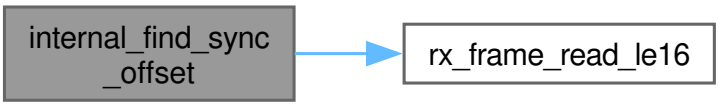
```
00482 {
00483   /* Pre-conditions: data and offset_out are valid non-null pointers (caller validated);
00484    * data_len >= k_frame_sync_size guaranteed by the caller's size check. */
00485
00486   /* Bound scan to at most k_frame_max_scan_bytes positions beyond offset 0 */
00487   uint32_t scan_limit = data_len - k_frame_sync_size;
00488   if (scan_limit > k_frame_max_scan_bytes) {
00489     scan_limit = k_frame_max_scan_bytes;
00490   }
00491
00492   /* Scan starting at k_frame_scan_start_offset; offset 0 already failed sync check */
00493   for (uint32_t i = k_frame_scan_start_offset; i <= scan_limit; i++) {
00494     const uint16_t candidate = rx_frame_read_le16(&data[i]);
```

```
00495     if (candidate == k_frame_sync_word) {
00496       *offset_out = i;
00497       return k_rx_ok;
00498     }
00499   }
00500
00501   return k_rx_err_protocol_error;
00502 }
```

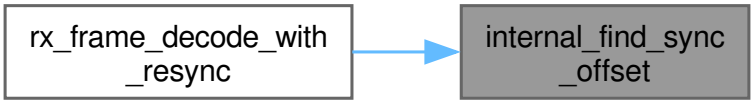
References k_frame_max_scan_bytes, k_frame_scan_start_offset, k_frame_sync_size, k_frame_sync_word, and rx_frame_read_le16().

Referenced by rx_frame_decode_with_resync().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.3 internal_read_le32()

```
rx_err_t internal_read_le32 (
                const uint8_t * buf,
                const uint32_t buf_len,
                uint32_t * out_val)    [static]
```

Read uint32 in little-endian format (for CRC-32).

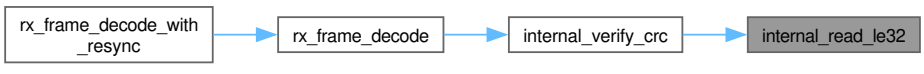
Definition at line 324 of file rx_frame.c.

```
00325 {
00326   /* Pre-conditions: buf and out_val are always valid pointers from the caller;
00327    * buf_len >= k_frame_crc_size guaranteed by the caller's bounds check. */
00328   (void)buf_len; /* suppress unused warning in non-debug builds */
00329
00330   *out_val = (uint32_t)buf[k_le32_byte_0] | ((uint32_t)buf[k_le32_byte_1] << k_shift_byte_1) |
00331              ((uint32_t)buf[k_le32_byte_2] << k_shift_byte_2) |
00332              ((uint32_t)buf[k_le32_byte_3] << k_shift_byte_3);
00333   return k_rx_ok;
00334 }
```

References k_le32_byte_0, k_le32_byte_1, k_le32_byte_2, k_le32_byte_3, k_shift_byte_1, k_shift_byte_2, and k_shift_byte_3.

Referenced by internal_verify_crc().

Here is the caller graph for this function:



4.3.3.4 internal˙verify˙crc()

```
rx_err_t internal_verify_crc (
                const uint8_t * data,
                uint32_t data_len,
                uint32_t offset,
                uint32_t * crc_out)    [static]
```

Verify CRC-32 checksum at specified offset in data buffer.

Reads the CRC-32 value stored at the specified offset in the buffer, calculates the expected CRC over the data preceding it, and compares for match. Uses IEEE 802.3 CRC-32 polynomial (0x04C11DB7), compatible with Go's crc32.ChecksumIEEE(). On success, outputs the received CRC value for caller inspection.

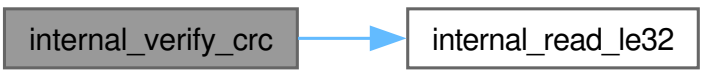
Definition at line 403 of file rx_frame.c.

```
00404 {
00405   /* Pre-conditions: data and crc_out are valid non-null pointers (caller validated);
00406    * data_len >= k_frame_min_size and offset+CRC size fits in buffer (caller validated). */
00407   uint32_t received_crc = 0;
00408   /* internal_read_le32 always returns k_rx_ok when preconditions are met. */
00409   (void)internal_read_le32(&data[offset], data_len - offset, &received_crc);
00410   uint32_t calculated_crc = k_frame_crc32_init;
00411   /* rx_crc32_ieee always returns k_rx_ok for non-null data with valid length. */
00412   (void)rx_crc32_ieee(data, offset, &calculated_crc);
00413
00414   if (received_crc != calculated_crc) {
00415     return k_rx_err_crc_mismatch;
00416   }
00417
00418   *crc_out = received_crc;
00419
00420   return k_rx_ok;
00421 }
```

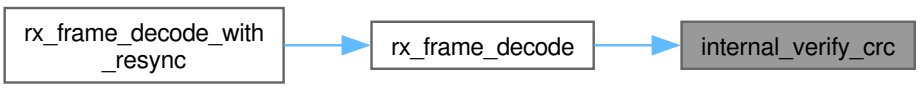
References internal_read_le32(), and k_frame_crc32_init.

Referenced by rx_frame_decode().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.5 internal_write_le32()

```
rx_err_t internal_write_le32 (
                uint8_t * buf,
                const uint32_t buf_len,
                const uint32_t val)   [static]
```

Write uint32 in little-endian format (for CRC-32).

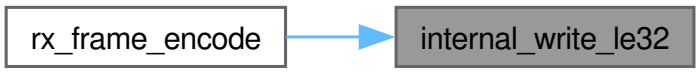
Definition at line 306 of file rx_frame.c.

```
00307 {
00308   /* Pre-conditions: buf is always a subslice of output buffer (never nullptr);
00309    * buf_len >= k_frame_crc_size is guaranteed by the caller's frame-size calculation. */
00310   (void)buf_len; /* suppress unused warning in non-debug builds */
00311
00312   buf[k_le32_byte_0] = (uint8_t)(val & k_rx_byte_mask);
00313   buf[k_le32_byte_1] = (uint8_t)((val » k_shift_byte_1) & k_rx_byte_mask);
00314   buf[k_le32_byte_2] = (uint8_t)((val » k_shift_byte_2) & k_rx_byte_mask);
00315   buf[k_le32_byte_3] = (uint8_t)((val » k_shift_byte_3) & k_rx_byte_mask);
00316   return k_rx_ok;
00317 }
```

References k_le32_byte_0, k_le32_byte_1, k_le32_byte_2, k_le32_byte_3, k_rx_byte_mask, k_shift_byte_1, k_shift_byte_2, and k_shift_byte_3.

Referenced by rx_frame_encode().

Here is the caller graph for this function:



4.3.3.6 rx_frame_create_ack()

```
rx_err_t rx_frame_create_ack (
                rx_frame_t * frame,
                const uint16_t sequence)   [nodiscard]
```

Create an ACK (acknowledgment) frame.

Create ACK frame for given sequence.

Constructs a frame with type=ACK, empty payload, and the specified sequence number. The ACK response frames are used by the receiver to confirm successful reception of a command or data frame from the sender.

Definition at line 820 of file rx_frame.c.

```
00821 {
00822   if (frame == nullptr) {
00823     return k_rx_err_invalid_arg;
00824   }
00825
00826   *frame                = (rx_frame_t){};
00827   frame->header.sequence = sequence;
00828   frame->header.length   = 0;
00829   frame->header.type     = k_frame_type_ack;
00830   frame->header.flags    = k_frame_flag_none;
00831
00832   /* Post-condition: type == k_frame_type_ack guaranteed by the direct assignment above. */
00833   return k_rx_ok;
00834 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_type_ack, rx_frame_header_t::length, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.3.3.7 rx˙frame˙create˙nack()

```
rx_err_t rx_frame_create_nack (
                rx_frame_t * frame,
                const uint16_t sequence,
                uint8_t flags)   [nodiscard]
```

Create a NACK (negative acknowledgment) frame.

Create NACK frame for given sequence.

Constructs a frame with type=NACK, empty payload, the specified sequence number, and error/status flags. NACK frames are sent by the receiver to indicate that a frame was not processed successfully due to an error condition (specified in flags).

Definition at line 845 of file rx_frame.c.

```
00846 {
00847   if (frame == nullptr) {
00848     return k_rx_err_invalid_arg;
00849   }
00850
00851   *frame                = (rx_frame_t){};
00852   frame->header.sequence = sequence;
00853   frame->header.length   = 0;
00854   frame->header.type     = k_frame_type_nack;
00855   frame->header.flags    = flags;
00856
00857   /* Post-condition: type == k_frame_type_nack guaranteed by the direct assignment above. */
00858   return k_rx_ok;
00859 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_type_nack, rx_frame_header_t::length, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.3.3.8 rx˙frame˙create˙ping()

```
rx_err_t rx_frame_create_ping (
                rx_frame_t * frame,
                const uint16_t sequence,
                const uint8_t * payload,
                const uint32_t payload_len)   [nodiscard]
```

Create a PING keepalive frame.

Create PING frame for keepalive/heartbeat.

Initializes frame as a PING request for connection liveness checks. Receiver should respond with a PONG frame echoing the payload. Payload is optional; a 4-byte LE counter is typical.

Precondition

frame !=nullptr

payload != NULL when payload_len > 0

Postcondition

frame->header.type == k_frame_type_ping

frame->header.length == payload_len

Note

Stateless; safe to call from any context.

See also

rx_frame_create_pong() Corresponding response creator

Since

Version 1.0.0

Definition at line 880 of file rx_frame.c.

```
00884 {
00885   if (frame == nullptr) {
00886     return k_rx_err_invalid_arg;
00887   }
00888
00889   if (payload_len > 0 && payload == nullptr) {
00890     return k_rx_err_invalid_arg;
00891   }
00892
00893   if (payload_len > k_frame_max_payload) {
00894     return k_rx_err_invalid_size;
00895   }
00896
00897   *frame                 = (rx_frame_t){};
00898   frame->header.sequence = sequence;
00899   frame->header.length   = (uint16_t)payload_len;
00900   frame->header.type     = k_frame_type_ping;
00901   frame->header.flags    = k_frame_flag_none;
00902
00903   if (payload_len > 0) {
00904     for (uint32_t i = 0U; i < payload_len; i++) {
00905       frame->payload[i] = payload[i];
00906     }
00907   }
00908
00909   /* Post-condition: type == k_frame_type_ping guaranteed by the direct assignment above. */
00910   return k_rx_ok;
00911 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_max_payload, k_frame_type_ping, rx_frame_header_t::length, rx_frame_t::payload, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.3.3.9 rx˙frame˙create˙pong()

```
rx_err_t rx_frame_create_pong (
                rx_frame_t * frame,
                const uint16_t sequence,
                const uint8_t * payload,
                const uint32_t payload_len)   [nodiscard]
```

Create a PONG response frame.

Create PONG frame (response to PING).

Initializes frame as a PONG response to a received PING. Echoes the PING payload so the sender can validate round-trip data.

Precondition

frame !=nullptr

payload != NULL when payload_len > 0

Postcondition

frame->header.type == k_frame_type_pong

frame->header.length == payload_len

Note

Stateless; safe to call from any context.

See also

rx_frame_create_ping() Corresponding request creator

Since

Version 1.0.0

Definition at line 931 of file rx_frame.c.

```
00935 {
00936   if (frame == nullptr) {
00937     return k_rx_err_invalid_arg;
00938   }
00939
00940   if (payload_len > 0 && payload == nullptr) {
00941     return k_rx_err_invalid_arg;
00942   }
00943
00944   if (payload_len > k_frame_max_payload) {
00945     return k_rx_err_invalid_size;
00946   }
00947
00948   *frame                 = (rx_frame_t){};
00949   frame->header.sequence = sequence;
00950   frame->header.length   = (uint16_t)payload_len;
00951   frame->header.type     = k_frame_type_pong;
00952   frame->header.flags    = k_frame_flag_none;
00953
00954   if (payload_len > 0) {
00955     for (uint32_t i = 0U; i < payload_len; i++) {
00956       frame->payload[i] = payload[i];
00957     }
00958   }
00959
00960   /* Post-condition: type == k_frame_type_pong guaranteed by the direct assignment above. */
00961   return k_rx_ok;
00962 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_max_payload, k_frame_type_pong, rx_frame_header_t::length, rx_frame_t::payload, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.3.3.10 rx_frame_create_reset()

```
rx_err_t rx_frame_create_reset (
                rx_frame_t * frame,
                const uint16_t sequence)   [nodiscard]
```

Create a RESET request frame.

Create RESET frame to reset communication state.

Initializes frame as a RESET request that asks the peer to reset communication state (sequence numbers, buffers). The receiver should respond with a RESET_ACK frame. Payload is always empty.

Precondition

frame !=nullptr

Caller has determined that a reset is necessary

Postcondition

frame->header.type == k_frame_type_reset

frame->header.length == 0

Note

Stateless; safe to call from any context.

Warning

Reset clears all sequence number state on both sides.

See also

rx_frame_create_reset_ack() Corresponding acknowledgment creator

Since

Version 1.0.0

Definition at line 984 of file rx_frame.c.

```
00985 {
00986   if (frame == nullptr) {
00987     return k_rx_err_invalid_arg;
00988   }
00989
00990   *frame                = (rx_frame_t){};
00991   frame->header.sequence = sequence;
00992   frame->header.length   = 0;
00993   frame->header.type     = k_frame_type_reset;
00994   frame->header.flags    = k_frame_flag_none;
00995
00996   /* Post-condition: type == k_frame_type_reset guaranteed by the direct assignment above. */
00997   return k_rx_ok;
00998 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_type_reset, rx_frame_header_t::length, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.3.3.11 rx˙frame˙create˙reset˙ack()

```
rx_err_t rx_frame_create_reset_ack (
                rx_frame_t * frame,
                const uint16_t sequence)   [nodiscard]
```

Create a RESET_ACK confirmation frame.

Create RESET_ACK frame (response to RESET).

Initializes frame as a RESET_ACK response to a received RESET request. Confirms that the receiver has completed its reset procedure. Payload is always empty; sequence matches the RESET request.

Precondition

frame !=nullptr

A RESET frame was received and processed

Postcondition

frame->header.type == k_frame_type_reset_ack

frame->header.length == 0

Note

Stateless; safe to call from any context.

See also

rx_frame_create_reset() Corresponding request creator

Since

Version 1.0.0

Definition at line 1019 of file rx_frame.c.

```
01020 {
01021   if (frame == nullptr) {
01022     return k_rx_err_invalid_arg;
01023   }
01024
01025   *frame                = (rx_frame_t){};
01026   frame->header.sequence = sequence;
01027   frame->header.length   = 0;
01028   frame->header.type     = k_frame_type_reset_ack;
01029   frame->header.flags    = k_frame_flag_none;
01030
01031   /* Post-condition: type == k_frame_type_reset_ack guaranteed by the direct assignment above. */
01032   return k_rx_ok;
01033 }
```

References rx_frame_header_t::flags, rx_frame_t::header, k_frame_flag_none, k_frame_type_reset_ack, rx_frame_header_t::length, rx_frame_header_t::sequence, and rx_frame_header_t::type.

4.3.3.12 rx_frame_decode()

```
rx_err_t rx_frame_decode (
        const rx_frame_decoder_t * dec,
        const uint8_t * data,
        const uint32_t data_len,
        rx_frame_t * frame)    [nodiscard]
```

Decode a frame from raw data.

Decode wire format to frame.

Validates the decoder state, extracts the frame header, copies the payload, and verifies the CRC-32 checksum.

Note

This function performs validation at multiple points:

- Null pointer checks on all parameters
- Decoder initialization check
- Header parsing via internal_decode_header()
- CRC verification via internal_verify_crc() (may surface CRC backend errors)

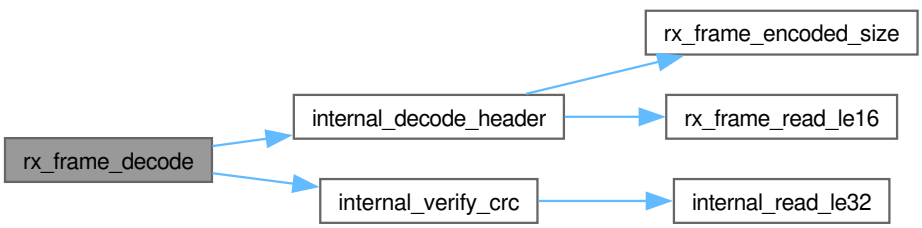
Definition at line 671 of file rx_frame.c.

```
00675 {
00676   if (dec == nullptr || data == nullptr || frame == nullptr) {
00677     return k_rx_err_invalid_arg;
00678   }
00679
00680   if (dec->initialized != k_state_initialized) {
00681     return k_rx_err_invalid_state;
00682   }
00683
00684   uint32_t offset;
00685   rx_err_t err = internal_decode_header(data, data_len, frame, &offset);
00686   if (err != k_rx_ok) {
00687     return err;
00688   }
00689
00690   /* Read PAYLOAD */
00691   if (frame->header.length > 0) {
00692     for (uint32_t i = 0U; i < (uint32_t)frame->header.length; i++) {
00693       frame->payload[i] = data[offset + i];
00694     }
00695     offset += frame->header.length;
00696   }
00697
00698   err = internal_verify_crc(data, data_len, offset, &frame->crc);
00699   if (err != k_rx_ok) {
00700     return err;
00701   }
00702   return k_rx_ok;
00703 }
```

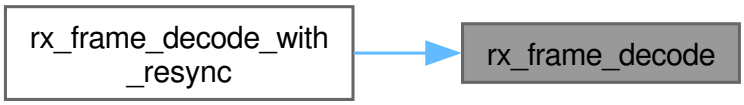
References rx_frame_t::crc, rx_frame_t::header, rx_frame_decoder_t::initialized, internal_decode_header(), internal_verify_crc(), k_state_initialized, rx_frame_header_t::length, and rx_frame_t::payload.

Referenced by rx_frame_decode_with_resync().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.13 rx˙frame˙decode˙with˙resync()

```
rx_err_t rx_frame_decode_with_resync (
                const rx_frame_decoder_t * dec,
                const uint8_t * data,
                const uint32_t data_len,
                rx_frame_t * frame,
                uint32_t * bytes_discarded_out)   [nodiscard]
```

Decode a frame from raw data with bounded sync-word resynchronization.

Decode wire format to frame with bounded sync-word resynchronization.

Provides stream recovery when the byte stream has become misaligned due to a dropped or inserted byte. On sync mismatch, scans forward up to k_frame_max_scan_bytes positions for the next occurrence of the sync word, discards the leading bytes, and reattempts decoding from that position.

This function is safe for repeated calls on a stream: the caller must advance its stream read pointer by *bytes_discarded_out on k_rx_ok and also on k_rx_err_crc_mismatch (any case where bytes_↩ discarded_out is set) to avoid re-processing discarded bytes in subsequent calls.

Recovery algorithm:

1. Attempt rx_frame_decode() on data[0..data_len-1].
2. If result is not k_rx_err_protocol_error, return immediately.
3. Call internal_find_sync_offset() to locate next sync word in data[1..].
4. If not found, return k_rx_err_protocol_error with *bytes_discarded_out = 0.
5. Decode from data[sync_offset..data_len-1] using the trimmed sub-buffer.
6. Write sync_offset to *bytes_discarded_out and return result.

Precondition

dec must be initialized via rx_frame_decoder_init()

data must point to a buffer of at least data_len bytes

frame must point to a valid rx_frame_t storage location (not NULL)

bytes_discarded_out must point to a valid uint32_t storage location (not NULL)

Postcondition

On k_rx_ok, *bytes_discarded_out == bytes skipped to reach sync word

On k_rx_ok, frame contains a fully verified decoded frame

On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer

Note

Parameter order follows the canonical convention: decoder input -> data input -> frame output -> diagnostic output (matching rx_frame_decode()).

Not thread-safe. Caller must synchronize access.

The scan is bounded at k_frame_max_scan_bytes (NASA Rule 2).

Log *bytes_discarded_out > 0 as a stream-health diagnostic warning.

Warning

Caller must advance stream read pointer by *bytes_discarded_out after any call that returns k_↩ rx_ok or k_rx_err_crc_mismatch to avoid infinite re-scan; *bytes_discarded_out is unmodified only when k_rx_err_invalid_arg is returned.

Example:

```
uint32_t discarded = k_bytes_discarded_none;
rx_err_t err = rx_frame_decode_with_resync(dec, data, data_len, &frame, &discarded);
if (err == k_rx_ok) {
    // Advance stream read pointer past any discarded bytes
    stream_read_ptr += discarded;
} else if (err == k_rx_err_crc_mismatch) {
    // Sync was found but CRC failed -- still advance to avoid infinite re-scan
    stream_read_ptr += discarded;
    rx_log_error(TAG, "sync found but CRC failed");
} else {
    rx_log_error(TAG, "no sync word within scan window");
}
```

See also

rx_frame_decode() Simple decode without stream recovery

internal_find_sync_offset() Bounded sync word scanner

k_frame_max_scan_bytes Scan window upper bound

Since

Version 1.0.0

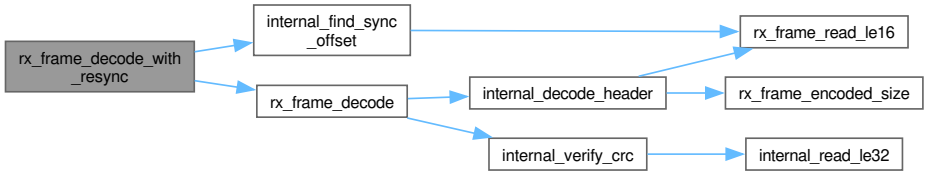
Definition at line 769 of file rx_frame.c.

```
00774 {
00775   if (dec == nullptr || data == nullptr || frame == nullptr || bytes_discarded_out == nullptr) {
00776     return k_rx_err_invalid_arg;
00777   }
00778
00779   *bytes_discarded_out = k_bytes_discarded_none;
00780
00781   /* Fast path: attempt aligned decode first */
00782   rx_err_t err = rx_frame_decode(dec, data, data_len, frame);
00783   if (err != k_rx_err_protocol_error) {
00784     /* Either success, or an error other than sync mismatch (CRC, size, etc.) */
00785     return err;
00786   }
00787
00788   /* Sync word not at offset 0: scan forward for next sync word */
00789   uint32_t sync_offset = (uint32_t)k_frame_offset_start;
00790   err                  = internal_find_sync_offset(data, data_len, &sync_offset);
00791   if (err != k_rx_ok) {
00792     /* No sync word found within bounded window */
00793     return k_rx_err_protocol_error;
00794   }
00795
00796   /* Report discarded bytes now: caller needs this to advance past the sync
00797    * even if the subsequent decode fails (e.g. CRC mismatch at new offset) */
00798   *bytes_discarded_out = sync_offset;
```

```
00799
00800   /* Reattempt decode from the discovered sync offset */
00801   err = rx_frame_decode(dec, &data[sync_offset], data_len - sync_offset, frame);
00802
00803   return err;
00804 }
```

References internal_find_sync_offset(), k_bytes_discarded_none, k_frame_offset_start, and rx_frame_decode().

Here is the call graph for this function:



4.3.3.14 rx_frame_decoder_deinit()

```
rx_err_t rx_frame_decoder_deinit (
                rx_frame_decoder_t * dec)   [nodiscard]
```

Deinitialize a frame decoder instance.

Deinitialize frame decoder.

Marks the decoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Definition at line 645 of file rx_frame.c.

```
00646 {
00647   if (dec == nullptr) {
00648     return k_rx_err_invalid_arg;
00649   }
00650
00651   dec->initialized = k_state_uninitialized;
00652   /* Post-condition: dec->initialized == k_state_uninitialized is guaranteed by the
00653    * direct assignment above. */
00654   return k_rx_ok;
00655 }
```

References rx_frame_decoder_t::initialized, and k_state_uninitialized.

4.3.3.15 rx_frame_decoder_init()

```
rx_err_t rx_frame_decoder_init (
                rx_frame_decoder_t * dec)   [nodiscard]
```

Initialize a frame decoder instance.

Initialize frame decoder.

Prepares the decoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Definition at line 625 of file rx_frame.c.

```
00626 {
00627   if (dec == nullptr) {
00628     return k_rx_err_invalid_arg;
00629   }
00630
00631   dec->initialized = k_state_initialized;
00632   /* Post-condition: dec->initialized == k_state_initialized is guaranteed by the
00633    * direct assignment above. */
00634   return k_rx_ok;
00635 }
```

References rx_frame_decoder_t::initialized, and k_state_initialized.

4.3.3.16 rx_frame_encode()

```
rx_err_t rx_frame_encode (
        const rx_frame_encoder_t * enc,
        const rx_frame_t * frame,
        uint8_t * output,
        uint32_t * output_len)    [nodiscard]
```

Encode frame to wire format.

Serializes frame to wire format with SYNC, header, payload, and CRC-32. All multi-byte fields are little-endian (SYNC, SEQ, LEN, CRC-32).

Parameters

in	enc	Encoder handle
in	frame	Frame to encode
out	output	Output buffer (min k_frame_min_size + payload bytes)
out	output_len	Actual number of bytes written

Returns

k_rx_ok on success

k_rx_err_invalid_arg if any pointer is nullptr

k_rx_err_invalid_state if encoder not initialized

k_rx_err_invalid_size if payload exceeds max

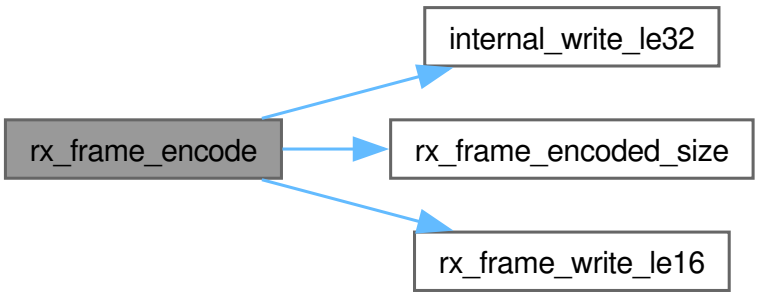
Definition at line 549 of file rx_frame.c.

```
00553 {
00554   if (enc == nullptr || frame == nullptr || output == nullptr || output_len == nullptr) {
00555     return k_rx_err_invalid_arg;
00556   }
00557
00558   if (enc->initialized != k_state_initialized) {
00559     return k_rx_err_invalid_state;
00560   }
00561
00562   /* Validate payload size */
00563   if (frame->header.length > k_frame_max_payload) {
00564     return k_rx_err_invalid_size;
00565   }
00566
00567   /* Calculate total frame size */
00568   uint32_t frame_size = rx_frame_encoded_size(frame->header.length);
00569   uint32_t offset     = k_frame_offset_start;
00570
00571   /* Write SYNC word (little-endian: wire bytes 0xAA, 0x55) */
00572   rx_frame_write_le16(&output[offset], k_frame_sync_word);
00573   offset += k_frame_sync_size;
00574
00575   /* Write SEQ (little-endian) */
00576   rx_frame_write_le16(&output[offset], frame->header.sequence);
00577   offset += k_frame_seq_size;
00578
00579   /* Write LEN (little-endian) */
00580   rx_frame_write_le16(&output[offset], frame->header.length);
00581   offset += k_frame_len_size;
00582
00583   /* Write TYPE (1 byte) */
00584   output[offset] = frame->header.type;
00585   offset += k_frame_type_size;
00586
00587   /* Write FLAGS (1 byte) */
00588   output[offset] = frame->header.flags;
00589   offset += k_frame_flags_size;
00590
```

```
00591   /* Write PAYLOAD */
00592   if (frame->header.length > 0) {
00593     for (uint32_t i = 0U; i < (uint32_t)frame->header.length; i++) {
00594       output[offset + i] = frame->payload[i];
00595     }
00596     offset += frame->header.length;
00597   }
00598
00599   /* Calculate CRC-32 over SYNC + Header + Payload (IEEE 802.3 polynomial).
00600    * rx_crc32_ieee always returns k_rx_ok for non-null buffer with valid length. */
00601   uint32_t crc = k_frame_crc32_init;
00602   (void)rx_crc32_ieee(output, offset, &crc);
00603
00604   /* Write CRC-32 (little-endian to match IEEE 802.3 LSB-first order).
00605    * internal_write_le32 always returns k_rx_ok when preconditions are met. */
00606   (void)internal_write_le32(&output[offset], frame_size - offset, crc);
00607
00608   *output_len = frame_size;
00609   return k_rx_ok;
00610 }
```

References rx_frame_header_t::flags, rx_frame_t::header, rx_frame_encoder_t::initialized, internal_write_le32(), k_frame_crc32_init, k_frame_flags_size, k_frame_len_size, k_frame_max_payload, k_frame_offset_start, k_frame_seq_size, k_frame_sync_size, k_frame_sync_word, k_frame_type_size, k_state_initialized, rx_frame_header_t::length, rx_frame_t::payload, rx_frame_encoded_size(), rx_frame_write_le16(), rx_frame_header_t::sequence, and rx_frame_header_t::type.

Here is the call graph for this function:



4.3.3.17 rx˙frame˙encoder˙deinit()

```
rx_err_t rx_frame_encoder_deinit (
                rx_frame_encoder_t * enc)   [nodiscard]
```

Deinitialize a frame encoder instance.

Deinitialize frame encoder.

Marks the encoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Definition at line 537 of file rx_frame.c.

```
00538 {
00539   if (enc == nullptr) {
00540     return k_rx_err_invalid_arg;
00541   }
00542
00543   enc->initialized = k_state_uninitialized;
00544   /* Post-condition: enc->initialized == k_state_uninitialized is guaranteed by the
00545    * direct assignment above. */
00546   return k_rx_ok;
00547 }
```

References rx_frame_encoder_t::initialized, and k_state_uninitialized.

4.3.3.18 rx_frame_encoder_init()

```
rx_err_t rx_frame_encoder_init (
                rx_frame_encoder_t * enc)    [nodiscard]
```

Initialize a frame encoder instance.

Initialize frame encoder.

Prepares the encoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Definition at line 517 of file rx_frame.c.

```
00518 {
00519   if (enc == nullptr) {
00520     return k_rx_err_invalid_arg;
00521   }
00522
00523   enc->initialized = k_state_initialized;
00524   /* Post-condition: enc->initialized == k_state_initialized is guaranteed by the
00525    * direct assignment above. */
00526   return k_rx_ok;
00527 }
```

References rx_frame_encoder_t::initialized, and k_state_initialized.

4.4 rx_frame.c

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_frame.c
00003  * @brief Frame Layer Protocol Implementation
00004  *
00005  * @details
00006  * ## Overview
00007  *
00008  * This file implements the frame encoding and decoding operations for the
00009  * SPI communication protocol between RPi5 and RX72N. The implementation is
00010  * bit-exact compatible with the Go reference in star-gateway/internal/frame/.
00011  *
00012  * ## Module Architecture
00013  *
00014  * @dot
00015  * digraph frame_arch {
00016  *   rankdir=TB;
00017  *   node [shape=box, style=rounded];
00018  *
00019  *   subgraph cluster_public {
00020  *     label="Public API";
00021  *     style=filled;
00022  *     color=lightgrey;
00023  *     encoder_init [label="rx_frame_encoder_init()"];
00024  *     encoder_deinit [label="rx_frame_encoder_deinit()"];
00025  *     encode [label="rx_frame_encode()"];
00026  *     decoder_init [label="rx_frame_decoder_init()"];
00027  *     decoder_deinit [label="rx_frame_decoder_deinit()"];
00028  *     decode [label="rx_frame_decode()"];
00029  *     decode_resync [label="rx_frame_decode_with_resync()"];
00030  *     create_ack [label="rx_frame_create_ack()"];
00031  *     create_nack [label="rx_frame_create_nack()"];
00032  *   }
00033  *
00034  *   subgraph cluster_internal {
00035  *     label="Internal Functions";
00036  *     style=filled;
00037  *     color=lightyellow;
00038  *     write_le32 [label="internal_write_le32()"];
00039  *     read_le32 [label="internal_read_le32()"];
00040  *     decode_header [label="internal_decode_header()"];
00041  *     verify_crc [label="internal_verify_crc()"];
00042  *     find_sync [label="internal_find_sync_offset()"];
00043  *   }
00044  *
```

```
00045 *   subgraph cluster_deps {
00046 *     label="Dependencies";
00047 *     style=filled;
00048 *     color=lightblue;
00049 *     rx_crc [label="rx_crc32_ieee()"];
00050 *     rx_check [label="RX_CHECK_NULL_PTR()"];
00051 *     memcpy_fn [label="memcpy()"];
00052 *     memset_fn [label="memset()"];
00053 *   }
00054 *
00055 *   encode -> write_le32;
00056 *   encode -> rx_crc;
00057 *   decode -> decode_header;
00058 *   decode -> verify_crc;
00059 *   decode_resync -> decode;
00060 *   decode_resync -> find_sync;
00061 *   find_sync -> rx_check;
00062 *   decode_header -> rx_crc [style=invis];
00063 *   verify_crc -> read_le32;
00064 *   verify_crc -> rx_crc;
00065 *   write_le32 -> rx_check;
00066 *   read_le32 -> rx_check;
00067 *   create_ack -> memset_fn;
00068 *   create_nack -> memset_fn;
00069 * }
00070 * @enddot
00071 *
00072 * ## Encoding Process
00073 *
00074 * The encoder serializes a rx_frame_t structure to wire format:
00075 *
00076 * 1. **Validate inputs** - Check pointers, initialization state, payload size
00077 * 2. **Write SYNC** - 2-byte marker (0x55AA) in little-endian (wire: 0xAA, 0x55)
00078 * 3. **Write header** - SEQ, LEN in little-endian; TYPE, FLAGS as bytes
00079 * 4. **Copy payload** - Direct byte copy (no endian conversion)
00080 * 5. **Compute CRC** - IEEE 802.3 CRC-32 over SYNC+header+payload
00081 * 6. **Write CRC** - 4-byte CRC in little-endian format
00082 *
00083 * @dot
00084 * digraph encode_flow {
00085 *   rankdir=LR;
00086 *   node [shape=box, style=rounded];
00087 *
00088 *   input [label="rx_frame_t\n(host format)"];
00089 *   validate [label="Validate"];
00090 *   sync [label="Write SYNC\n(LE)"];
00091 *   header [label="Write Header\n(LE/bytes)"];
00092 *   payload [label="Copy Payload"];
00093 *   crc_calc [label="Calculate CRC"];
00094 *   crc_write [label="Write CRC\n(LE)"];
00095 *   output [label="Wire Buffer"];
00096 *
00097 *   input -> validate -> sync -> header -> payload -> crc_calc -> crc_write -> output;
00098 * }
00099 * @enddot
00100 *
00101 * ## Decoding Process
00102 *
00103 * The decoder parses wire format into a rx_frame_t structure:
00104 *
00105 * 1. **Validate inputs** - Check pointers, initialization state
00106 * 2. **Verify SYNC** - Check for 0x55AA marker
00107 * 3. **Parse header** - Extract SEQ, LEN, TYPE, FLAGS with endian conversion
00108 * 4. **Validate length** - Ensure payload fits in buffer
00109 * 5. **Copy payload** - Direct byte copy
00110 * 6. **Verify CRC** - Compute expected CRC, compare with received
00111 *
00112 * ## Memory Usage
00113 *
00114 * | Function | Stack | Notes |
00115 * |----------|-------|-------|
00116 * | rx_frame_encode() | 32 bytes | Variables, return address |
00117 * | rx_frame_decode() | 32 bytes | Variables, return address |
00118 * | internal_decode_header() | 24 bytes | Local variables |
00119 * | internal_verify_crc() | 20 bytes | CRC values |
00120 *
00121 * ## Performance (RX72N @ 240 MHz)
00122 *
00123 * | Operation | Empty | 64B | 256B | 1KB |
00124 * |-----------|-------|-----|------|-----|
00125 * | Encode | 5 us | 12 us | 35 us | 130 us |
00126 * | Decode | 5 us | 12 us | 35 us | 130 us |
00127 *
00128 * **Bottleneck:** CRC calculation dominates for larger payloads.
00129 * Hardware CRC reduces time by ~4x.
00130 *
00131 * ## NASA Power of 10 Compliance
```

```
00132  *
00133  * | Rule | Status | Notes |
00134  * |------|--------|-------|
00135  * | 1. Simple control flow | [PASS] | Sequential processing, no goto |
00136  * | 2. Fixed loop bounds | [PASS] | Payload loop bounded by payload length; sync scanner bounded by
       k_frame_max_scan_bytes |
00137  * | 3. No dynamic memory | [PASS] | All buffers caller-provided |
00138  * | 4. Short functions | [PASS] | Largest function ~50 lines |
00139  * | 5. Assertions | [PASS] | All inputs validated, post-conditions checked |
00140  * | 6. Small scope | [PASS] | Variables at point of use |
00141  * | 7. Check returns | [PASS] | All errors propagated |
00142  * | 8. Limited preprocessor | [PASS] | Only includes |
00143  * | 9. Restrict pointers | [PASS] | No function pointers |
00144  * | 10. Compiler warnings | [PASS] | Clean build with -Wall -Wextra -Werror |
00145  *
00146  * ## SOLID Principles
00147  *
00148  * | Principle | Implementation |
00149  * |-----------|----------------|
00150  * | **Single Responsibility** | Encode/decode frames only |
00151  * | **Open/Closed** | New types/flags via enums |
00152  * | **Liskov Substitution** | N/A (no inheritance) |
00153  * | **Interface Segregation** | Separate encoder/decoder |
00154  * | **Dependency Inversion** | Depends on rx_crc abstraction |
00155  *
00156  * ## Test Coverage
00157  *
00158  * See tests/test_rx_frame.c for comprehensive unit tests covering:
00159  * - Encode/decode round-trip
00160  * - CRC validation
00161  * - Boundary conditions (min/max payload)
00162  * - Error cases (NULL pointers, invalid sizes)
00163  *
00164  * @see rx_frame.h Public API and frame format documentation
00165  * @see rx_crc.h CRC-32 computation
00166  * @see star-gateway/internal/frame/ Go reference implementation
00167  *
00168  * @author Locked, Inc.
00169  * @date 2026-01-29
00170  * @copyright Copyright (c) 2026 Locked Inc.
00171  * SPDX-License-Identifier: MIT
00172  * @since Version 1.0.0
00173  */
00174
00175 #include "rx_frame.h"
00176
00177 #include <string.h>
00178
00179 #include "rx_check.h"
00180 #include "rx_crc.h"
00181
00182 /*
       ================================================================================
00183  * Module State
00184  *
       ================================================================================
00185  */
00186
00187 /*
       ================================================================================
00188  * Byte Serialization Constants
00189  *
00190  * Named constants for byte manipulation in serialization functions.
00191  * Eliminates magic numbers and documents the bit/byte operations.
00192  *
       ================================================================================
00193  */
00194
00195 /**
00196  * @brief Bit shift amounts for extracting bytes from multi-byte values
00197  *
00198  * Each byte position requires shifting by (position * 8) bits.
00199  */
00200 typedef enum : uint8_t {
00201   k_shift_byte_0 = 0,  /**< No shift needed for byte 0 (LSB) */
00202   k_shift_byte_1 = 8,  /**< Shift 8 bits for byte 1 */
00203   k_shift_byte_2 = 16, /**< Shift 16 bits for byte 2 */
00204   k_shift_byte_3 = 24, /**< Shift 24 bits for byte 3 (MSB) */
00205 } byte_shift_t;
00206
00207 /**
00208  * @brief Initialization state values
00209  */
00210 typedef enum : uint8_t {
00211   k_state_uninitialized = 0, /**< Object not initialized */
00212   k_state_initialized   = 1, /**< Object initialized and ready */
00213 } init_state_t;
```

```
00214
00215 /**
00216  * @enum frame_offset_t
00217  * @brief Frame byte offsets used during encoding, decoding, and sync scanning
00218  *
00219  * @details
00220  * Defines named byte-offset constants used by the frame encoder, decoder, and
00221  * the bounded sync-word scanner. k_frame_offset_start marks the beginning of a
00222  * frame buffer for normal aligned decoding. k_frame_scan_start_offset is the
00223  * index at which internal_find_sync_offset() begins its forward scan -- offset 0
00224  * is omitted because rx_frame_decode() has already tested it before invoking
00225  * the scanner, so the scan starts at the next candidate position.
00226  *
00227  * @invariant k_frame_scan_start_offset > k_frame_offset_start, ensuring the
00228  *            scanner never re-tests the byte that rx_frame_decode() already checked.
00229  *
00230  * @code{.c}
00231  * // Normal aligned decode starts at k_frame_offset_start (0)
00232  * rx_err_t err = rx_frame_decode(dec, data, data_len, frame);
00233  *
00234  * // On sync mismatch, internal_find_sync_offset() scans from offset 1:
00235  * uint8_t sync_off = k_frame_scan_start_offset;
00236  * // ... scanner advances sync_off until sync word found or scan limit reached
00237  * @endcode
00238  *
00239  * @see rx_frame_decode() Uses k_frame_offset_start for normal aligned decoding
00240  * @see internal_find_sync_offset() Begins scan at k_frame_scan_start_offset
00241  *
00242  * @since Version 1.0.0
00243  */
00244 typedef enum : uint8_t {
00245   k_frame_offset_start      = 0, /**< Start offset for frame parsing */
00246   k_frame_scan_start_offset = 1, /**< First offset checked in internal_find_sync_offset();
00247                                       offset 0 is presumed already tested by rx_frame_decode() */
00248 } frame_offset_t;
00249
00250 /**
00251  * @enum frame_resync_discarded_t
00252  * @brief Byte-discard counter initial value for rx_frame_decode_with_resync()
00253  *
00254  * @details
00255  * k_bytes_discarded_none is the zero sentinel written to *bytes_discarded_out
00256  * at the start of rx_frame_decode_with_resync() before any sync search.  Using
00257  * a named constant makes the intent auditable and prevents a raw 0 literal from
00258  * appearing in the implementation (NASA Rule 8 / STAR no-magic-numbers policy).
00259  *
00260  * @invariant k_bytes_discarded_none == 0 (matches the initial no-discard state)
00261  *
00262  * @code{.c}
00263  * uint32_t bytes_discarded = k_bytes_discarded_none;
00264  * rx_err_t err = rx_frame_decode_with_resync(&dec, data, data_len, &frame, &bytes_discarded);
00265  * @endcode
00266  *
00267  * @see rx_frame_decode_with_resync() Only caller of this constant
00268  *
00269  * @since Version 1.0.0
00270  */
00271 typedef enum : uint32_t {
00272   k_bytes_discarded_none = 0U, /**< No bytes have been discarded; stream is aligned */
00273 } frame_resync_discarded_t;
00274
00275 /**
00276  * @enum frame_crc_init_t
00277  * @brief Initial value for CRC-32 output variable before rx_crc32_ieee() writes it
00278  *
00279  * @details
00280  * k_frame_crc32_init is used to pre-initialize the calculated_crc output
00281  * variable in internal_verify_crc() before passing its address to
00282  * rx_crc32_ieee(). The value is immediately overwritten on success; the
00283  * named constant satisfies the STAR no-magic-numbers policy (NASA Rule 8)
00284  * and makes the intent of the initialization explicit.
00285  *
00286  * @invariant k_frame_crc32_init == 0 (matches zero-init convention)
00287  *
00288  * @see internal_verify_crc() Only user of this constant
00289  *
00290  * @since Version 1.0.0
00291  */
00292 typedef enum : uint32_t {
00293   k_frame_crc32_init = 0U, /**< Pre-initialization value for CRC output variable */
00294 } frame_crc_init_t;
00295
00296 /*
      ============================================================================
00297  * Private Helper Functions
00298  *
      ============================================================================
```

```
00299 */
00300
00301 /**
00302  * @brief Write uint32 in little-endian format (for CRC-32)
00303  *
00304  *
00305  */
00306 static rx_err_t internal_write_le32(uint8_t* buf, const uint32_t buf_len, const uint32_t val)
00307 {
00308   /* Pre-conditions: buf is always a subslice of output buffer (never nullptr);
00309    * buf_len >= k_frame_crc_size is guaranteed by the caller's frame-size calculation. */
00310   (void)buf_len; /* suppress unused warning in non-debug builds */
00311
00312   buf[k_le32_byte_0] = (uint8_t)(val & k_rx_byte_mask);
00313   buf[k_le32_byte_1] = (uint8_t)((val » k_shift_byte_1) & k_rx_byte_mask);
00314   buf[k_le32_byte_2] = (uint8_t)((val » k_shift_byte_2) & k_rx_byte_mask);
00315   buf[k_le32_byte_3] = (uint8_t)((val » k_shift_byte_3) & k_rx_byte_mask);
00316   return k_rx_ok;
00317 }
00318
00319 /**
00320  * @brief Read uint32 in little-endian format (for CRC-32)
00321  *
00322  *
00323  */
00324 static rx_err_t internal_read_le32(const uint8_t* buf, const uint32_t buf_len, uint32_t* out_val)
00325 {
00326   /* Pre-conditions: buf and out_val are always valid pointers from the caller;
00327    * buf_len >= k_frame_crc_size guaranteed by the caller's bounds check. */
00328   (void)buf_len; /* suppress unused warning in non-debug builds */
00329
00330   *out_val = (uint32_t)buf[k_le32_byte_0] | ((uint32_t)buf[k_le32_byte_1] « k_shift_byte_1) |
00331              ((uint32_t)buf[k_le32_byte_2] « k_shift_byte_2) |
00332              ((uint32_t)buf[k_le32_byte_3] « k_shift_byte_3);
00333   return k_rx_ok;
00334 }
00335
00336 /**
00337  * @brief Decode frame header from raw data buffer
00338  *
00339  * Extracts and validates frame header fields (sync word, sequence, length, type, flags)
00340  * from the beginning of a raw data buffer. Verifies the sync word matches expected value
00341  * and validates payload length is within bounds. Outputs the offset where payload data
00342  * begins for subsequent processing.
00343  *
00344  *
00345  */
00346 static rx_err_t internal_decode_header(const uint8_t* data,
00347                                        const uint32_t data_len,
00348                                        rx_frame_t*    frame,
00349                                        uint32_t*      offset_out)
00350 {
00351   /* Pre-conditions: data, frame, and offset_out are always valid non-null pointers;
00352    * validated by the public callers before forwarding to this internal function. */
00353
00354   if (data_len < k_frame_min_size) {
00355     return k_rx_err_invalid_size;
00356   }
00357
00358   uint32_t offset    = k_frame_offset_start;
00359   uint16_t sync_word = rx_frame_read_le16(&data[offset]);
00360   if (sync_word != k_frame_sync_word) {
00361     return k_rx_err_protocol_error;
00362   }
00363   offset += k_frame_sync_size;
00364
00365   frame->header.sequence = rx_frame_read_le16(&data[offset]);
00366   offset += k_frame_seq_size;
00367
00368   frame->header.length = rx_frame_read_le16(&data[offset]);
00369   offset += k_frame_len_size;
00370
00371   if (frame->header.length > k_frame_max_payload) {
00372     return k_rx_err_invalid_size;
00373   }
00374
00375   uint32_t expected_size = rx_frame_encoded_size(frame->header.length);
00376   if (data_len < expected_size) {
00377     return k_rx_err_invalid_size;
00378   }
00379
00380   frame->header.type = data[offset];
00381   offset += k_frame_type_size;
00382
00383   frame->header.flags = data[offset];
00384   offset += k_frame_flags_size;
00385
```

```
00386   /* offset_out is always non-null (caller guarantees); assign unconditionally. */
00387   *offset_out = offset;
00388
00389   return k_rx_ok;
00390 }
00391
00392 /**
00393  * @brief Verify CRC-32 checksum at specified offset in data buffer
00394  *
00395  * Reads the CRC-32 value stored at the specified offset in the buffer, calculates
00396  * the expected CRC over the data preceding it, and compares for match. Uses IEEE 802.3
00397  * CRC-32 polynomial (0x04C11DB7), compatible with Go's crc32.ChecksumIEEE(). On success,
00398  * outputs the received CRC value for caller inspection.
00399  *
00400  *
00401  */
00402 static rx_err_t
00403 internal_verify_crc(const uint8_t* data, uint32_t data_len, uint32_t offset, uint32_t* crc_out)
00404 {
00405   /* Pre-conditions: data and crc_out are valid non-null pointers (caller validated);
00406    * data_len >= k_frame_min_size and offset+CRC size fits in buffer (caller validated). */
00407   uint32_t received_crc = 0;
00408   /* internal_read_le32 always returns k_rx_ok when preconditions are met. */
00409   (void)internal_read_le32(&data[offset], data_len - offset, &received_crc);
00410   uint32_t calculated_crc = k_frame_crc32_init;
00411   /* rx_crc32_ieee always returns k_rx_ok for non-null data with valid length. */
00412   (void)rx_crc32_ieee(data, offset, &calculated_crc);
00413
00414   if (received_crc != calculated_crc) {
00415     return k_rx_err_crc_mismatch;
00416   }
00417
00418   *crc_out = received_crc;
00419
00420   return k_rx_ok;
00421 }
00422
00423 /**
00424  * @brief Scan a byte buffer for the frame sync word with a fixed upper bound
00425  *
00426  * @details
00427  * Performs a linear scan of the buffer starting at offset 1 (offset 0 is
00428  * presumed to have already failed sync check) looking for the 2-byte sync
00429  * pattern k_frame_sync_word in little-endian wire encoding (first byte 0xAA,
00430  * second byte 0x55). The scan is bounded at k_frame_max_scan_bytes to satisfy
00431  * NASA Rule 2 (fixed loop upper-bounds) and prevent infinite consumption of
00432  * a permanently corrupt stream.
00433  *
00434  * The function stops and reports success at the first occurrence of the sync
00435  * pattern. If no pattern is found within the window, k_rx_err_protocol_error
00436  * is returned.
00437  *
00438  * Algorithm:
00439  * 1. Validate preconditions (non-null pointers, minimum buffer size).
00440  * 2. Compute scan limit = min(data_len - k_frame_sync_size, k_frame_max_scan_bytes).
00441  * 3. Iterate i from 1 to scan_limit inclusive:
00442  *    a. Read 16-bit LE value at data[i].
00443  *    b. If it matches k_frame_sync_word, write i to offset_out and return k_rx_ok.
00444  * 4. Return k_rx_err_protocol_error if no match found.
00445  *
00446  *
00447  *
00448  * @pre data must point to a readable buffer of at least data_len bytes
00449  * @pre data_len must be >= k_frame_sync_size (2)
00450  * @post On k_rx_ok, offset_out is in range [1, min(data_len - k_frame_sync_size, k_frame_max_scan_bytes)]
00451  * @post On error, offset_out is unchanged
00452  *
00453  * @note Not thread-safe. Caller must ensure exclusive access to data buffer.
00454  * @note The loop bound is min(data_len - k_frame_sync_size, k_frame_max_scan_bytes), which is
00455  *       statically bounded by k_frame_max_scan_bytes = 1036 (NASA Rule 2).
00456  *
00457  * @par Performance:
00458  * O(N) where N <= k_frame_max_scan_bytes. Each iteration performs one 16-bit
00459  * LE read and one comparison. Worst case: 1036 iterations (no sync found).
00460  *
00461  * @par Example:
00462  * @code{.c}
00463  * // Buffer where offset k_frame_offset_start failed sync; sync is at index 1
00464  * uint8_t data[] = {0x00, 0xAA, 0x55, 0x01, 0x00};
00465  * uint32_t offset = k_frame_offset_start;
00466  * rx_err_t err = internal_find_sync_offset(data, sizeof(data), &offset);
00467  * if (err == k_rx_ok) {
00468  *     // offset == 1: decode from data[offset]
00469  * } else {
00470  *     // k_rx_err_protocol_error: no sync found within scan window
00471  * }
00472  * @endcode
```

```
00473  *
00474  * @see rx_frame_decode_with_resync() Public API that calls this function
00475  * @see k_frame_sync_word Sync word value (0x55AA)
00476  * @see k_frame_max_scan_bytes Maximum scan window (1036 bytes)
00477  *
00478  * @since Version 1.0.0
00479  */
00480 static rx_err_t
00481 internal_find_sync_offset(const uint8_t* data, const uint32_t data_len, uint32_t* offset_out)
00482 {
00483   /* Pre-conditions: data and offset_out are valid non-null pointers (caller validated);
00484    * data_len >= k_frame_sync_size guaranteed by the caller's size check. */
00485
00486   /* Bound scan to at most k_frame_max_scan_bytes positions beyond offset 0 */
00487   uint32_t scan_limit = data_len - k_frame_sync_size;
00488   if (scan_limit > k_frame_max_scan_bytes) {
00489     scan_limit = k_frame_max_scan_bytes;
00490   }
00491
00492   /* Scan starting at k_frame_scan_start_offset; offset 0 already failed sync check */
00493   for (uint32_t i = k_frame_scan_start_offset; i <= scan_limit; i++) {
00494     const uint16_t candidate = rx_frame_read_le16(&data[i]);
00495     if (candidate == k_frame_sync_word) {
00496       *offset_out = i;
00497       return k_rx_ok;
00498     }
00499   }
00500
00501   return k_rx_err_protocol_error;
00502 }
00503
00504 /*
      ============================================================================
00505  * Encoder API Implementation
00506  *
      ============================================================================
00507  */
00508
00509 /**
00510  * @brief Initialize a frame encoder instance
00511  *
00512  * Prepares the encoder for use by setting the initialized flag and verifying
00513  * the initialization completed successfully (redundant validation per Rule 5).
00514  *
00515  *
00516  */
00517 rx_err_t rx_frame_encoder_init(rx_frame_encoder_t* enc)
00518 {
00519   if (enc == nullptr) {
00520     return k_rx_err_invalid_arg;
00521   }
00522
00523   enc->initialized = k_state_initialized;
00524   /* Post-condition: enc->initialized == k_state_initialized is guaranteed by the
00525    * direct assignment above. */
00526   return k_rx_ok;
00527 }
00528
00529 /**
00530  * @brief Deinitialize a frame encoder instance
00531  *
00532  * Marks the encoder as uninitialized, preventing further use.
00533  * Verifies deinitialization completed successfully (redundant validation per Rule 5).
00534  *
00535  *
00536  */
00537 rx_err_t rx_frame_encoder_deinit(rx_frame_encoder_t* enc)
00538 {
00539   if (enc == nullptr) {
00540     return k_rx_err_invalid_arg;
00541   }
00542
00543   enc->initialized = k_state_uninitialized;
00544   /* Post-condition: enc->initialized == k_state_uninitialized is guaranteed by the
00545    * direct assignment above. */
00546   return k_rx_ok;
00547 }
00548
00549 rx_err_t rx_frame_encode(const rx_frame_encoder_t* enc,
00550                          const rx_frame_t*         frame,
00551                          uint8_t*                  output,
00552                          uint32_t*                 output_len)
00553 {
00554   if (enc == nullptr || frame == nullptr || output == nullptr || output_len == nullptr) {
00555     return k_rx_err_invalid_arg;
00556   }
00557
```

```
00558   if (enc->initialized != k_state_initialized) {
00559     return k_rx_err_invalid_state;
00560   }
00561
00562   /* Validate payload size */
00563   if (frame->header.length > k_frame_max_payload) {
00564     return k_rx_err_invalid_size;
00565   }
00566
00567   /* Calculate total frame size */
00568   uint32_t frame_size = rx_frame_encoded_size(frame->header.length);
00569   uint32_t offset     = k_frame_offset_start;
00570
00571   /* Write SYNC word (little-endian: wire bytes 0xAA, 0x55) */
00572   rx_frame_write_le16(&output[offset], k_frame_sync_word);
00573   offset += k_frame_sync_size;
00574
00575   /* Write SEQ (little-endian) */
00576   rx_frame_write_le16(&output[offset], frame->header.sequence);
00577   offset += k_frame_seq_size;
00578
00579   /* Write LEN (little-endian) */
00580   rx_frame_write_le16(&output[offset], frame->header.length);
00581   offset += k_frame_len_size;
00582
00583   /* Write TYPE (1 byte) */
00584   output[offset] = frame->header.type;
00585   offset += k_frame_type_size;
00586
00587   /* Write FLAGS (1 byte) */
00588   output[offset] = frame->header.flags;
00589   offset += k_frame_flags_size;
00590
00591   /* Write PAYLOAD */
00592   if (frame->header.length > 0) {
00593     for (uint32_t i = 0U; i < (uint32_t)frame->header.length; i++) {
00594       output[offset + i] = frame->payload[i];
00595     }
00596     offset += frame->header.length;
00597   }
00598
00599   /* Calculate CRC-32 over SYNC + Header + Payload (IEEE 802.3 polynomial).
00600    * rx_crc32_ieee always returns k_rx_ok for non-null buffer with valid length. */
00601   uint32_t crc = k_frame_crc32_init;
00602   (void)rx_crc32_ieee(output, offset, &crc);
00603
00604   /* Write CRC-32 (little-endian to match IEEE 802.3 LSB-first order).
00605    * internal_write_le32 always returns k_rx_ok when preconditions are met. */
00606   (void)internal_write_le32(&output[offset], frame_size - offset, crc);
00607
00608   *output_len = frame_size;
00609   return k_rx_ok;
00610 }
00611
00612 /*
      ============================================================================
00613  * Decoder API Implementation
00614  *
      ============================================================================
00615  */
00616
00617 /**
00618  * @brief Initialize a frame decoder instance
00619  *
00620  * Prepares the decoder for use by setting the initialized flag and verifying
00621  * the initialization completed successfully (redundant validation per Rule 5).
00622  *
00623  *
00624  */
00625 rx_err_t rx_frame_decoder_init(rx_frame_decoder_t* dec)
00626 {
00627   if (dec == nullptr) {
00628     return k_rx_err_invalid_arg;
00629   }
00630
00631   dec->initialized = k_state_initialized;
00632   /* Post-condition: dec->initialized == k_state_initialized is guaranteed by the
00633    * direct assignment above. */
00634   return k_rx_ok;
00635 }
00636
00637 /**
00638  * @brief Deinitialize a frame decoder instance
00639  *
00640  * Marks the decoder as uninitialized, preventing further use.
00641  * Verifies deinitialization completed successfully (redundant validation per Rule 5).
00642  *
```

```
00643  *
00644  */
00645 rx_err_t rx_frame_decoder_deinit(rx_frame_decoder_t* dec)
00646 {
00647   if (dec == nullptr) {
00648     return k_rx_err_invalid_arg;
00649   }
00650
00651   dec->initialized = k_state_uninitialized;
00652   /* Post-condition: dec->initialized == k_state_uninitialized is guaranteed by the
00653    * direct assignment above. */
00654   return k_rx_ok;
00655 }
00656
00657 /**
00658  * @brief Decode a frame from raw data
00659  *
00660  * Validates the decoder state, extracts the frame header, copies the payload,
00661  * and verifies the CRC-32 checksum.
00662  *
00663  *
00664  *
00665  * @note This function performs validation at multiple points:
00666  *       - Null pointer checks on all parameters
00667  *       - Decoder initialization check
00668  *       - Header parsing via internal_decode_header()
00669  *       - CRC verification via internal_verify_crc() (may surface CRC backend errors)
00670  */
00671 rx_err_t rx_frame_decode(const rx_frame_decoder_t* dec,
00672                          const uint8_t*            data,
00673                          const uint32_t            data_len,
00674                          rx_frame_t*               frame)
00675 {
00676   if (dec == nullptr || data == nullptr || frame == nullptr) {
00677     return k_rx_err_invalid_arg;
00678   }
00679
00680   if (dec->initialized != k_state_initialized) {
00681     return k_rx_err_invalid_state;
00682   }
00683
00684   uint32_t offset;
00685   rx_err_t err = internal_decode_header(data, data_len, frame, &offset);
00686   if (err != k_rx_ok) {
00687     return err;
00688   }
00689
00690   /* Read PAYLOAD */
00691   if (frame->header.length > 0) {
00692     for (uint32_t i = 0U; i < (uint32_t)frame->header.length; i++) {
00693       frame->payload[i] = data[offset + i];
00694     }
00695     offset += frame->header.length;
00696   }
00697
00698   err = internal_verify_crc(data, data_len, offset, &frame->crc);
00699   if (err != k_rx_ok) {
00700     return err;
00701   }
00702   return k_rx_ok;
00703 }
00704
00705 /**
00706  * @brief Decode a frame from raw data with bounded sync-word resynchronization
00707  *
00708  * @details
00709  * Provides stream recovery when the byte stream has become misaligned due to
00710  * a dropped or inserted byte. On sync mismatch, scans forward up to
00711  * k_frame_max_scan_bytes positions for the next occurrence of the sync word,
00712  * discards the leading bytes, and reattempts decoding from that position.
00713  *
00714  * This function is safe for repeated calls on a stream: the caller must advance
00715  * its stream read pointer by *bytes_discarded_out on k_rx_ok and also on
00716  * k_rx_err_crc_mismatch (any case where bytes_discarded_out is set) to avoid
00717  * re-processing discarded bytes in subsequent calls.
00718  *
00719  * Recovery algorithm:
00720  * 1. Attempt rx_frame_decode() on data[0..data_len-1].
00721  * 2. If result is not k_rx_err_protocol_error, return immediately.
00722  * 3. Call internal_find_sync_offset() to locate next sync word in data[1..].
00723  * 4. If not found, return k_rx_err_protocol_error with *bytes_discarded_out = 0.
00724  * 5. Decode from data[sync_offset..data_len-1] using the trimmed sub-buffer.
00725  * 6. Write sync_offset to *bytes_discarded_out and return result.
00726  *
00727  *
00728  *
00729  * @pre dec must be initialized via rx_frame_decoder_init()
```

```
00730  * @pre data must point to a buffer of at least data_len bytes
00731  * @pre frame must point to a valid rx_frame_t storage location (not NULL)
00732  * @pre bytes_discarded_out must point to a valid uint32_t storage location (not NULL)
00733  * @post On k_rx_ok, *bytes_discarded_out == bytes skipped to reach sync word
00734  * @post On k_rx_ok, frame contains a fully verified decoded frame
00735  * @post On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer
00736  *
00737  * @note Parameter order follows the canonical convention: decoder input -> data input ->
00738  *       frame output -> diagnostic output (matching rx_frame_decode()).
00739  * @note Not thread-safe. Caller must synchronize access.
00740  * @note The scan is bounded at k_frame_max_scan_bytes (NASA Rule 2).
00741  * @note Log *bytes_discarded_out > 0 as a stream-health diagnostic warning.
00742  *
00743  * @warning Caller must advance stream read pointer by *bytes_discarded_out after any call
00744  *          that returns k_rx_ok or k_rx_err_crc_mismatch to avoid infinite re-scan;
00745  *          *bytes_discarded_out is unmodified only when k_rx_err_invalid_arg is returned.
00746  *
00747  * @par Example:
00748  * @code{.c}
00749  * uint32_t discarded = k_bytes_discarded_none;
00750  * rx_err_t err = rx_frame_decode_with_resync(dec, data, data_len, &frame, &discarded);
00751  * if (err == k_rx_ok) {
00752  *     // Advance stream read pointer past any discarded bytes
00753  *     stream_read_ptr += discarded;
00754  * } else if (err == k_rx_err_crc_mismatch) {
00755  *     // Sync was found but CRC failed -- still advance to avoid infinite re-scan
00756  *     stream_read_ptr += discarded;
00757  *     rx_log_error(TAG, "sync found but CRC failed");
00758  * } else {
00759  *     rx_log_error(TAG, "no sync word within scan window");
00760  * }
00761  * @endcode
00762  *
00763  * @see rx_frame_decode() Simple decode without stream recovery
00764  * @see internal_find_sync_offset() Bounded sync word scanner
00765  * @see k_frame_max_scan_bytes Scan window upper bound
00766  *
00767  * @since Version 1.0.0
00768  */
00769 rx_err_t rx_frame_decode_with_resync(const rx_frame_decoder_t* dec,
00770                                      const uint8_t*            data,
00771                                      const uint32_t            data_len,
00772                                      rx_frame_t*               frame,
00773                                      uint32_t*                 bytes_discarded_out)
00774 {
00775   if (dec == nullptr || data == nullptr || frame == nullptr || bytes_discarded_out == nullptr) {
00776     return k_rx_err_invalid_arg;
00777   }
00778
00779   *bytes_discarded_out = k_bytes_discarded_none;
00780
00781   /* Fast path: attempt aligned decode first */
00782   rx_err_t err = rx_frame_decode(dec, data, data_len, frame);
00783   if (err != k_rx_err_protocol_error) {
00784     /* Either success, or an error other than sync mismatch (CRC, size, etc.) */
00785     return err;
00786   }
00787
00788   /* Sync word not at offset 0: scan forward for next sync word */
00789   uint32_t sync_offset = (uint32_t)k_frame_offset_start;
00790   err                  = internal_find_sync_offset(data, data_len, &sync_offset);
00791   if (err != k_rx_ok) {
00792     /* No sync word found within bounded window */
00793     return k_rx_err_protocol_error;
00794   }
00795
00796   /* Report discarded bytes now: caller needs this to advance past the sync
00797    * even if the subsequent decode fails (e.g. CRC mismatch at new offset) */
00798   *bytes_discarded_out = sync_offset;
00799
00800   /* Reattempt decode from the discovered sync offset */
00801   err = rx_frame_decode(dec, &data[sync_offset], data_len - sync_offset, frame);
00802
00803   return err;
00804 }
00805
00806 /*
      ============================================================================
00807  * Utility Functions
00808  *
      ============================================================================
00809  */
00810
00811 /**
00812  * @brief Create an ACK (acknowledgment) frame
00813  *
00814  * Constructs a frame with type=ACK, empty payload, and the specified sequence number.
```

```
00815  * The ACK response frames are used by the receiver to confirm successful reception
00816  * of a command or data frame from the sender.
00817  *
00818  *
00819  */
00820 rx_err_t rx_frame_create_ack(rx_frame_t* frame, const uint16_t sequence)
00821 {
00822   if (frame == nullptr) {
00823     return k_rx_err_invalid_arg;
00824   }
00825
00826   *frame                = (rx_frame_t){};
00827   frame->header.sequence = sequence;
00828   frame->header.length   = 0;
00829   frame->header.type     = k_frame_type_ack;
00830   frame->header.flags    = k_frame_flag_none;
00831
00832   /* Post-condition: type == k_frame_type_ack guaranteed by the direct assignment above. */
00833   return k_rx_ok;
00834 }
00835
00836 /**
00837  * @brief Create a NACK (negative acknowledgment) frame
00838  *
00839  * Constructs a frame with type=NACK, empty payload, the specified sequence number,
00840  * and error/status flags. NACK frames are sent by the receiver to indicate that a
00841  * frame was not processed successfully due to an error condition (specified in flags).
00842  *
00843  *
00844  */
00845 rx_err_t rx_frame_create_nack(rx_frame_t* frame, const uint16_t sequence, uint8_t flags)
00846 {
00847   if (frame == nullptr) {
00848     return k_rx_err_invalid_arg;
00849   }
00850
00851   *frame                = (rx_frame_t){};
00852   frame->header.sequence = sequence;
00853   frame->header.length   = 0;
00854   frame->header.type     = k_frame_type_nack;
00855   frame->header.flags    = flags;
00856
00857   /* Post-condition: type == k_frame_type_nack guaranteed by the direct assignment above. */
00858   return k_rx_ok;
00859 }
00860
00861 /**
00862  * @brief Create a PING keepalive frame
00863  *
00864  * @details
00865  * Initializes frame as a PING request for connection liveness checks.
00866  * Receiver should respond with a PONG frame echoing the payload.
00867  * Payload is optional; a 4-byte LE counter is typical.
00868  *
00869  *
00870  *
00871  * @pre frame !=nullptr
00872  * @pre payload != NULL when payload_len > 0
00873  * @post frame->header.type == k_frame_type_ping
00874  * @post frame->header.length == payload_len
00875  *
00876  * @note Stateless; safe to call from any context.
00877  * @see rx_frame_create_pong() Corresponding response creator
00878  * @since Version 1.0.0
00879  */
00880 rx_err_t rx_frame_create_ping(rx_frame_t*    frame,
00881                               const uint16_t sequence,
00882                               const uint8_t* payload,
00883                               const uint32_t payload_len)
00884 {
00885   if (frame == nullptr) {
00886     return k_rx_err_invalid_arg;
00887   }
00888
00889   if (payload_len > 0 && payload == nullptr) {
00890     return k_rx_err_invalid_arg;
00891   }
00892
00893   if (payload_len > k_frame_max_payload) {
00894     return k_rx_err_invalid_size;
00895   }
00896
00897   *frame                = (rx_frame_t){};
00898   frame->header.sequence = sequence;
00899   frame->header.length   = (uint16_t)payload_len;
00900   frame->header.type     = k_frame_type_ping;
00901   frame->header.flags    = k_frame_flag_none;
```

```
00902
00903   if (payload_len > 0) {
00904     for (uint32_t i = 0U; i < payload_len; i++) {
00905       frame->payload[i] = payload[i];
00906     }
00907   }
00908
00909   /* Post-condition: type == k_frame_type_ping guaranteed by the direct assignment above. */
00910   return k_rx_ok;
00911 }
00912
00913 /**
00914  * @brief Create a PONG response frame
00915  *
00916  * @details
00917  * Initializes frame as a PONG response to a received PING.
00918  * Echoes the PING payload so the sender can validate round-trip data.
00919  *
00920  *
00921  *
00922  * @pre frame !=nullptr
00923  * @pre payload != NULL when payload_len > 0
00924  * @post frame->header.type == k_frame_type_pong
00925  * @post frame->header.length == payload_len
00926  *
00927  * @note Stateless; safe to call from any context.
00928  * @see rx_frame_create_ping() Corresponding request creator
00929  * @since Version 1.0.0
00930  */
00931 rx_err_t rx_frame_create_pong(rx_frame_t*    frame,
00932                               const uint16_t sequence,
00933                               const uint8_t* payload,
00934                               const uint32_t payload_len)
00935 {
00936   if (frame == nullptr) {
00937     return k_rx_err_invalid_arg;
00938   }
00939
00940   if (payload_len > 0 && payload == nullptr) {
00941     return k_rx_err_invalid_arg;
00942   }
00943
00944   if (payload_len > k_frame_max_payload) {
00945     return k_rx_err_invalid_size;
00946   }
00947
00948   *frame                = (rx_frame_t){};
00949   frame->header.sequence = sequence;
00950   frame->header.length   = (uint16_t)payload_len;
00951   frame->header.type     = k_frame_type_pong;
00952   frame->header.flags    = k_frame_flag_none;
00953
00954   if (payload_len > 0) {
00955     for (uint32_t i = 0U; i < payload_len; i++) {
00956       frame->payload[i] = payload[i];
00957     }
00958   }
00959
00960   /* Post-condition: type == k_frame_type_pong guaranteed by the direct assignment above. */
00961   return k_rx_ok;
00962 }
00963
00964 /**
00965  * @brief Create a RESET request frame
00966  *
00967  * @details
00968  * Initializes frame as a RESET request that asks the peer to reset
00969  * communication state (sequence numbers, buffers). The receiver
00970  * should respond with a RESET_ACK frame. Payload is always empty.
00971  *
00972  *
00973  *
00974  * @pre frame !=nullptr
00975  * @pre Caller has determined that a reset is necessary
00976  * @post frame->header.type == k_frame_type_reset
00977  * @post frame->header.length == 0
00978  *
00979  * @note Stateless; safe to call from any context.
00980  * @warning Reset clears all sequence number state on both sides.
00981  * @see rx_frame_create_reset_ack() Corresponding acknowledgment creator
00982  * @since Version 1.0.0
00983  */
00984 rx_err_t rx_frame_create_reset(rx_frame_t* frame, const uint16_t sequence)
00985 {
00986   if (frame == nullptr) {
00987     return k_rx_err_invalid_arg;
00988   }
```

```
00989
00990   *frame                = (rx_frame_t){};
00991   frame->header.sequence = sequence;
00992   frame->header.length   = 0;
00993   frame->header.type     = k_frame_type_reset;
00994   frame->header.flags    = k_frame_flag_none;
00995
00996   /* Post-condition: type == k_frame_type_reset guaranteed by the direct assignment above. */
00997   return k_rx_ok;
00998 }
00999
01000 /**
01001  * @brief Create a RESET_ACK confirmation frame
01002  *
01003  * @details
01004  * Initializes frame as a RESET_ACK response to a received RESET request.
01005  * Confirms that the receiver has completed its reset procedure.
01006  * Payload is always empty; sequence matches the RESET request.
01007  *
01008  *
01009  *
01010  * @pre frame !=nullptr
01011  * @pre A RESET frame was received and processed
01012  * @post frame->header.type == k_frame_type_reset_ack
01013  * @post frame->header.length == 0
01014  *
01015  * @note Stateless; safe to call from any context.
01016  * @see rx_frame_create_reset() Corresponding request creator
01017  * @since Version 1.0.0
01018  */
01019 rx_err_t rx_frame_create_reset_ack(rx_frame_t* frame, const uint16_t sequence)
01020 {
01021   if (frame == nullptr) {
01022     return k_rx_err_invalid_arg;
01023   }
01024
01025   *frame                = (rx_frame_t){};
01026   frame->header.sequence = sequence;
01027   frame->header.length   = 0;
01028   frame->header.type     = k_frame_type_reset_ack;
01029   frame->header.flags    = k_frame_flag_none;
01030
01031   /* Post-condition: type == k_frame_type_reset_ack guaranteed by the direct assignment above. */
01032   return k_rx_ok;
01033 }
```